

Chapter-03

STRUCTURAL, BEHAVIOURAL AND ARCHITECTURAL MODELING (8 HOURS)

- 3.1 Classes Relationship,**
- 3.2 Conceptual Model of UML**
- 3.3. Class Diagram**
- 3.4 Advance Classes**
- 3.5 Advanced Relationship**
- 3.6 Interface**
- 3.7 Object Diagram**
- 3.8 Interactions**
- 3.9 Use Cases**
- 3.10 Use Case Diagrams**
- 3.11 Interaction Diagrams**
- 3.12 Activity Diagram State Chart Diagram**
- 3.13 Component & Components Diagram**
- 3.14 Deployment Diagram**

3.2 Conceptual Model of UML

Conceptual Model of the Unified Modeling Language (UML)

- The Unified Modeling Language (UML) is a standard visual language for describing and modelling software blueprints.
- The UML is more than just a graphical language.
- Stated formally, the UML is for: Visualizing, Specifying, Constructing, and Documenting.
The artifacts of a software-intensive system (particularly systems built using the object-oriented style).

3.2 Conceptual Model of UML

What is an Artifact?

- To put it simply, an artifact is a by-product of software development.
- It's anything that is created so a piece of software can be developed.
- This might include things like data models, diagrams, setup scripts — the list goes on.
- “Artifact” is a pretty broad term when it comes to software development.
- Most pieces of software have a lot of artifacts that are necessary for them to run.
- Some artifacts explain how a piece of software is supposed to work, while others actually allow that program to run.

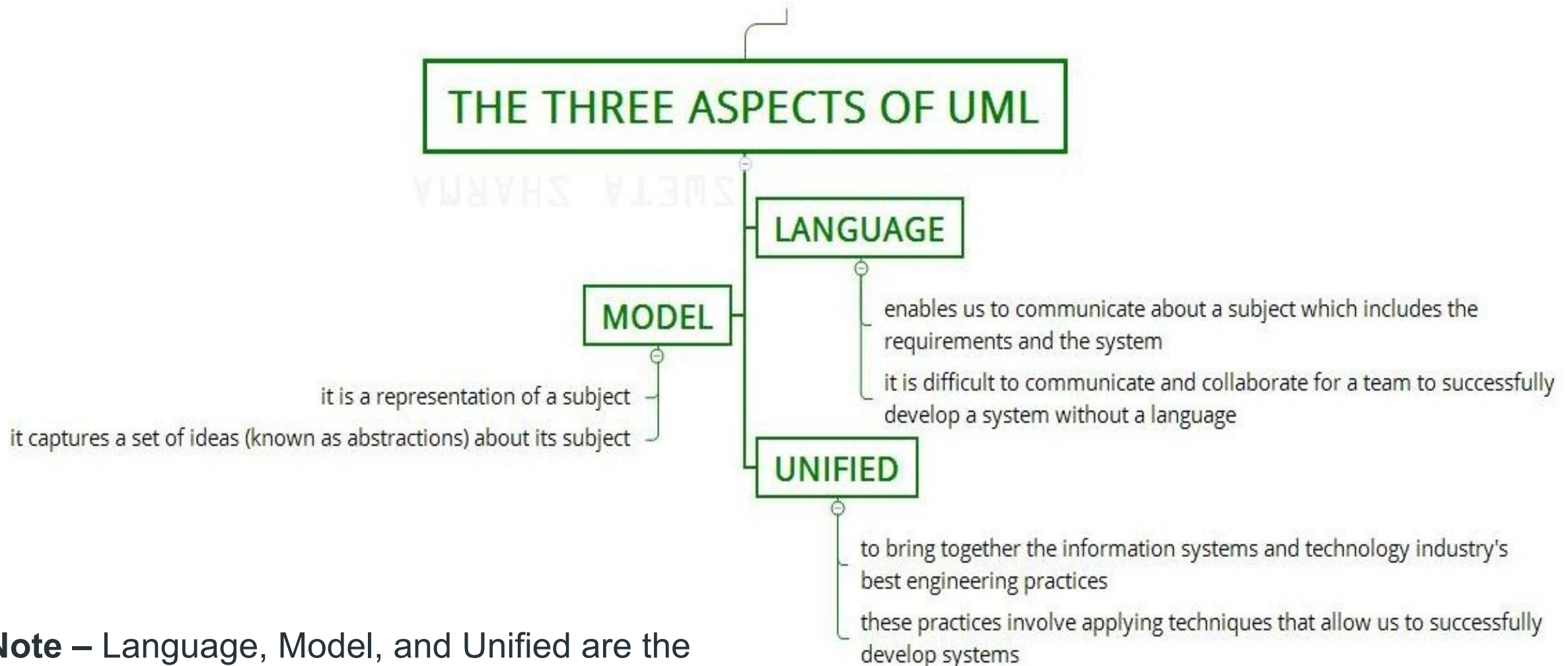
3.2 Conceptual Model of UML

Why is it called an Artifact?

- Think about an archaeological dig site. During an archaeological dig, any man-made object that's dug up is an artifact. These artifacts help us determine what civilization may have been like and help us paint a picture of their society.
- Similarly, digital artifacts are all products of people. They help other developers see the thought process behind what goes into developing a piece of software. This, in turn, helps lead developers to further decisions and gives them a better understanding of how to proceed.

3.2 Conceptual Model of UML

Three Aspects of UML:



Note – Language, Model, and Unified are the important aspect of UML as described in the map above.

3.2 Conceptual Model of UML

1. Language:

- It enables us to communicate about a subject which includes the requirements and the system.
- It is difficult to communicate and collaborate for a team to successfully develop a system without a language.

2. Model:

- It is a representation of a subject.
- It captures a set of ideas (*known as abstractions*) about its subject.

3. Unified:

- It is to bring together the information systems and technology industry's best engineering practices.
- These practices involve applying techniques that allow us to successfully develop systems.

3.2 Conceptual Model of UML

A Conceptual Model:

A conceptual model of the language underlines the three major elements:

- The Building Blocks
- The Rules
- Some Common Mechanisms

3.2 Conceptual Model of UML

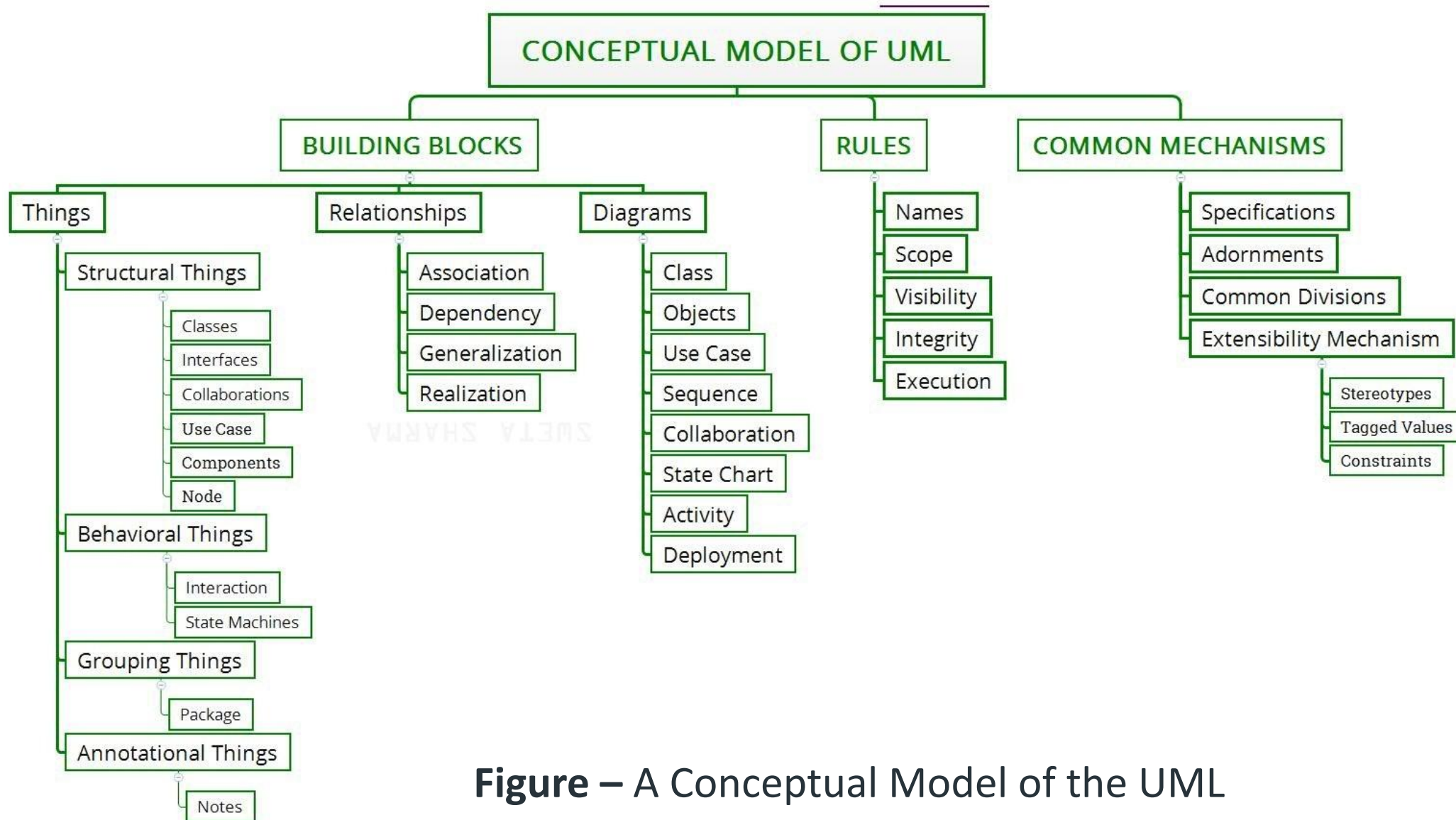


Figure – A Conceptual Model of the UML

3.2 Conceptual Model of UML

Building Blocks:

The vocabulary of the UML encompasses three kinds of building blocks:

I. Things:

Things are the abstractions that are first-class citizens in a model; relationships tie these things together; diagrams group interesting collections of things.

There are 4 kinds of things in the UML:

1. Structural things
2. Behavioral things
3. Grouping things
4. Annotational things

These things are the basic object-oriented building blocks of the UML. We can use them to write well-formed models.

3.2 Conceptual Model of UML

II. Relationships:

There are 4 kinds of relationships in the UML:

1. Dependency
2. Association
3. Generalization
4. Realization

These relationships are the basic relational building blocks of the UML.

III. Diagrams:

It is the graphical presentation of a set of elements.

It is rendered as a connected graph of vertices (things) and arcs (relationships).

3.2 Conceptual Model of UML

1. [Class diagram](#)
2. [Object diagram](#)
3. [Use case diagram](#)
4. [Sequence diagram](#)
5. [Collaboration diagram](#)
6. [State chart diagram](#)
7. [Activity diagram](#)
8. Component diagram
9. Deployment diagram

3.2 Conceptual Model of UML

Rules:

The UML has a number of rules that specify what a well-formed model should look like.

A well-formed model is one that is semantically self-consistent and in harmony with all its related models.

The UML has semantic rules for:

1. **Names** – What you can call things, relationships, and diagrams.
2. **Scope** – The context that gives specific meaning to a name.
3. **Visibility** – How those names can be seen and used by others.
4. **Integrity** – How things properly and consistently relate to one another.
5. **Execution** – What it means to run or simulate a dynamic model.

3.2 Conceptual Model of UML

Common Mechanisms:

The UML is made simpler by the four common mechanisms.

They are as follows:

1. Specifications
2. Adornments
3. Common divisions
4. Extensibility mechanisms

3.2 Conceptual Model of UML

Common Mechanisms:

The UML is made simpler by the four common mechanisms.

They are as follows:

1. Specifications
2. Adornments
3. Common divisions
4. Extensibility mechanisms

3.3 Class Diagram

What is Class in UML Diagram?

- A **Class in UML** diagram is a blueprint used to create an object or set of objects. The Class defines what an object can do.
- It is a template to create various objects and implement their behavior in the system.
- A Class in UML is represented by a rectangle that includes rows with class names, attributes, and operations.

3.3 Class Diagram

What is Class Diagram?

- A **Class Diagram** is a static structure that gives an overview of a software system by displaying classes, attributes, operations, and their relationships between each other.
- This Diagram includes the class name, attributes, and operation in separate designated compartments.
- Class Diagram helps construct the code for the software application development.
- Since it is a collection of classes, interfaces, associations, collaborations, and constraints, it is termed as a structural diagram.
- Class Diagram defines the types of objects in the system and the different types of relationships that exist among them.
- It gives a high-level view of an application. This modeling method can run with almost all Object-Oriented Methods.

3.3 Class Diagram

- A class can refer to another class. A class can have its objects or may inherit from other classes.

3.3 Class Diagram

Purpose of Class Diagrams

- The main purpose of class diagrams is to build a static view of an application.
- It is the only diagram that is widely used for construction, and it can be mapped with object-oriented languages.
- It is one of the most popular UML diagrams.
- Following are the purpose of class diagrams given below:
 1. It analyses and designs a static view of an application.
 2. It describes the major responsibilities of a system.
 3. It is a base for component and deployment diagrams.
 4. It incorporates forward and reverse engineering.

3.3 Class Diagram

Benefits of Class Diagrams

1. It can represent the object model for complex systems.
2. It reduces the maintenance time by providing an overview of how an application is structured before coding.
3. It provides a general schematic of an application for better understanding.
4. It represents a detailed chart by highlighting the desired code, which is to be programmed.
5. It is helpful for the stakeholders and the developers.

3.3 Class Diagram

Vital components of a Class Diagram

- The class diagram is made up of three sections:
 1. **Upper Section:**
 - The upper section encompasses the name of the class.
 - A class is a representation of similar objects that shares the same relationships, attributes, operations, and semantics.
 - Some of the following rules that should be taken into account while representing a class are given below:
 1. Capitalize the initial letter of the class name.
 2. Place the class name in the center of the upper section.
 3. A class name must be written in bold format.
 4. The name of the abstract class should be written in italics format.

3.3 Class Diagram

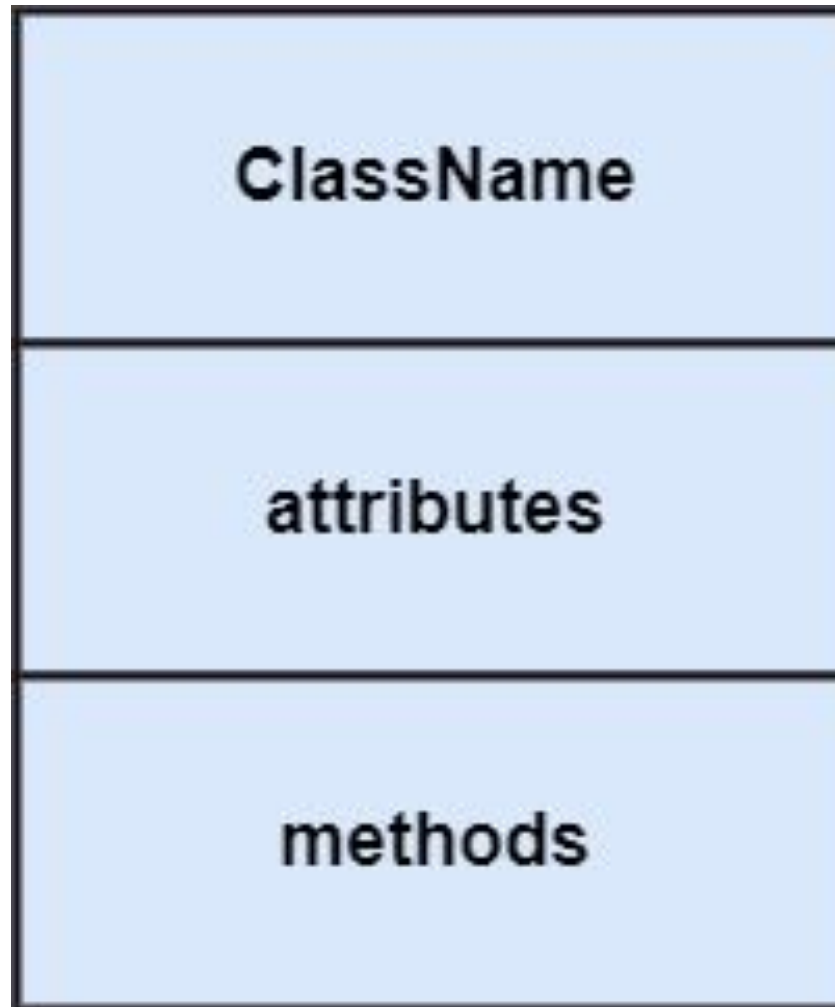
2. Middle Section:

- The middle section constitutes the attributes, which describe the quality of the class.
- The attributes have the following characteristics:
 1. The attributes are written along with its visibility factors, which are public (+), private (-), protected (#), and package (~).
 2. The accessibility of an attribute class is illustrated by the visibility factors.
 3. A meaningful name should be assigned to the attribute, which will explain its usage inside the class.

3. Lower Section:

- The lower section contain methods or operations.
- The methods are represented in the form of a list, where each method is written in a single line.
- It demonstrates how a class interacts with data.

3.3 Class Diagram



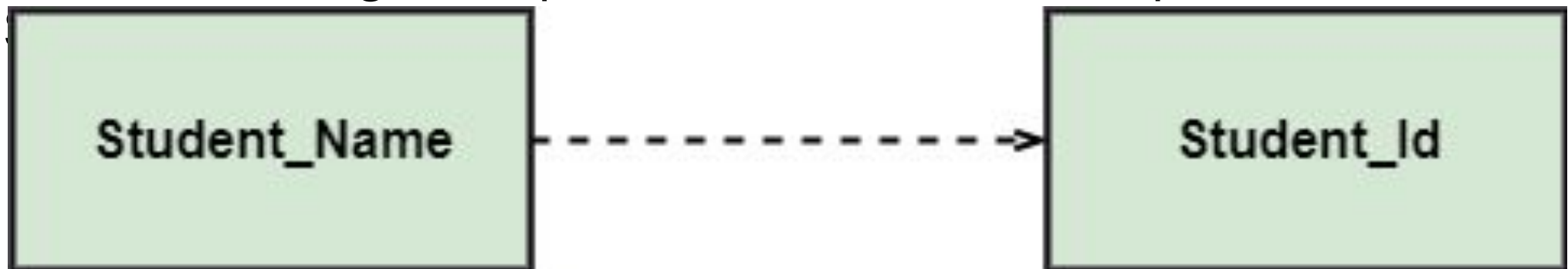
3.3 Class Diagram

Relationships

- In UML, relationships are of three types:

1. **Dependency:**

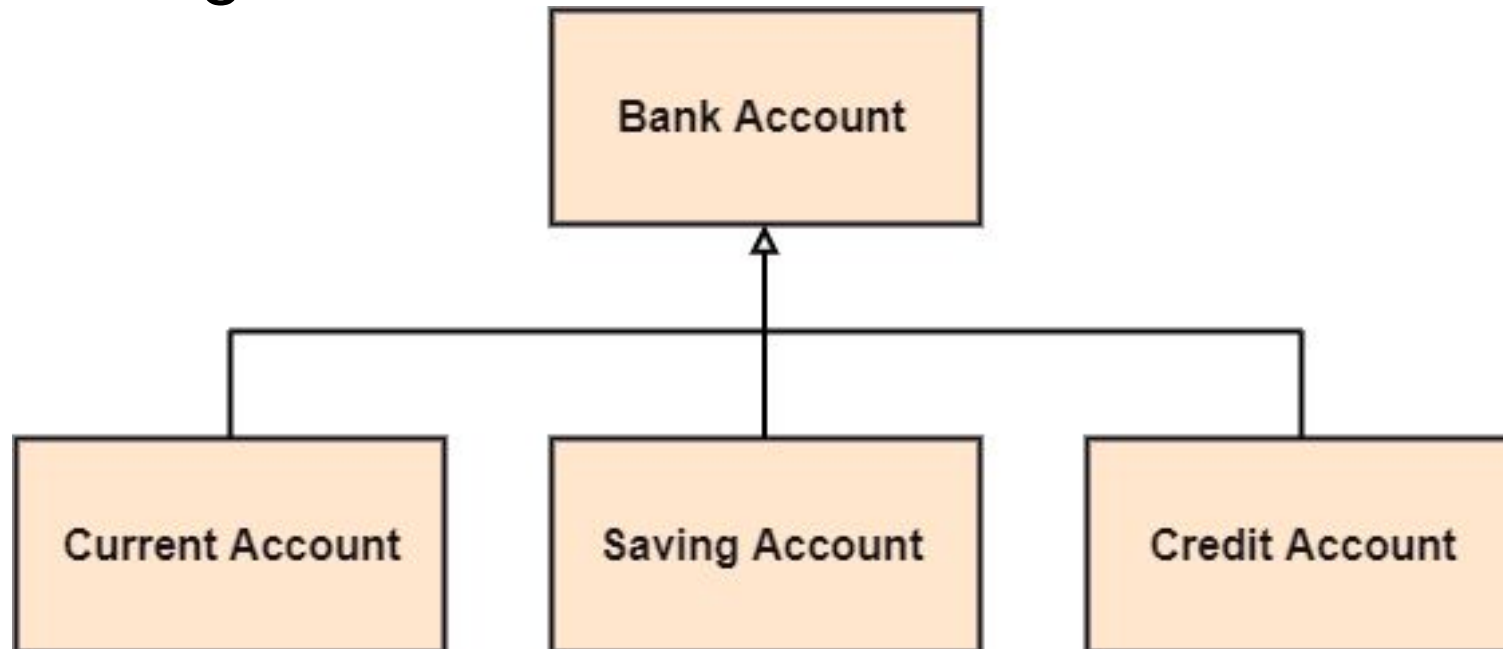
- A dependency is a semantic relationship between two or more classes where a change in one class cause changes in another class.
- It forms a weaker relationship.
- In the following example, Student_Name is dependent on the



3.3 Class Diagram

2. Generalization:

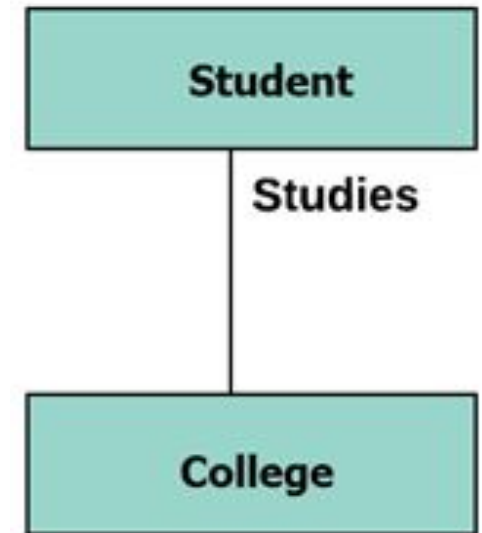
- A generalization is a relationship between a parent class (superclass) and a child class (subclass).
- In this, the child class is inherited from the parent class.
- For example, The Current Account, Saving Account, and Credit Account are the generalized form of Bank Account.



3.3 Class Diagram

3. Association:

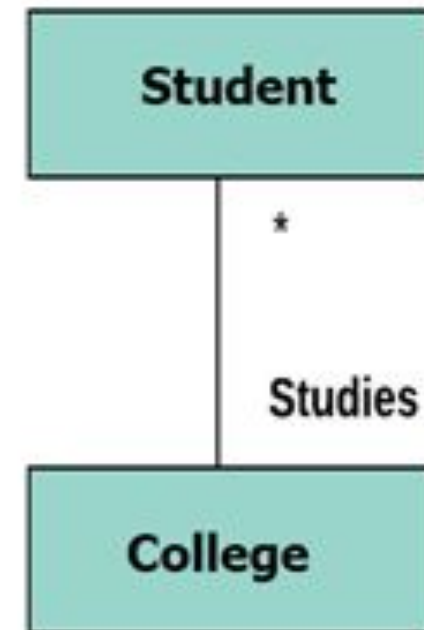
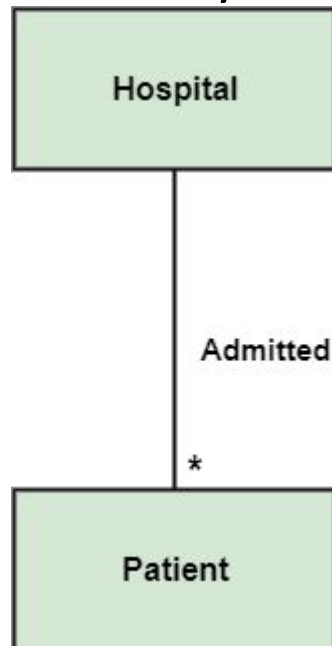
- It describes a static or physical connection between two or more objects.
- It depicts how many objects are there in the relationship.
- For example:
 1. a department is associated with the college.
 2. the relationship between student and college



3.3 Class Diagram

Multiplicity:

- It defines a specific range of allowable instances of attributes.
- In case if a range is not specified, one is considered as a default multiplicity.
- For example:
 1. multiple patients are admitted to one hospital.
 2. Let's say that there are 100 students in one college. The college can have multiple



3.3 Class Diagram

Aggregation:

- An aggregation is a subset of association, which represents has a relationship.
- It is more specific than association.
- It defines a part-whole or part-of relationship.
- In this kind of relationship, the child class can exist independently of its parent class.
- For example:
 1. The company encompasses a number of employees, and even if one employee resigns, the company still exists.
 2. the class college is made up of one or more student. In aggregation, the contained classes are never totally dependent on the lifecycle of the container. Here, the college class will remain even if the student is not available.



3.3 Class Diagram

Composition:

- The composition is a subset of aggregation.
- It portrays the dependency between the parent and its child, which means if one part is deleted, then the other part also gets discarded.
- It represents a whole-part relationship.
- For example:
 1. A contact book consists of multiple contacts, and if you delete the contact book, all the contacts will be lost.
 2. if college is composed of classes student. The college could contain many students, while each student belongs to only one college. So, if college is not functioning all the students also removed.



3.3 Class Diagram

Aggregation vs. Composition

Aggregation

Aggregation indicates a relationship where the child can exist separately from their parent class. Example: Automobile (Parent) and Car (Child). So, If you delete the Automobile, the child Car still exist.

Composition

Composition display relationship where the child will never exist independent of the parent. Example: House (parent) and Room (child). Rooms will never separate into a House.

3.3 Class Diagram

Cardinality

Cardinality is expressed in terms of:

- one to one
- one to many
- many to many



3.3 Class Diagram

Abstract Classes

- In the abstract class, no objects can be a direct entity of the abstract class.
- The abstract class can neither be declared nor be instantiated.
- It is used to find the functionalities across the classes.
- The notation of the abstract class is similar to that of class; the only difference is that the name of the class is written in italics.
- Since it does not involve any implementation for a given function, it is best to use the abstract class with multiple objects.

3.3 Class Diagram

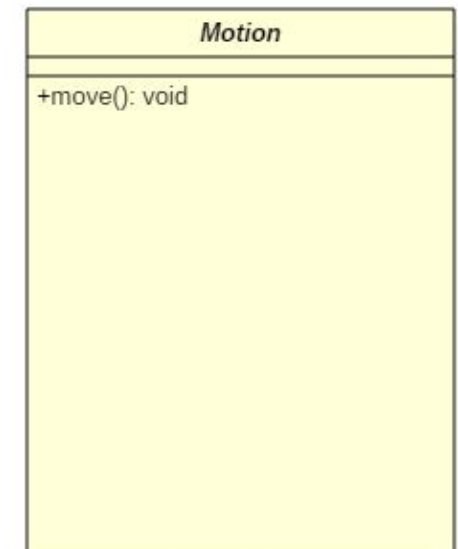
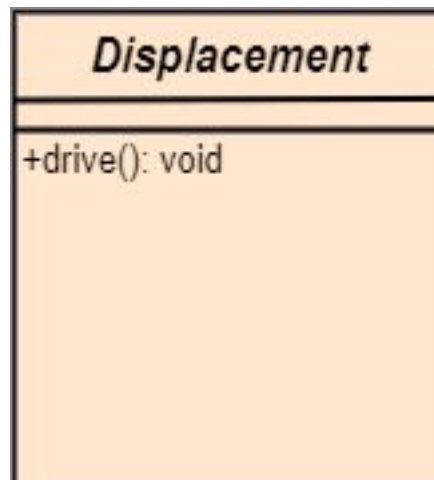
- For e.g.:

- 1.

- Let us assume that we have an abstract class named **displacement** with a method declared inside it, and that method will be called as a **drive ()**.
- Now, this abstract class method can be implemented by any object, for example, car, bike, scooter, cycle, etc.

- 2.

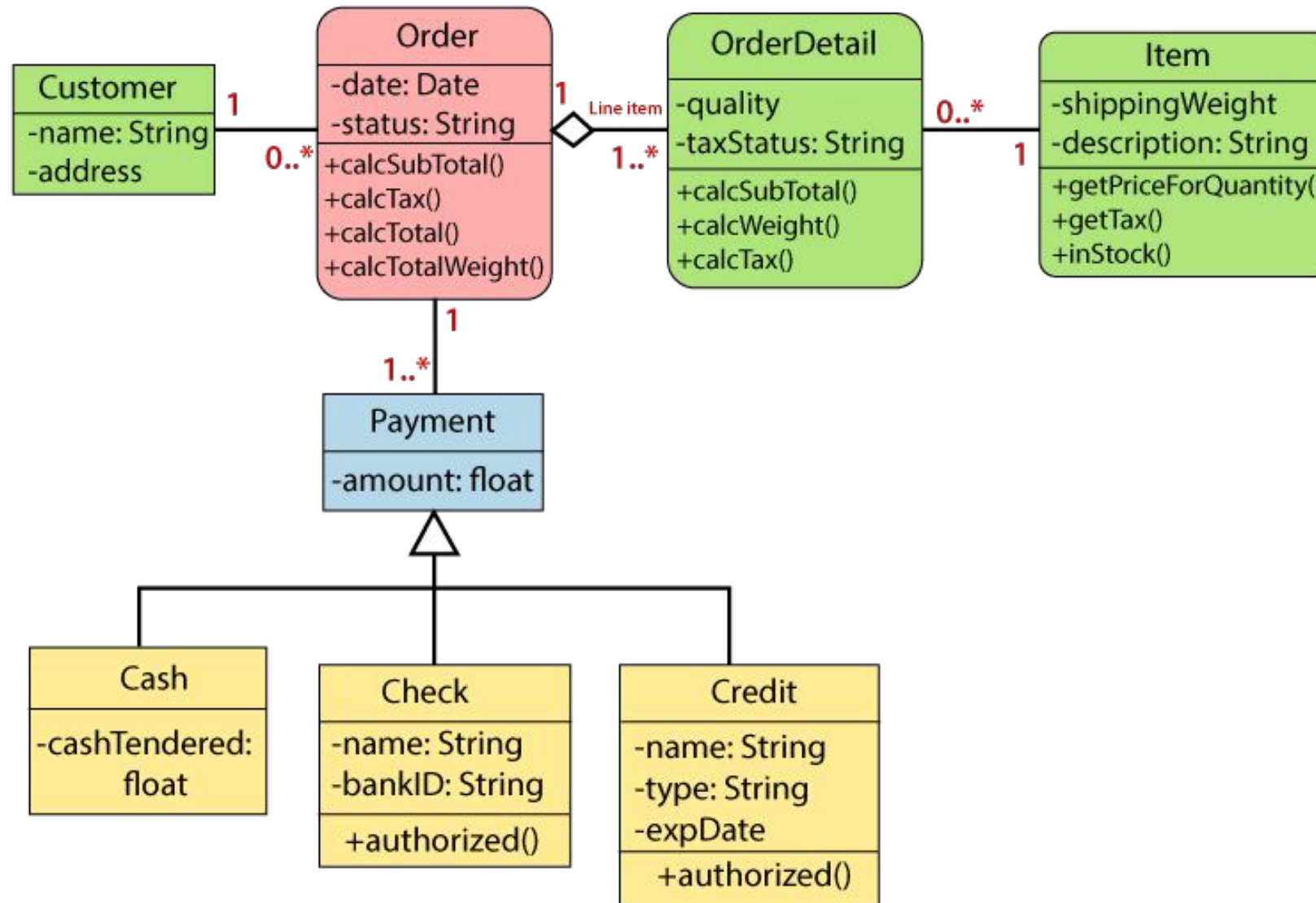
Suppose we have an abstract class called as a motion with a method or an operation declared inside of it. The method declared inside the abstract class is called a **move ()**.



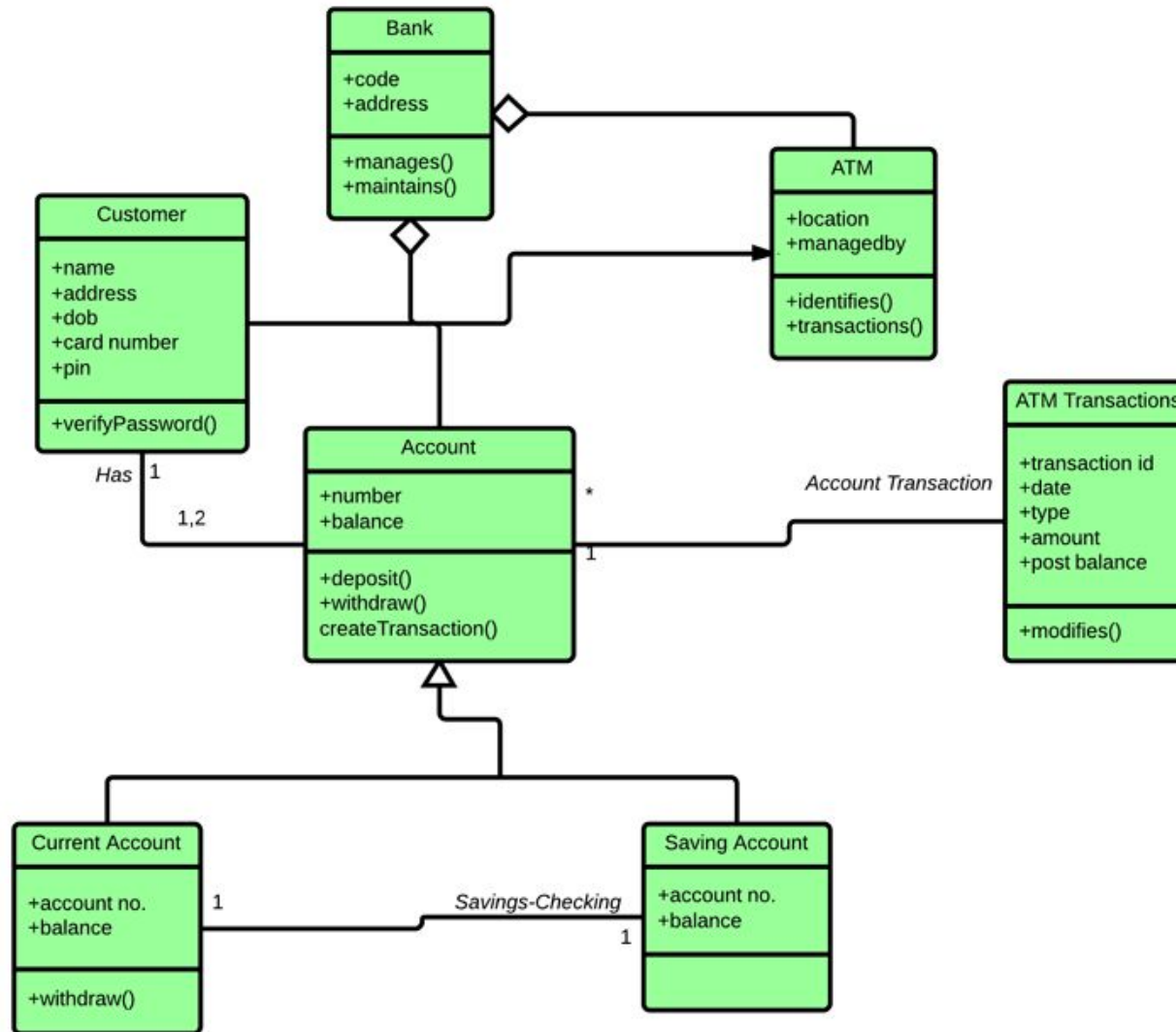
3.3 Class Diagram

- How to draw a Class Diagram?
- The class diagram is used most widely to construct software applications. It not only represents a static view of the system but also all the major aspects of an application. A collection of class diagrams as a whole represents a system.
- Some key points that are needed to keep in mind while drawing a class diagram are given below:
 1. To describe a complete aspect of the system, it is suggested to give a meaningful name to the class diagram.
 2. The objects and their relationships should be acknowledged in advance.
 3. The attributes and methods (responsibilities) of each class must be known.
 4. A minimum number of desired properties should be specified as more number of the unwanted property will lead to a complex diagram.
 5. Notes can be used as and when required by the developer to describe the aspects of a diagram.
 6. The diagrams should be redrawn and reworked as many times to make it correct before producing its final version.

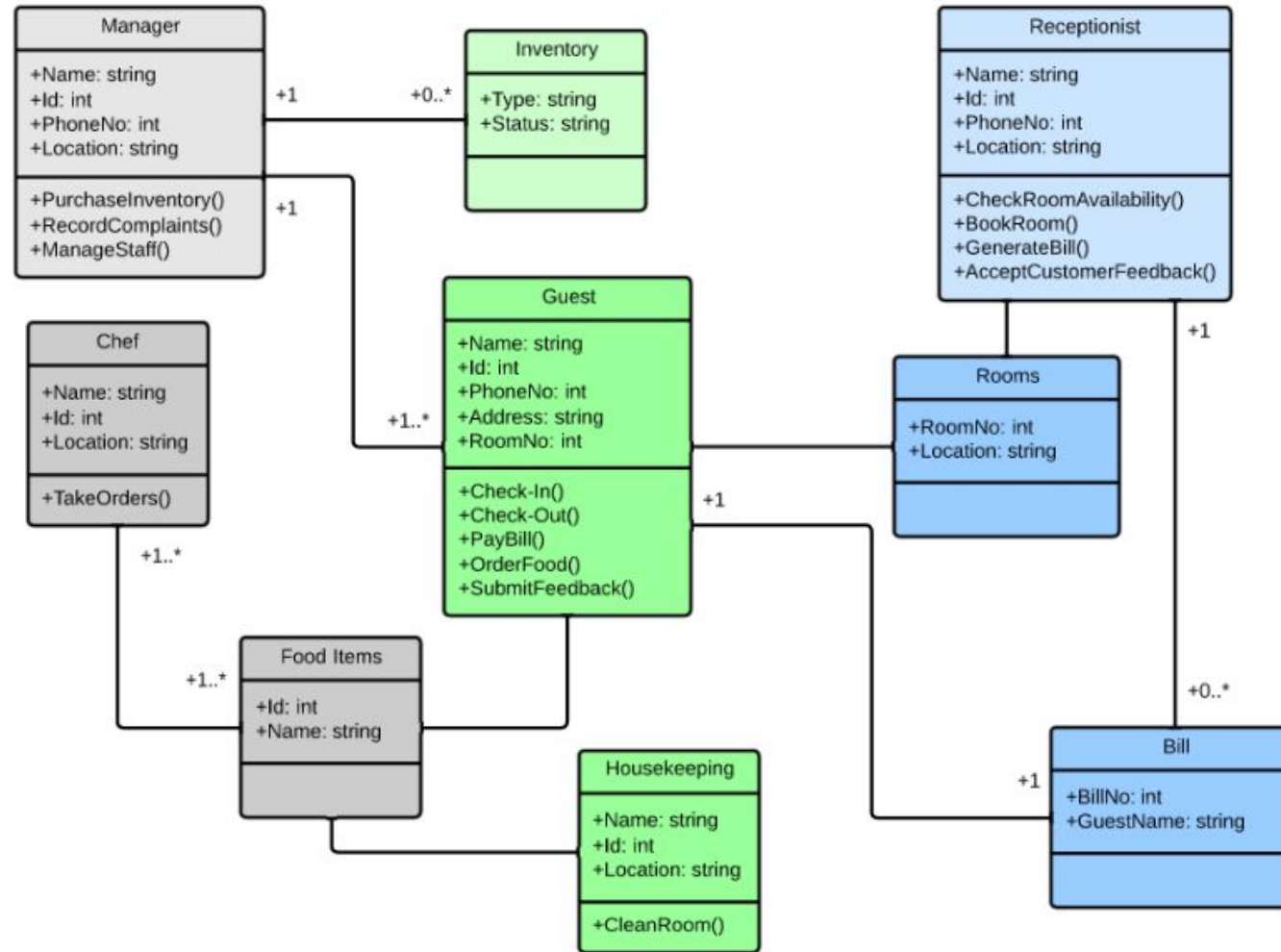
3.3 Class Diagram



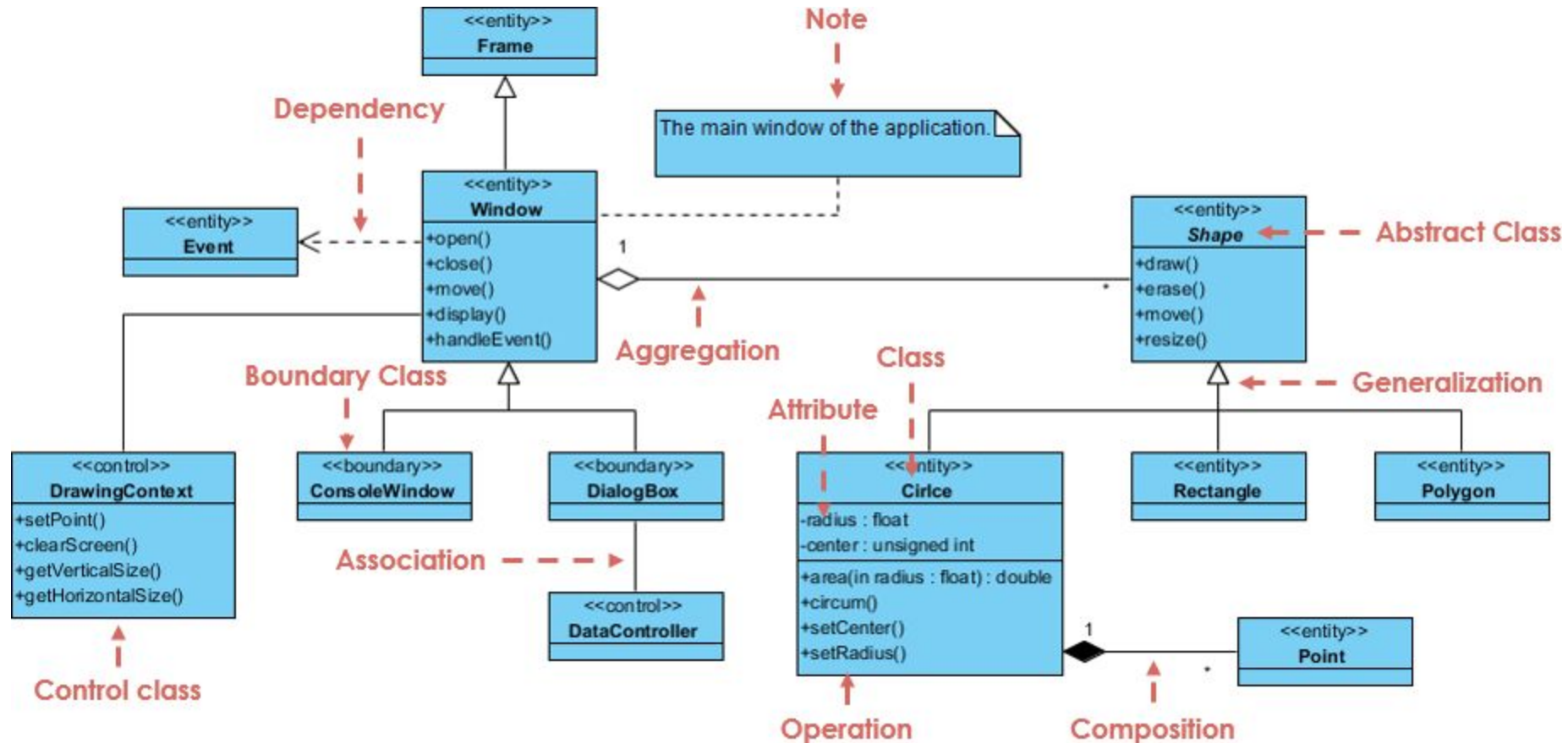
3.3 Class Diagram



3.3 Class Diagram



3.3 Class Diagram



3.3 Class Diagram

Usage of Class diagrams

- The class diagram is used to represent a static view of the system. It plays an essential role in the establishment of the component and deployment diagrams. It helps to construct an executable code to perform forward and backward engineering for any system, or we can say it is mainly used for construction. It represents the mapping with object-oriented languages that are C++, Java, etc. Class diagrams can be used for the following purposes:

1. To describe the static view of a system.
2. To show the collaboration among every instance in the static view.
3. To describe the functionalities performed by the system.
4. To construct the software application using object-oriented languages.

3.4 Advance Classes

Introduction

- The fundamental building block in a object-oriented system is an object or class.
- However, in UML, class is not the only general building block.
- It is only one of the general building blocks in UML, called as classifiers.
- A classifier is a mechanism which describes structural and behavioral features in a system.
- Other classifiers in UML are: interfaces, datatypes, signals, components, nodes, use cases and subsystems.

3.4 Advance Classes

- A **classifier** is a mechanism that has structural features (in the form of attributes), as well as behavioral features (in the form of operations).
- Classifiers include classes, interfaces, datatypes, signals, components, nodes, use cases, and subsystems.
- Those modeling elements that can have instances are called classifiers.
- For example, classes have instances known as objects.
- Some elements like packages and a generalization does not have instances.
- Every instance of a given classifier shares the same features.
- The most important kind of classifier in UML is class.

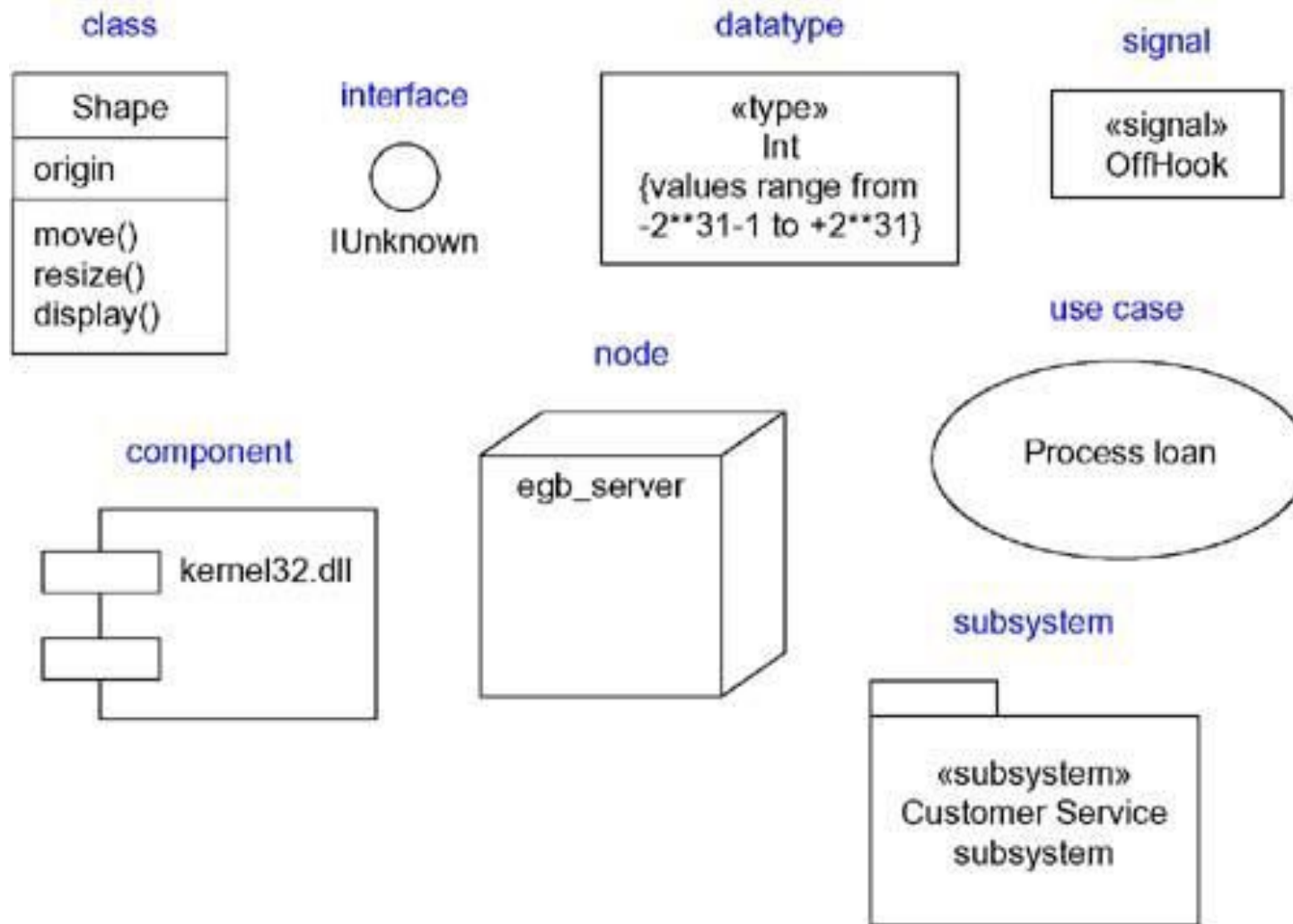
3.4 Advance Classes

The other kinds of classifiers are given in Table below:

Classifier	Description
Class	A class is a blueprint or a template for its objects
Interface	An interface is a collection of operations that must be realized by another element
Datatype	A type whose values have no identity
Signal	An asynchronous event communicated between objects
Component	Physical replaceable part of the system
Node	A physical existing at runtime which represents a computational resource
Use case	Collection of actions that a role performs on the system or the system does for a user.
Sub system	A grouping of elements which specify the behavior of a part of the system

3.4 Advance Classes

The graphical representation of the classifiers is as follows:



3.4 Advance Classes

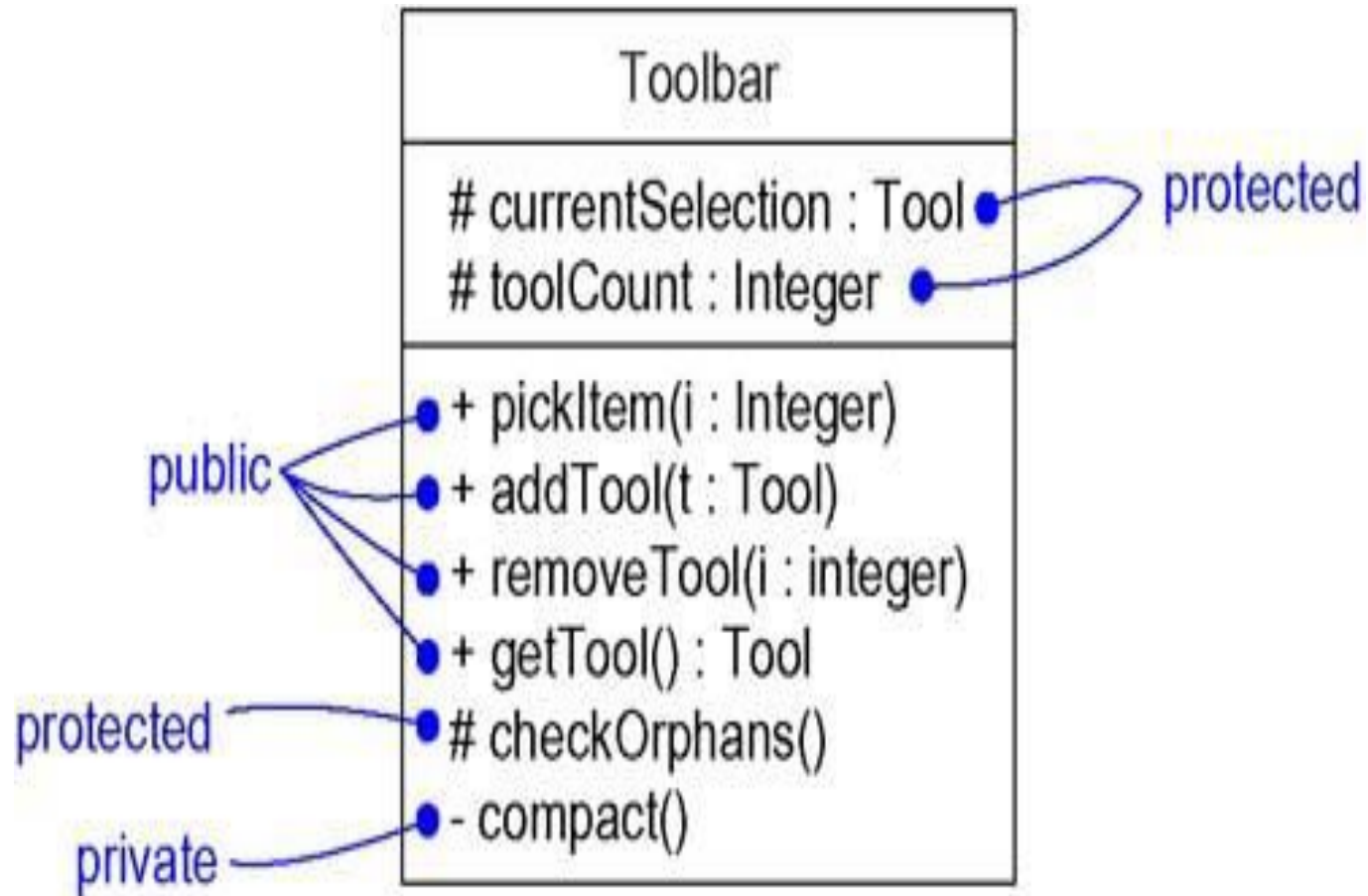
- Let's look at the advanced features (specific information that can be represented) of classes in UML.

1. Visibility

- One of the important features that can be applied to the classifier's attributes and operations is the visibility.
- The visibility feature specifies whether a classifier can be used or accessed by the other classifiers.
- The four access specifiers in UML are as shown below:

Symbol	Name	Description
+	Public	Accessible by all other classifiers
#	Protected	Accessible only by the descendents and the classifier itself
~	Package	Accessible by all the classifiers with the same package
-	Private	Accessible only by the classifier in which they are available

3.4 Advance Classes



3.4 Advance Classes

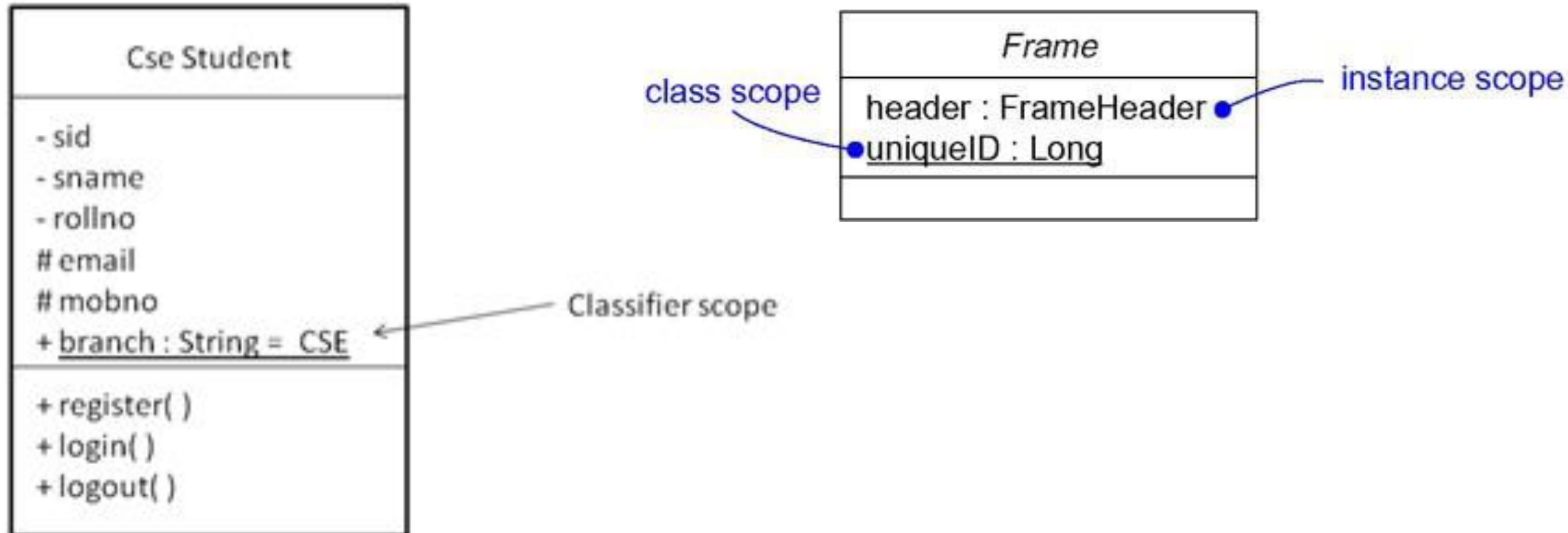
2. Scope

- Another feature that can be applied to the classifier's attributes and operations is the scope.
- The scope of an attribute or an operation denotes whether they have their existence in all the instances of the classifier or only one copy is available and is shared across all the instances of the classifier.
- The scope specifiers in UML are:

Scope	Description
instance	Each instance of the classifier contains a value of the feature
classifier	One copy of the feature's value is shared by all the instances of a classifier

3.4 Advance Classes

- A classifier scoped feature is graphically represented by underlining the feature within the classifier.
- All other normal features are treated as instanced scope.
- Example:



3.4 Advance Classes

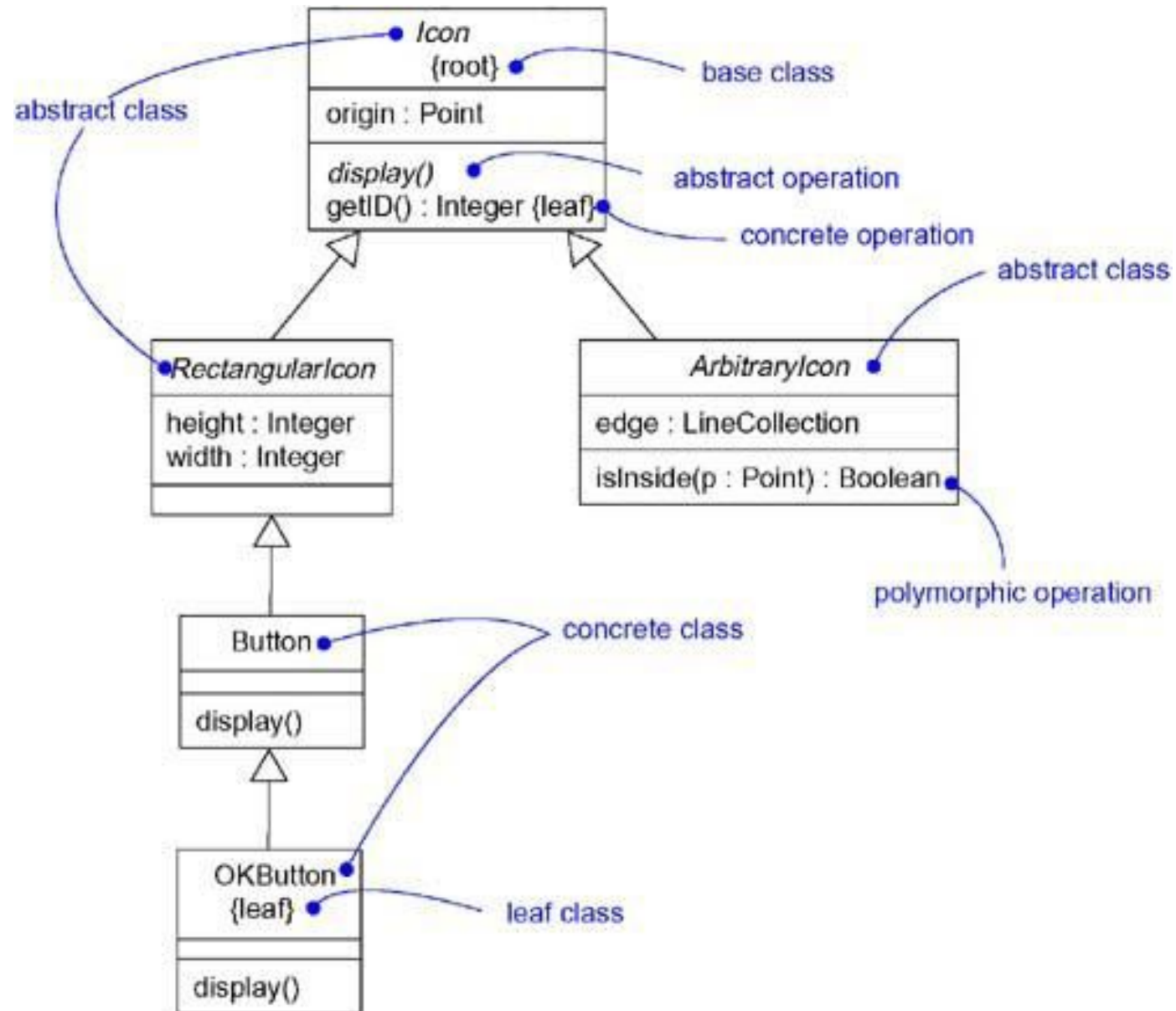
3. Abstract, Root, Leaf and Polymorphic elements

- While developing class diagrams, it is common to use generalization relationship to model generalization-specialization relationships between a group of classes.
- The classes at the top of the hierarchy are more generalized or abstract and the classes lower in the hierarchy are more specialized.
- To represent abstract classes for which we cannot create instances, the class name is written in italics in UML.
- To represent abstract methods, we write the operation signature in italics.
- We can also represent leaf and root classes in UML.
- A class which has no child classes is known as a leaf class.
- Such a class can be represented by writing leaf as a property under the class name.

3.4 Advance Classes

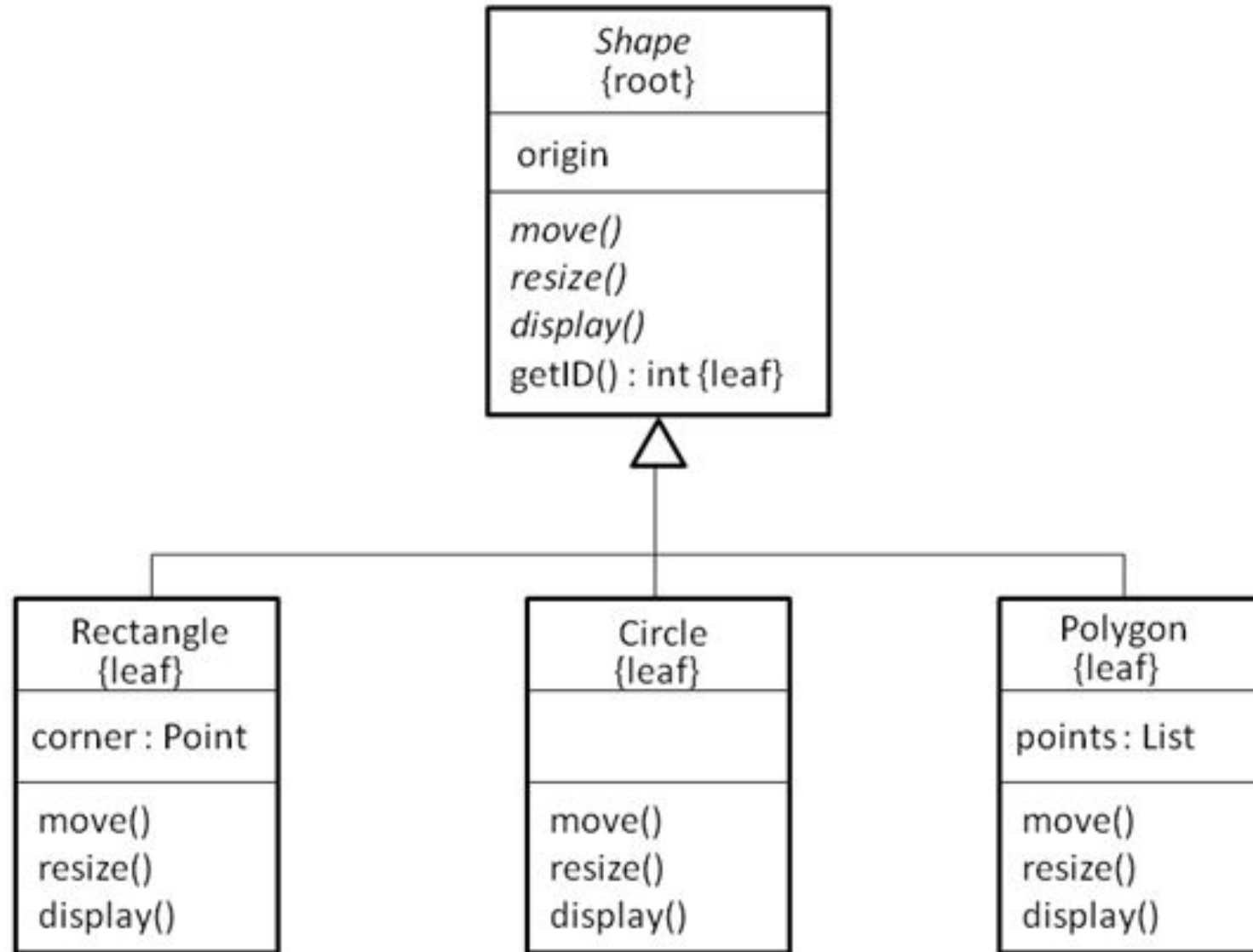
- Similarly, a class with no parents is known as a root class and such a class can be represented by writing root as a property as a property under the class name.
- In class hierarchies it is common for the classes to have methods with same signatures.
- This feature of declaring methods with same signatures is known as polymorphism.
- Such methods are represented in UML by writing their signatures in italics.
- We can also specify that a method cannot be overridden by marking the method with the leaf property written beside the method signature.

3.4 Advance Classes



3.4 Advance Classes

Example:



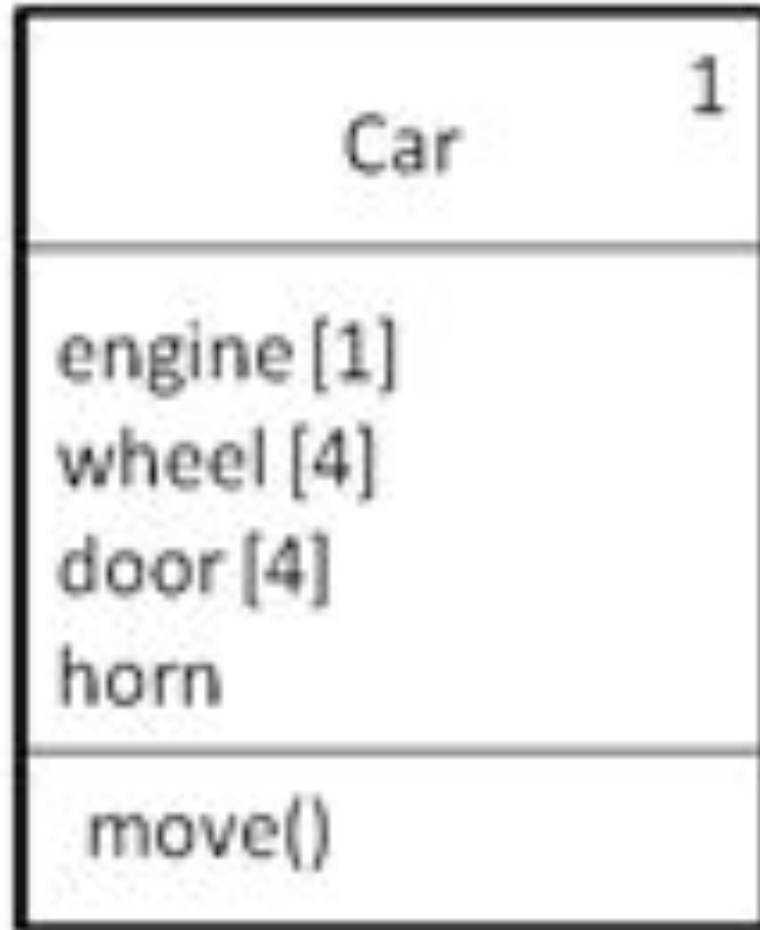
3.4 Advance Classes

4. Multiplicity

- When modeling classes it is reasonable to assume that a class can have any number of instances.
- In some cases a class might not have any instances at all. Such a class with no instances is called a utility class which contains only attributes and operations with the classifier scope.
- A class with exactly one instance is called a singleton class. Similarly other classes might have any number of instances which is the default in UML.
- The specification of how many instances a class can have is known as multiplicity.
- In UML, we can mention the multiplicity as an expression in the top right hand corner of the class icon.
- Attributes too can have multiplicity.
- The multiplicity of an attribute is represented in brackets after the attribute name.

3.4 Advance Classes

Example:



3.4 Advance Classes

5. Attributes

- While modeling classes, it is reasonable to write only the attribute name.
- Along with the name, you can also specify multiplicity, visibility and scope.
- Apart from these features, we can also specify the type, initial value and changeability of each attribute.
- The syntax of an attribute in UML is:
[visibility] name [multiplicity] [: type] [=initial value] [{property string}]

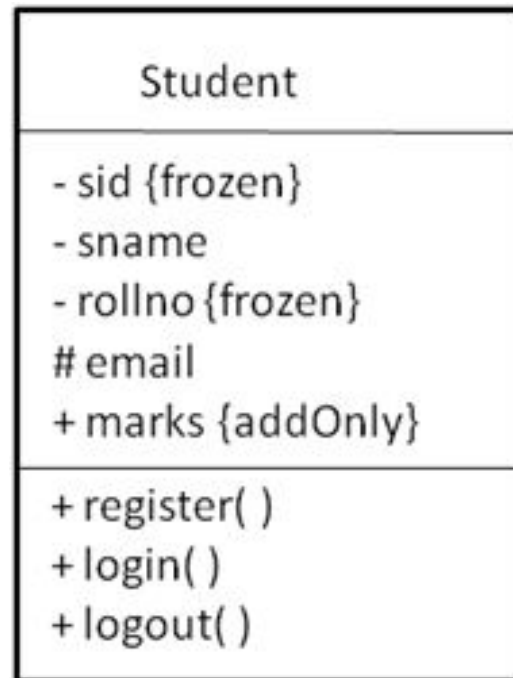
origin	Name only
+ origin	Name and visibility
origin : Point	Name and type
head : *Item	Name and complex type
name [0..1] : String	Name, multiplicity and type
origin : Point = {0,0}	Name, type and initial value
id : Integer {frozen}	Name and property

3.4 Advance Classes

The three predefined properties that can be used with attributes in UML are:

changeable	No restrictions on modifying the attributes value
addOnly	For attributes with multiplicity greater than one, values may be added but not removed or altered once it is created
frozen	Once created the value of an attribute cannot be changed

Example:



3.4 Advance Classes

6. Operations

- While modeling classes, it is simple to write only the operation name.
- Along with the name we can also mention the visibility and scope.
- Apart from these basic details, we can also specify the return type, parameters, concurrency semantics and other features.
- The syntax of an operation in UML is:
[visibility] name [(parameters-list)] [: return-type] [{property-string}]

3.4 Advance Classes

Example:

display	Name only
+ display	Name and visibility
set(n : Name, s : String)	Name and parameters
getID() : Integer	Name and return type
restart() {guarded}	Name and property

The signature of parameter in the parameters list of an operation is as follows:
[direction] name :type [=initial value]

Direction may be anyone of the following values:

in	An input parameter, may not be modified
out	An output parameter, may be modified
inout	An input parameter, may be modified

3.4 Advance Classes

- The operation name, parameters list and its return type is collectively known as the signature of the operation.
- There are four predefined properties (stereotypes) that can be applied to operations, in UML.
- They are:

isQuery	Execution of the operation does not change the state of the system.
sequential	Denotes that the operation is suitable for sequential access only.
guarded	Indicates that the operation guarantees integrity in case of concurrent access. Concurrent calls will be performed in a sequential manner.
concurrent	Denotes that the operation can be called concurrently and the integrity of the object will be preserved. Operation executes on the object concurrently.

3.4 Advance Classes

Template Classes

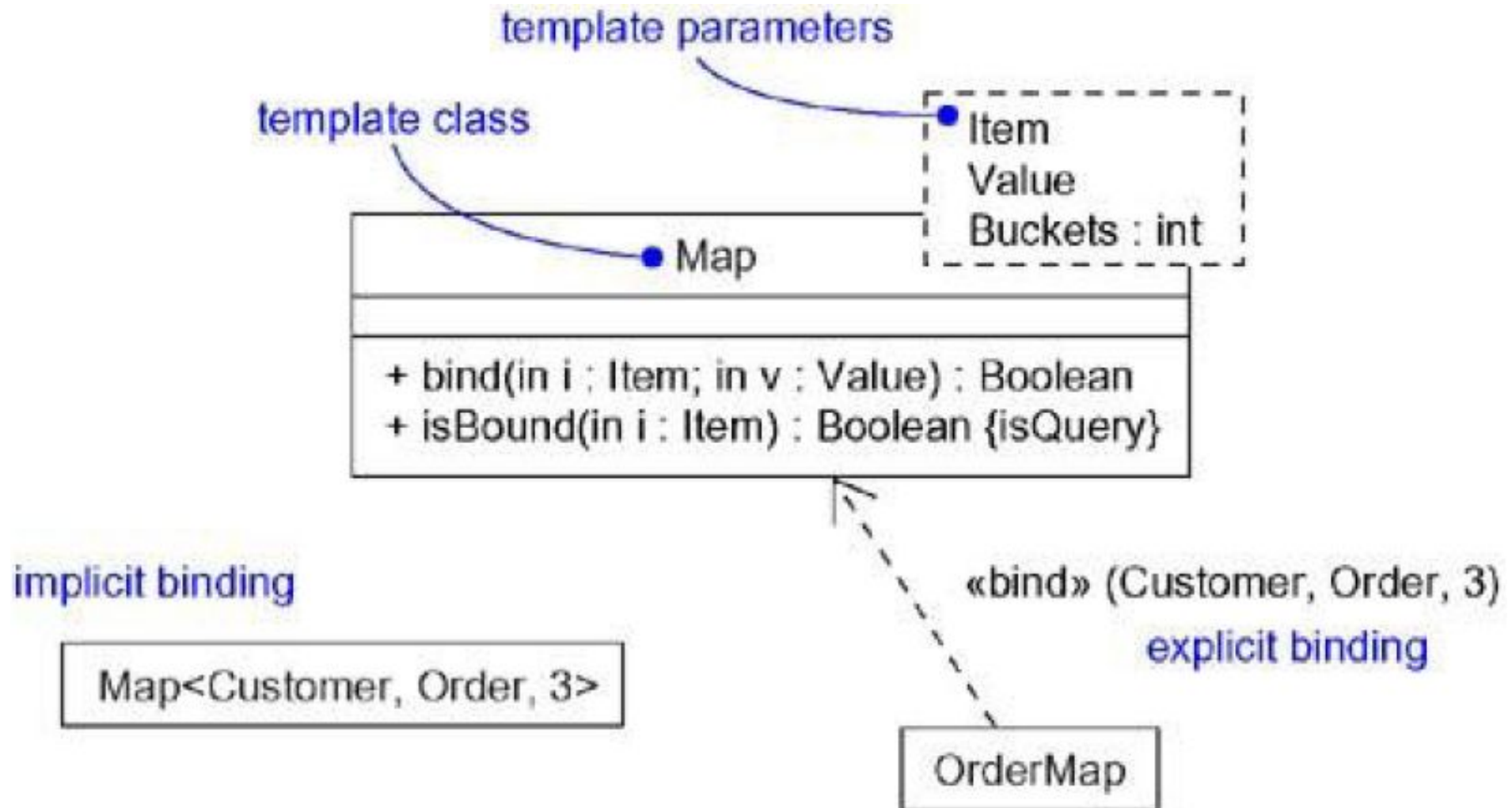
- A template is a parameterized element.
- A template includes slots for classes, objects, and values, and these slots serve as the template's parameters.
- Every templates should be instantiated first. Instantiation involves binding these formal template parameters to actual ones.
- For a template class, the result is a concrete class that can be used just like any ordinary class.

The ***instantiation of a template class can be modelled in two ways.***

- **First** done implicitly, by declaring a class whose name provides the binding.
- **Second**, explicitly by using a dependency stereotyped as bind, which specifies that the source instantiates the target template using the actual parameters.

3.4 Advance Classes

- A template class is indicated in figure below:



3.4 Advance Classes

3.5 Advance Relationship in UML

- The things in diagrams are connected with one another through relationships. So, relationships are the connections between things.
- In UML, the four important relationships are dependency, generalization, association and realization. Each type of relationship has its own graphical representation.
- After being familiar with the basics regarding the UML relationships, let's have a look at the advanced information that can be represented using the above relationships:
- Advanced Relationships can be categorized as:
 - Dependency
 - Generalization
 - Association
 - Realization

3.5 Advance Relationship in UML

1. Dependency:

- The dependency relationship is also known as using relationship i.e., if the specification of one thing (for example Graphics class code) changes then it will automatically affect the behavior of another thing (for example Hello class using the drawstring method of Graphics class) that uses it.
- A dependency relationship is graphically represented as a dashed arrow.



3.5 Advance Relationship in UML

- In general, a plain dependency relationship is reasonable for representing the using relationship between two things.
- But, if the user needs to specify extra information like the nature of the dependency relationship, he/she can use the predefined stereotypes that can be applied to dependency relationship.
- There are about 17 such stereotypes and are organized into 6 groups based on which things are participating in the dependency relationship.
- The stereotypes are as follows:
- First group consists of eight stereotypes that apply to dependency relationship between classes.
- They are as follows:

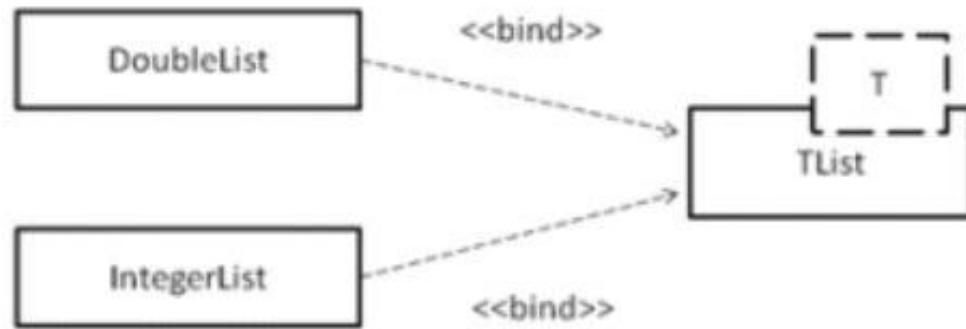
3.5 Advance Relationship in UML

bind	Specifies that the source instantiates (creates object) the target template using the given actual parameters
derive	Specifies that the source may be computed from the target
friend	Specifies that the source is accessible by the target regardless of the visibility of the source element
instanceOf	Specifies that the source object is an instance of the target classifier
instantiate	Specifies that the source creates instances of the target
powertype	Specifies that the target is a powertype of the source. A powertype is a classifier whose objects are the children of a given parent.
refine	Specifies that the target is at a lower level of abstraction than the source
use	Specifies that the source element depends on the target for its functionality.

3.5 Advance Relationship in UML

Examples:

<<bind>>



<<derive>>

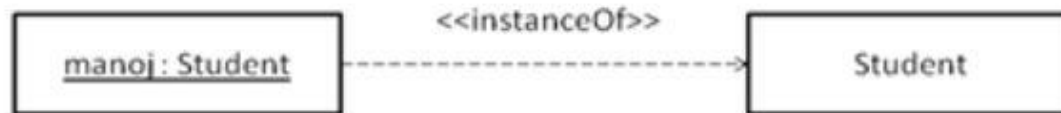


3.5 Advance Relationship in UML

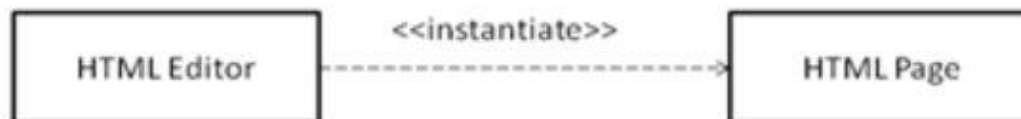
<<friend>>



<<instanceOf>>

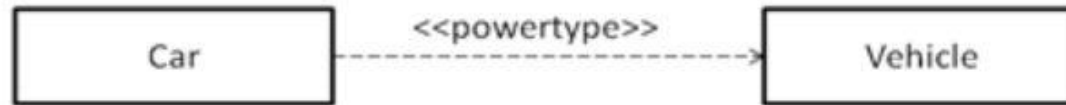


<<instantiate>>



3.5 Advance Relationship in UML

<<powertype>>



<<refine>>



<<use>>



3.5 Advance Relationship in UML

- There are two stereotypes that apply to dependencies between packages.
- They are:

access	Specifies that the source package has the right to access the elements of the target package
import	Specifies that the public elements of the target package enter the namespace of the source package as if they were declared in the source.

Examples:

<<access>>



<<import>>



3.5 Advance Relationship in UML

- Two stereotypes apply to dependency relationship among use case.
- They are:

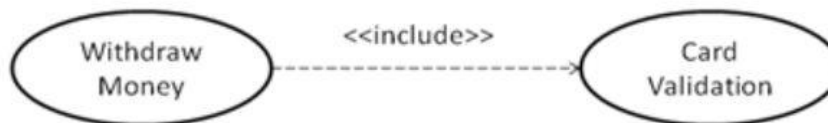
extend	Specifies that the target use case extends the functionality of the source use case
include	Specifies that the source use case incorporates the behavior of the target use case to function as a whole.

Examples:

<<extend>>



<<include>>



3.5 Advance Relationship in UML

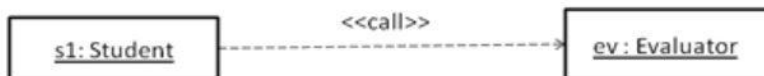
- Three stereotypes apply to interactions among objects.
- They are:

become	Specifies that the source object becomes the target object at some point in time
call	Specifies that an operation in source calls another operation of the target
copy	Specifies that the target is an exact, but independent copy of the source

<<become>>



<<call>>



<<copy>>



3.5 Advance Relationship in UML

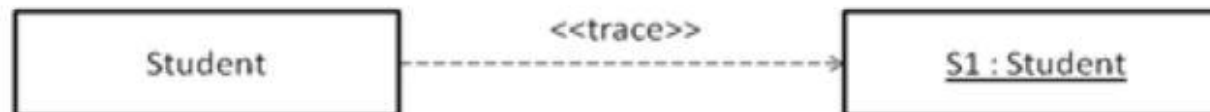
- One stereotype applies to dependencies in the context of state machines:

send	Specifies that the source, an operation whose target is a signal, sends the target signal
------	---

- One stereotype applies to dependencies in the context of subsystem:

trace	Specifies that the source element has a conceptual connection to the target which individually belong to different models
-------	---

Example:

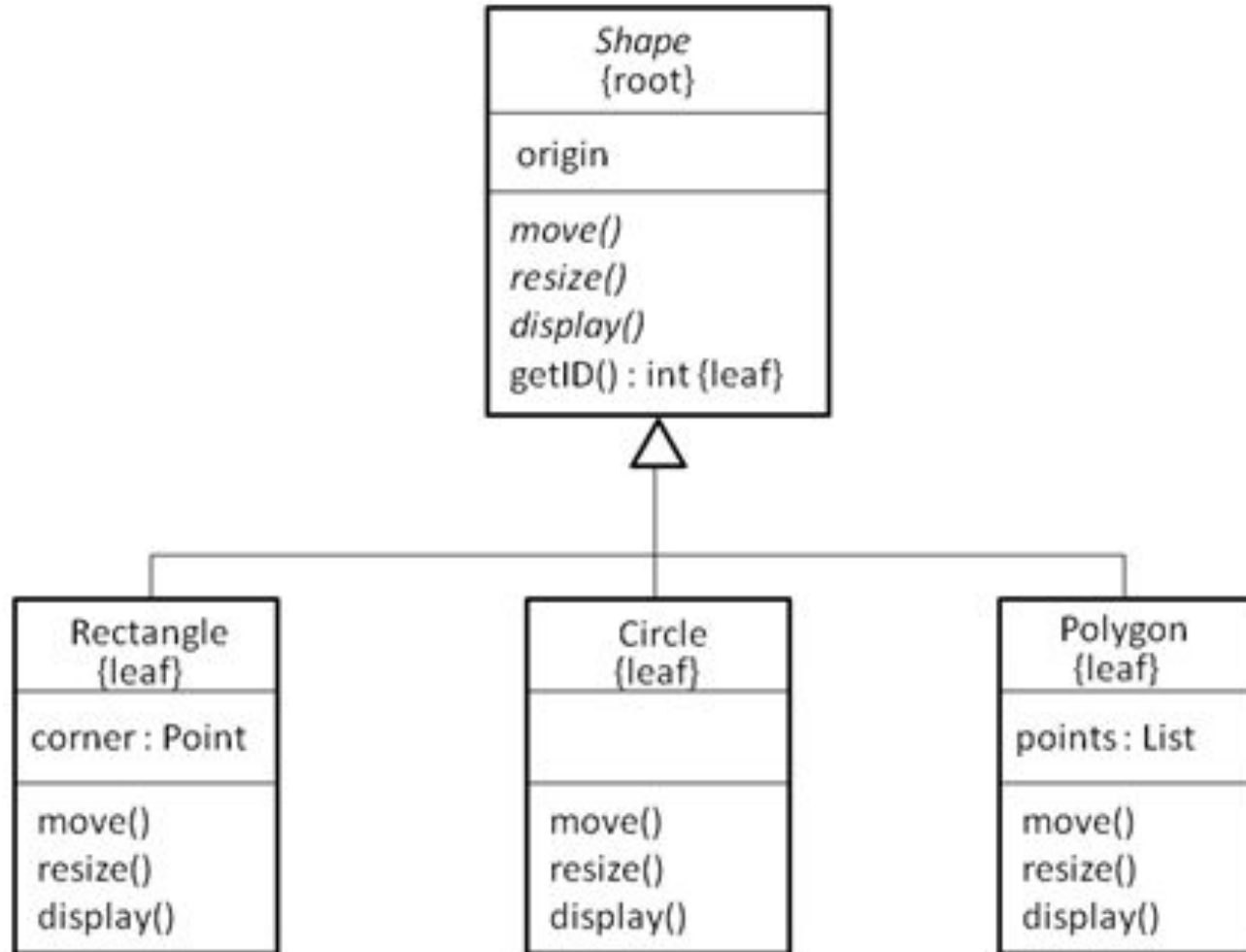


3.5 Advance Relationship in UML

2. Generalization:

- A generalization is a relationship between a general thing and a more specific kind of that thing.
- In multiple Inheritance one class may have more than one parent
- A generalization relationship represents generalization-specialization relationship between classes.
- The class with the general structure and behavior is known as the parent or superclass and the class with specific structure and behavior is known as the child or subclass.
- Consider the class hierarchy represented in the next slide:

3.5 Advance Relationship in UML



3.5 Advance Relationship in UML

- Shape class is the parent or super class and the remaining three classes namely Rectangle, Circle and Polygon are the child or subclasses of the Shape class.
- A subclass in the generalization relationship automatically inherits the state and behavior of the superclass.
- The generalization relationship is also known as the “is-a” relationship.
- If a class has only one parent, such inheritance is known as single inheritance and if a class has one or more parents, such inheritance is known as multiple inheritance.
- To represent extra semantics in a generalization relationship, UML provides one stereotype and four constraints.

3.5 Advance Relationship in UML

- The stereotype on generalization relationship is:

implementation	Specifies that the child inherits the implementation of the parent but does not support its interfaces, thereby violating the principle of substitutability.
----------------	--

- There are four standard constraints that apply to the generalization relationship:

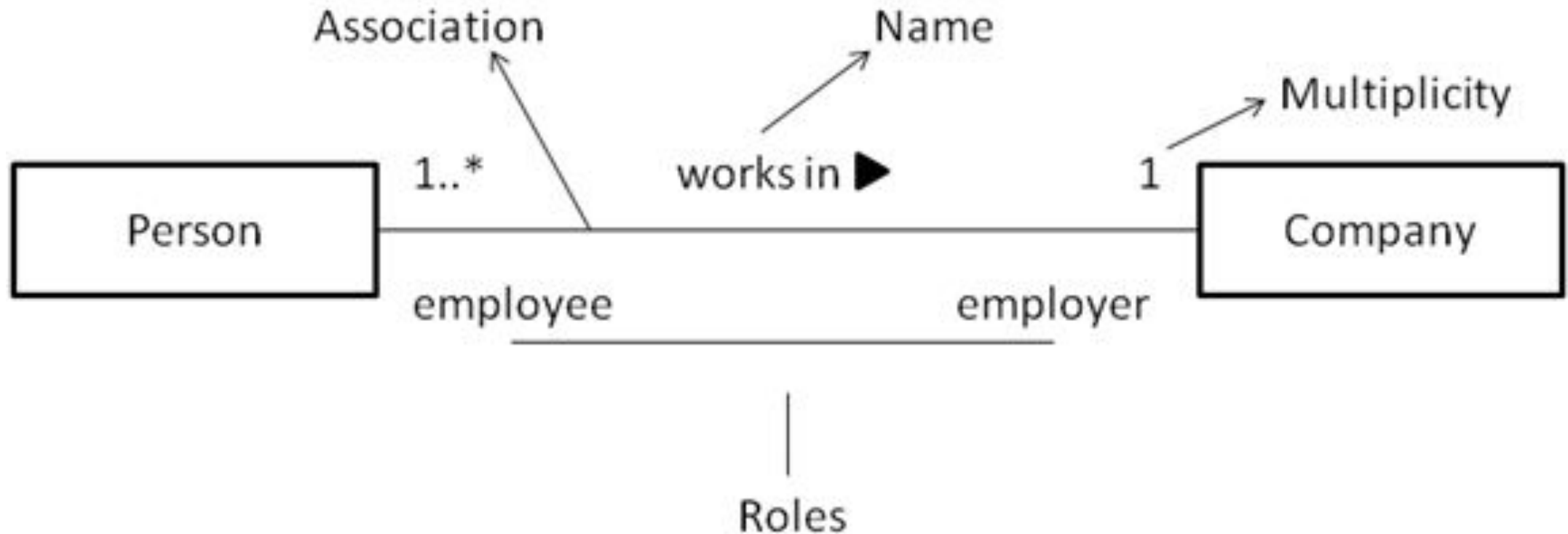
complete	Specifies that no additional children are permitted
incomplete	Specifies that additional children are permitted
disjoint	Specifies that the objects of the parent may not have more than one of the children as a type
overlapping	Specifies that the objects of the parent may have more than one of the children as a type.

3.5 Advance Relationship in UML

3. Association:

- Association is a structural relationship which denotes a connection between two or more things.
- The association relationship can represent either physical or logical connections between things.
- The graphical representation of the association relationship is a solid line.
- The four basic adornments for an association relationship are: name, role at each end of the association, multiplicity at each end of the association and aggregation.
- Over these basic features, there are other advanced features like: navigation, visibility, qualification, composition and association classes.

3.5 Advance Relationship in UML



3.5 Advance Relationship in UML

3.1 Navigation:

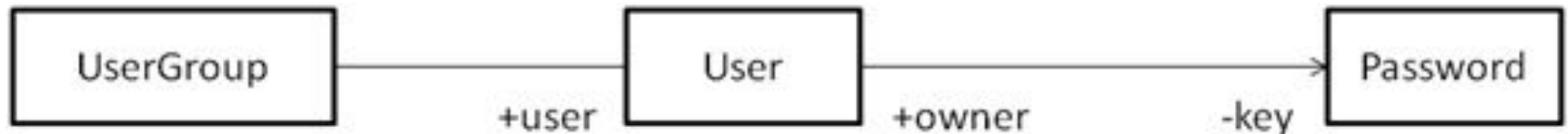
- Given an association between two things, we can navigate from one thing to another and vice versa.
- By default the navigation of an association is bidirectional.
- In some cases we may need to navigate in only direction.
- For example, given an order, we can get the details of the products in the order. But, if we consider a product, there might be no need to know about the order to which it belongs.



3.5 Advance Relationship in UML

3.2 Visibility:

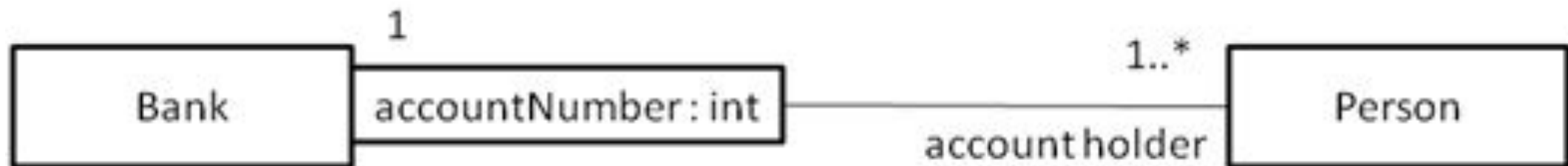
- Given an association, we can navigate from one object to another.
- However, in some situations we may want to limit the visibility (access) of an object to the objects outside the association relationship.
- To limit the visibility of an object, we can use the visibility specifiers: public (+), private (-) and protected (#).
- For example, a user-group object can access the user object and an user object can access the password. If, private visibility is specified for the password object, the user-group object cannot access the password of the user object



3.5 Advance Relationship in UML

3.3 Qualification:

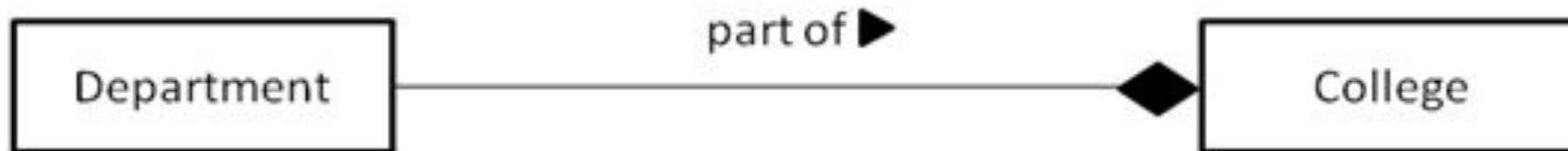
- In binary associations, where two classes are connected together, if one object of a class has to identify a set of instances of another class, we can use qualifiers.
- A qualifier is a set of attributes of an association which is used to identify a set of instances of a class.
- Each qualifier is represented with a name and type in a rectangle box at the qualified end of the association relationship.
- For example, a bank object is able to recognize the person (account holder) based on the account number. So, account number is the qualifier which is used to identify an account holder.



3.5 Advance Relationship in UML

3.4 Composition:

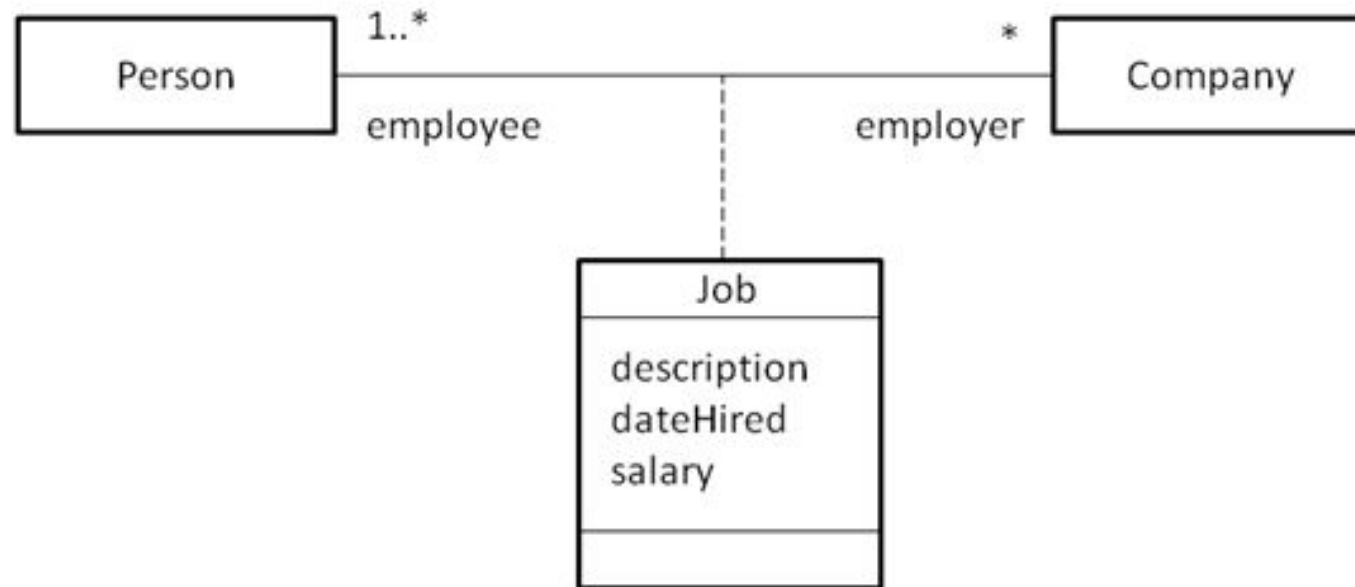
- The composition is a flavor of association relationship.
- Composition as well as aggregation relationships represent whole-part relationships, in which one thing is a part of the other thing.
- There is a simple difference between the association and composition relationships.
- In composition, the lifetime of the part is dependent of the lifetime of the whole thing. Where as in aggregation, the lifetime of the part is independent of the whole thing.
- Composition is graphically represented by adorning the association relationship with a filled diamond head near the whole end.



3.5 Advance Relationship in UML

Association Class

- Sometimes in an association relationship, the association might have attributes or properties like a class does.
- In such cases, it is modeled as an association class.
- An association class in UML is represented with a class icon attached to the association with a dashed line.



3.5 Advance Relationship in UML

- Apart from the above advanced features, UML provides six constraints that can be applied to an association.
- They are:

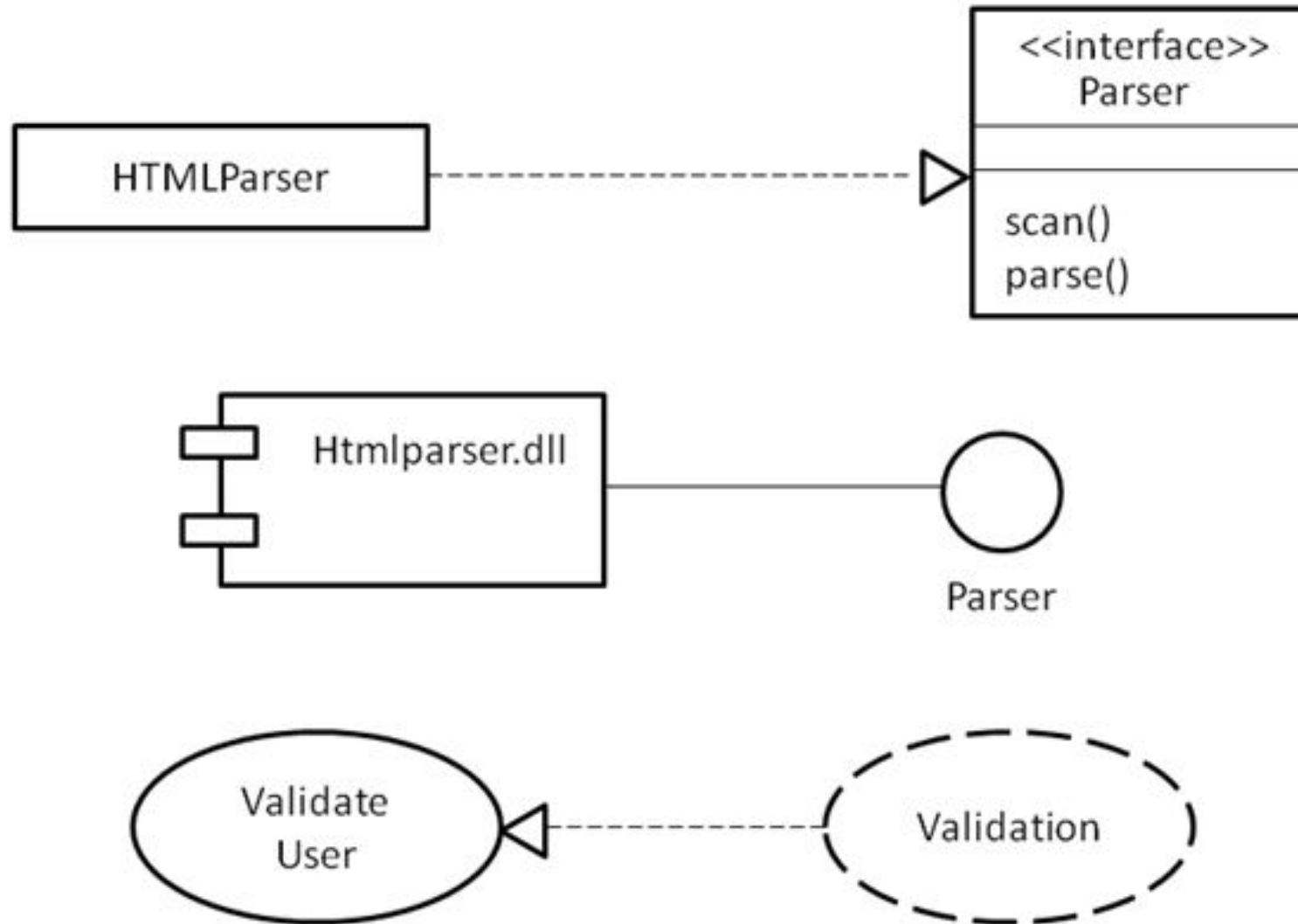
implicit	Specifies that the relationship is not physical, but rather conceptual
ordered	Specifies that the objects at one end of the association are in a certain order
changeable	Links between objects can be added, removed
addOnly	New links may be added from an object on the opposite side of the association
frozen	After adding a link it might not be modified later
xor	Indicates that among a set of associations, only one can be applied for each object

3.5 Advance Relationship in UML

4. Realization:

- Realization is a semantic relationship between classifiers, where one classifier provides the specification which is implemented by the other classifier.
- Realization can exist between an interface and class, interface and a component and between a use case and collaboration.
- Realization is graphically represented as dashed line with a hollow arrow head pointing towards the classifier which provides the specification.

3.5 Advance Relationship in UML

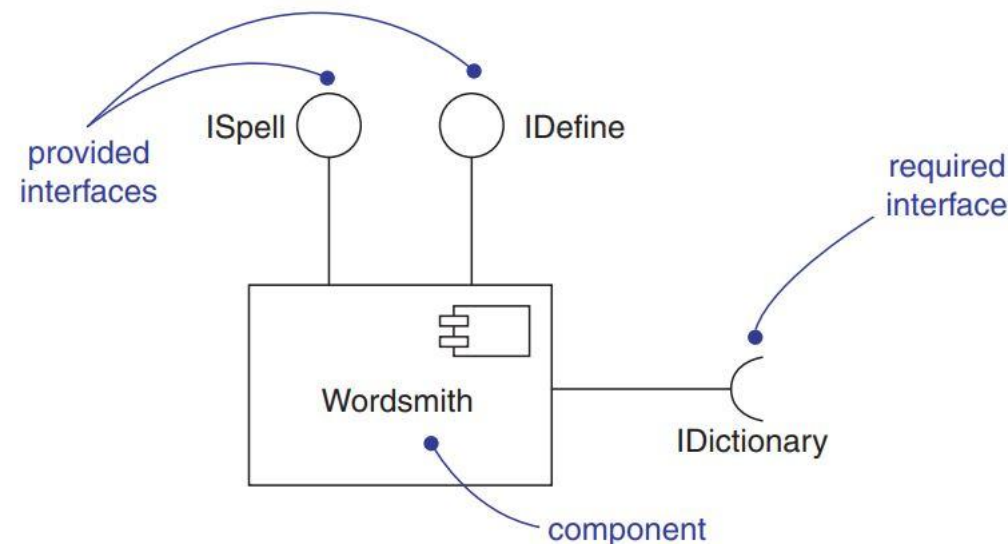


3.6 Interface, Types & Roles

- Interfaces define a line between the specification of what an abstraction does and the implementation of how that abstraction does it.
- An **interface** is a collection of operations that are used to specify a service of a class or a component.
- A **type** is a stereotype of a class used to specify a domain of objects, together with the operations applicable to the object.
- Types and roles provide a mechanism for you to model the static and dynamic conformance of an abstraction to an interface in a specific context.
- A **role** is the behavior of an entity participating in a particular context.
- Graphically, an interface may be rendered as a stereotyped class in order to expose its operations and other properties.

3.6 Interface, Types & Roles

- To show the relationship between a class and its interfaces, a special notation is provided. A provided interface (one that represents services provided by the class) is shown as a small circle attached to the class box. A required interface (one that represents services required by a class of another class) is shown as a small semicircle attached to the class box.



3.6 Interface, Types & Roles

Names

- Every interface must have a name that distinguishes it from other interfaces.
- A name is a textual string.
- That name alone is known as a *simple name*; a *path name* is the interface name prefixed by the name of the package in which that interface lives.

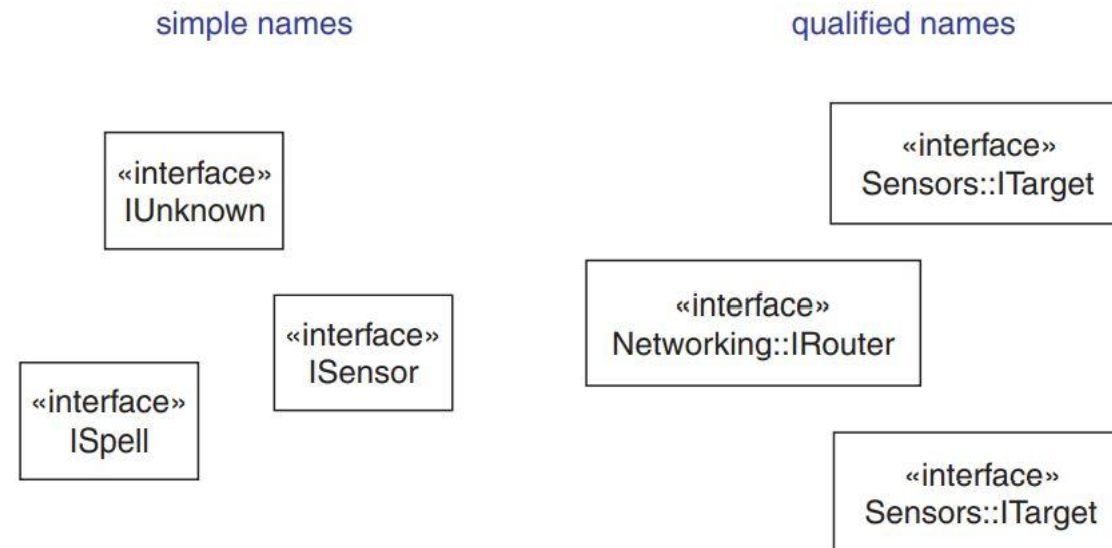


Fig: Simple and Path Names

3.6 Interface, Types & Roles

Note:

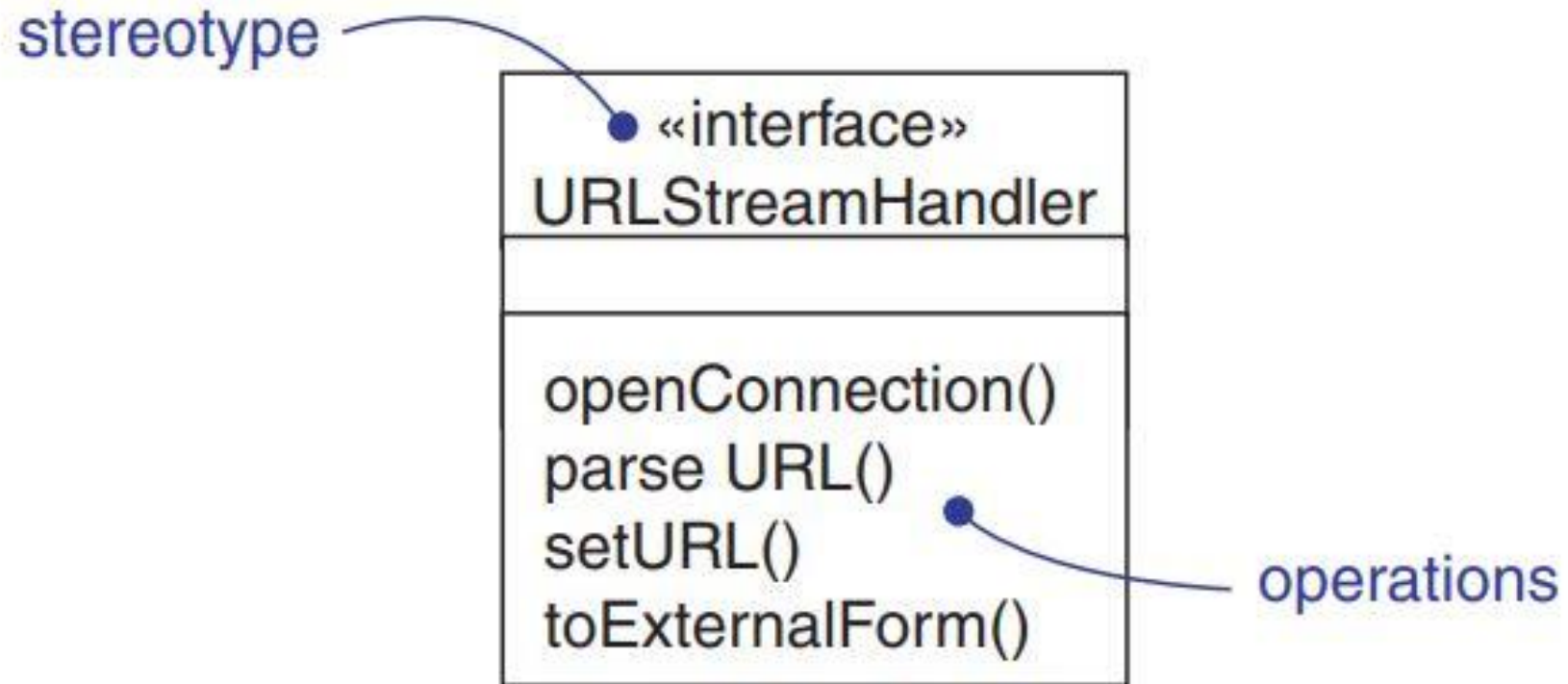
- An interface name may be text consisting of any number of letters, numbers, and certain punctuation marks (except for marks such as the colon, which is used to separate an interface name and the name of its enclosing package) and may continue over several lines.
- In practice, interface names are short nouns or noun phrases drawn from the vocabulary of the system you are modeling.

3.6 Interface, Types & Roles

Operations

- An interface is a named collection of operations used to specify a service of a class or of a component.
- Unlike classes or types, interfaces do not specify any implementation (so they may not include any methods, which provide the implementation of an operation).
- Like a class, an interface may have any number of operations.
- These operations may be adorned with visibility properties, concurrency properties, stereotypes, tagged values, and constraints.
- When you declare an interface, you render an interface as a stereotyped class, listing its operations in the appropriate compartment.
- Operations may be drawn showing only their name, or they may be augmented to show their full signature and other properties, as in Figure.

3.6 Interface, Types & Roles



3.6 Interface, Types & Roles

Relationships

- Like a class, an interface may participate in generalization, association, and dependency relationships.
- In addition, an interface may participate in realization relationships.
- Realization is a semantic relationship between two classifiers in which one classifier specifies a contract that another classifier guarantees to carry out.
- An interface specifies a contract for a class or a component without dictating its implementation.
- A class or component may realize many interfaces. In so doing, it commits to carry out all these contracts faithfully, which means that it provides a set of methods that properly implement the operations defined in the interface.
- The set of services that it agrees to provide are its provided interface.
- Similarly, a class or a component may depend on many interfaces.

3.6 Interface, Types & Roles

- In so doing, it expects that these contracts will be honored by some set of components that realize them.
- The set of services that a class requires from other classes are its required interface.
- This is why we say that an interface represents a seam in a system. An interface specifies a contract, and the client and the supplier on each side of the contract may change independently, as long as each fulfills its obligations to the contract.
- As in Figure illustrated in another slide, you can show that an element realizes an interface in two ways. First, you can use the simple form in which the interface and its realization relationship are rendered as a line between the class box and a small circle (for a provided interface) or a small semicircle (for a required interface).
- This form is useful when you simply want to expose the seams in your system; it is usually the preferred form.

3.6 Interface, Types & Roles

- However, the limitation of this style is that you can't directly visualize the operations or signals provided by the interface.
 - Second, you can use the expanded form in which you render an interface as a stereotyped class, which allows you to visualize its operations and other properties, and then draw a realization relationship (for a provided interface) or a dependency (for a required interface) from the classifier or component to the interface box.
 - In the UML, a realization relationship is rendered as a dashed directed line with a large triangular arrowhead pointing to the interface.
 - This notation is a cross between generalization and dependency.
- Note: Interfaces are similar to abstract classes. For example, neither may have direct instances. An abstract class can implement its concrete operations, however. An interface is more like an abstract class in which all of the operations are also abstract.

3.6 Interface, Types & Roles

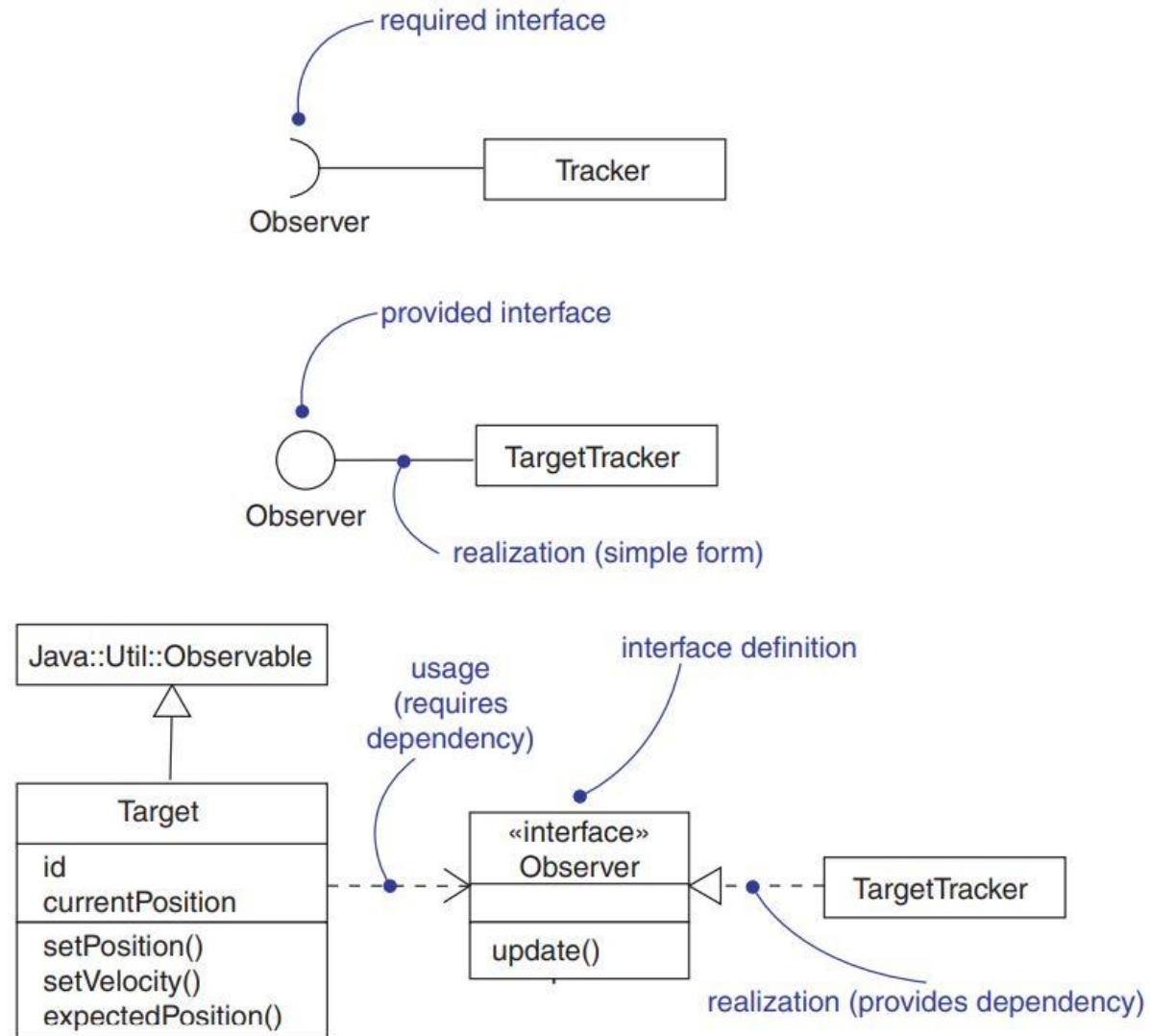


Fig: Realizations

3.7 Object Diagrams

- Object diagrams model the instances of things contained in class diagrams.
- An object diagram shows a set of objects and their relationships at a point in time.
- Object diagrams are used to model the static design view or static process view of a system.
- This involves modeling a snapshot of the system at a moment in time and rendering a set of objects, their state, and their relationships.
- Object diagrams are not only important for visualizing, specifying, and documenting structural models, but also for constructing the static aspects of systems through forward and reverse engineering.
- Graphically, an object diagram is a collection of vertices and arcs.

3.7 Object Diagrams

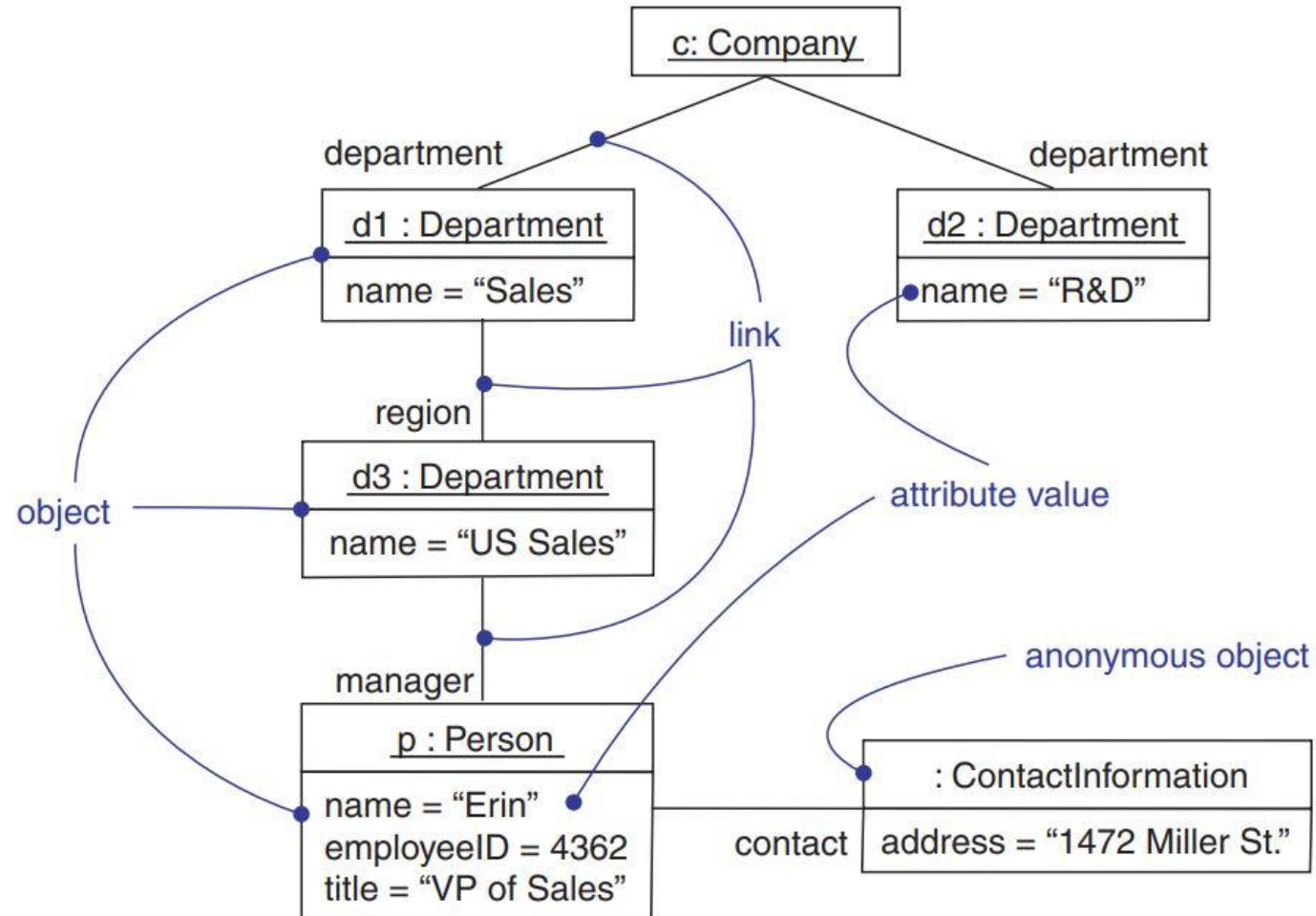


Fig: An Object Diagram

3.7 Object Diagrams

Common Properties

- An object diagram is a special kind of diagram and shares the same common properties as all other diagrams—that is, a name and graphical contents that are a projection into a model.
- What distinguishes an object diagram from all other kinds of diagrams is its particular content.

Contents

- Object diagrams commonly contain

■ Objects

■ Links

- Like all other diagrams, object diagrams may contain notes and constraints.

3.7 Object Diagrams

Note:

- An object diagram correlates with a class diagram:
- The class diagram describes the general situation, and the instance diagram describes specific instances derived from the class diagram.
- An object diagram contains primarily objects and links.
- Deployment diagrams may also occur in generic and instance forms: General deployment diagrams describe node types, and instance deployment diagrams describe a concrete configuration of node instances described by those types.

3.7 Object Diagrams

Common Uses

- Object diagrams are used to model the static design view or static process view of a system just as we do with class diagrams, but from the perspective of real or prototypical instances.
- This view primarily supports the functional requirements of a system—that is, the services the system should provide to its end users.
- Object diagrams let you model static data structures.
- Modeling object structures involves taking a snapshot of the objects in a system at a given moment in time. An object diagram represents one static frame in the dynamic storyboard represented by an interaction diagram.
- You use object diagrams to visualize, specify, construct, and document the existence of certain instances in your system, together with their relationships to one another.

3.8 Interactions

- In every interesting system, objects don't just sit idle; they interact with one another by passing messages.
- An interaction is a behavior that comprises a set of messages exchanged among a set of objects within a context to accomplish a purpose.
- A message is a specification of a communication between objects that conveys information with the expectation that activity will ensue.
- You may find an interaction wherever objects are linked to one another. You'll find interactions in the collaboration of objects that exist in the context of your system or subsystem. You will also find interactions in the context of an operation. Finally, you'll find interactions in the context of a class. Most often, you'll find interactions in the collaboration of objects that exist in the context of your system or subsystem as a whole.

3.8 Interactions

- For example, in a system for Web commerce, you'll find objects on the client (such as instances of the classes BookOrder and OrderForm) interacting with one another. You'll also find objects on the client (again, such as instances of BookOrder) interacting with objects on the server (such as instances of BackOrderManager). These interactions therefore not only involve localized collaborations of objects (such as the interactions surrounding OrderForm), but they may also cut across many conceptual levels of your system (such as the interactions surrounding BackOrderManager).
- You'll also find interactions among objects in the implementation of an operation. The parameters of an operation, any variables local to the operation, and any objects global to the operation (but still visible to the operation) may interact with one another to carry out the algorithm of that operation's implementation. For example, the operation moveToPosition(p : Position) defined for a class in a mobile robot will involve the interaction of a parameter (p), an object global to the operation (such as the object currentPosition), and possibly several local objects (such as local variables used by the operation to calculate intermediate points in a path to the new position).

3.8 Interactions

- Finally, you will find interactions in the context of a class. You can use interactions to visualize, specify, construct, and document the semantics of a class. For example, to understand the meaning of a class `RayTraceAgent`, you might create interactions that show how the attributes of that class collaborate with one another (and with objects global to instances of the class and with parameters defined in the class's operations).

3.8 Interactions

Objects and Roles

- The objects that participate in an interaction are either concrete things or prototypical things.
- As a concrete thing, an object represents something in the real world. For example, p, an instance of the class Person, might denote a particular human.
- Alternately, as a prototypical thing, p might represent any instance of Person.
 - Note: *In a collaboration, the interactors are usually prototypical things that play particular roles, not specific objects in the real world, although it is sometimes useful to describe collaborations among particular objects.*
- In the context of an interaction, you may find instances of classes, components, nodes, and use cases. Although abstract classes and interfaces, by definition, may not have any direct instances, you may represent instances of these things in an interaction.

3.8 Interactions

- Such instances do not represent direct instances of the abstract class or of the interface, but may represent, respectively, indirect (or prototypical) instances of any concrete children of the abstract class or some concrete class that realizes that interface.
- Think of an object diagram as a representation of the static aspect of an interaction, setting the stage for the interaction by specifying all the objects that work together.
- An interaction goes further by introducing a dynamic sequence of messages that may pass along the links that connect these objects.

3.8 Interactions

Links and Connectors

- A link is a semantic connection among objects.
- In general, a link is an instance of an association.
- As in figure, wherever a class has an association to another class, there may be a link between the instances of the two classes; wherever there is a link between two objects, one object can send a message to the other object.

3.8 Interactions

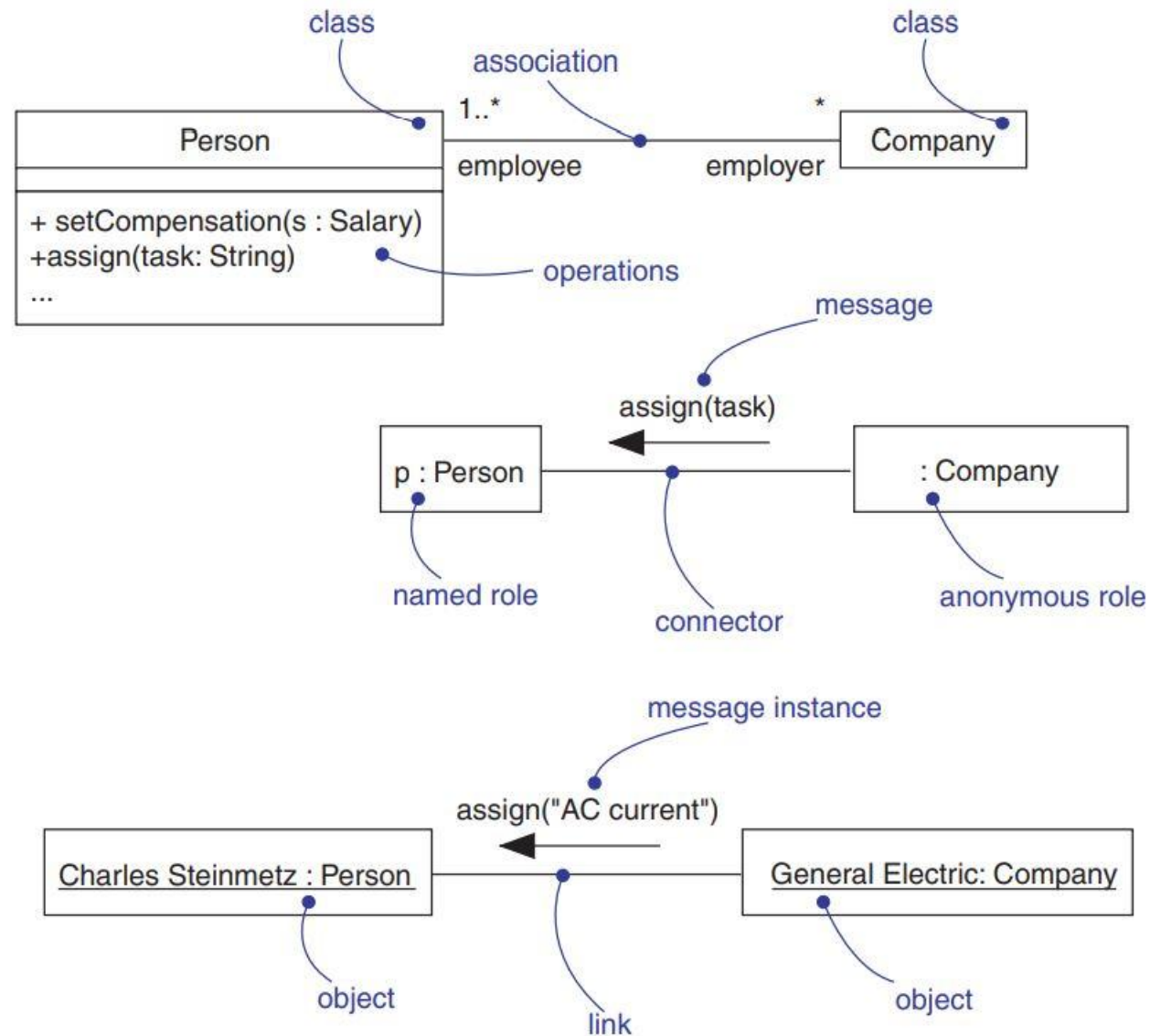


Fig: Associations, Links & Connectors

3.8 Interactions

- A link specifies a path along which one object can dispatch a message to another (or the same) object.
- Most of the time it is sufficient to specify that such a path exists. If you need to be more precise about how that path exists, you can adorn the appropriate end of the link with one of the following constraints:
 - association Specifies that the corresponding object is visible by association
 - self Specifies that the corresponding object is visible because it is the dispatcher of the operation
 - global Specifies that the corresponding object is visible because it is in an enclosing scope
 - local Specifies that the corresponding object is visible because it is in a local scope
 - parameter Specifies that the corresponding object is visible because it is a parameter

3.8 Interactions

- *Note: As an instance of an association, a link may be rendered with most of the adornments appropriate to associations, such as a name, association role name, navigation, and aggregation. Multiplicity, however, does not apply to links, since they are instances of an association.*
- In most models, we are more interested in prototypical objects and links within some context rather than individual objects and links.
- A prototypical object is called a role; a prototypical link is called a connector; the context is a collaboration or the internal structure of a classifier.
- The multiplicity of roles and connectors are defined relative to their enclosing context. For example, a multiplicity of 1 on a role means one object represents the role for each object that represents the context.
- A collaboration or internal structure can be used many times, just like a class declaration; each use is bound to a separate set of objects and links for the context, roles, and links.

3.8 Interactions

- Figure before in the slide shows an example:
 - The top of the figure shows a class diagram that declares classes Person and Company and the many-to-many employee-employer association between them.
 - The middle shows the contents of a collaboration WorkAssignment that assigns an employee to a job. It has two roles and a connector between them.
 - The bottom shows an instance of this collaboration, in which there are objects and links bound to the roles and connectors. A concrete message in the bottom represents the prototypical message declaration in the collaboration

3.8 Interactions

Messages

- Suppose you have a set of objects and a set of links that connect those objects. If that's all you have, then you have a completely static model that can be represented by an object diagram. Object diagrams model the state of a society of objects at a given moment in time and are useful when you want to visualize, specify, construct, or document a static object structure.
- Suppose you want to model the changing state of a society of objects over a period of time. Think of it as taking a motion picture of a set of objects, each frame representing a successive moment in time. If these objects are not totally idle, you'll see objects passing messages to other objects, sending events, and invoking operations. In addition, at each frame you can explicitly visualize the current state and role of individual instances.

3.8 Interactions

- A message is the specification of a communication among objects that conveys information with the expectation that activity will ensue.
- The receipt of a message instance may be considered an occurrence of an event. (An occurrence is the UML name for an instance of an event.)
- When you pass a message, an action usually results on its receipt.
- An action may result in a change in state of the target object and objects accessible from it.

3.8 Interactions

- In the UML, you can model several kinds of messages.
 - Call Invokes an operation on an object; an object may send a message to itself, resulting in the local invocation of an operation
 - Return Returns a value to the caller
 - Send Sends a signal to an object
 - Create Creates an object
 - Destroy Destroys an object; an object may commit suicide by destroying itself
- Note: *You can model complex actions in the UML as well. In addition to the five basic kinds of messages listed above, you can include actions on individual objects. The UML does not specify the syntax or semantics of such actions; it is expected that tools will supply various actions languages or use the syntax of programming languages.*

3.8 Interactions

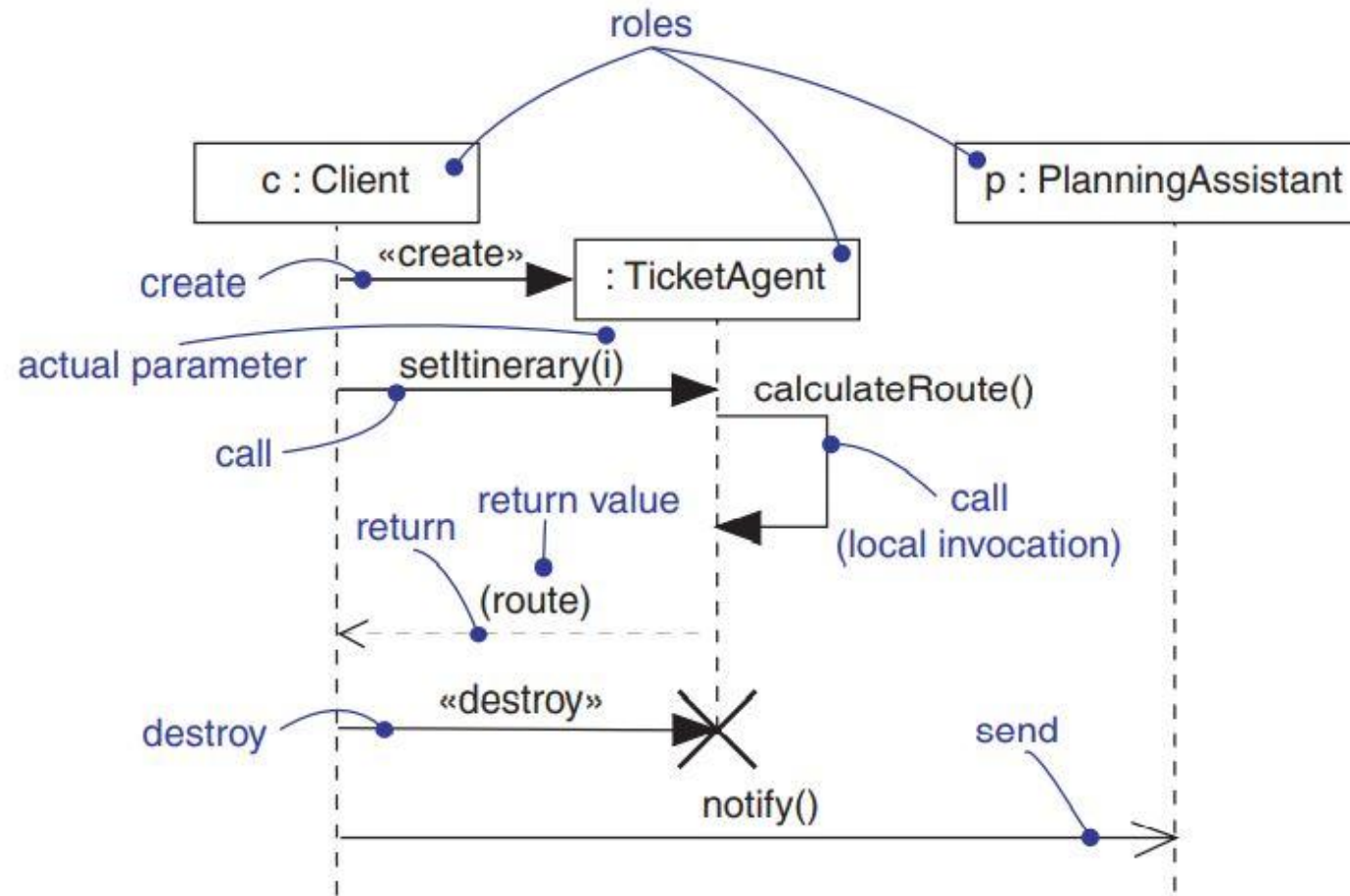


Fig: Messages

3.8 Interactions

- The most common kind of message you'll model is the call, in which one object invokes an operation of another (or the same) object.
- An object can't just call any random operation. If an object, such as *c* in the example above, calls the operation *setItinerary* on an instance of the class *TicketAgent*, the operation *setItinerary* must not only be defined for the class *TicketAgent* (that is, it must be declared in the class *TicketAgent* or one of its parents), it must also be visible to the caller *c*.

3.8 Interactions

- When an object calls an operation or sends a signal to another object, we can provide actual parameters to the message.
- Similarly, when an object returns control to another object, we can model the return value as well.
- Messages can also correspond to the sending of signals.
- A signal is an object value communicated to a target object asynchronously.
- After sending a signal, the sending object continues its own execution.
- When the target object receives the signal message, it independently decides what to do about it.
- Usually signals trigger transitions in the state machine of the target object.
- Firing a transition causes the target object to execute actions and change to a new state.

3.8 Interactions

- In an asynchronous message-passing system, communicating objects execute concurrently and independently.
- They share values only by passing messages, so there is no danger of conflict over shared memory.

3.8 Interactions

Sequencing:

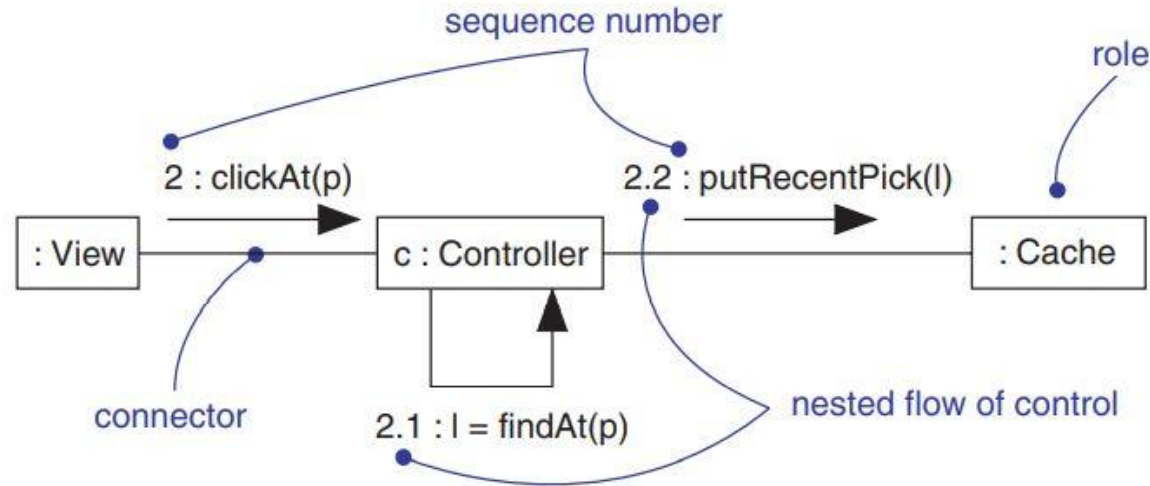


Fig: Procedural Sequence

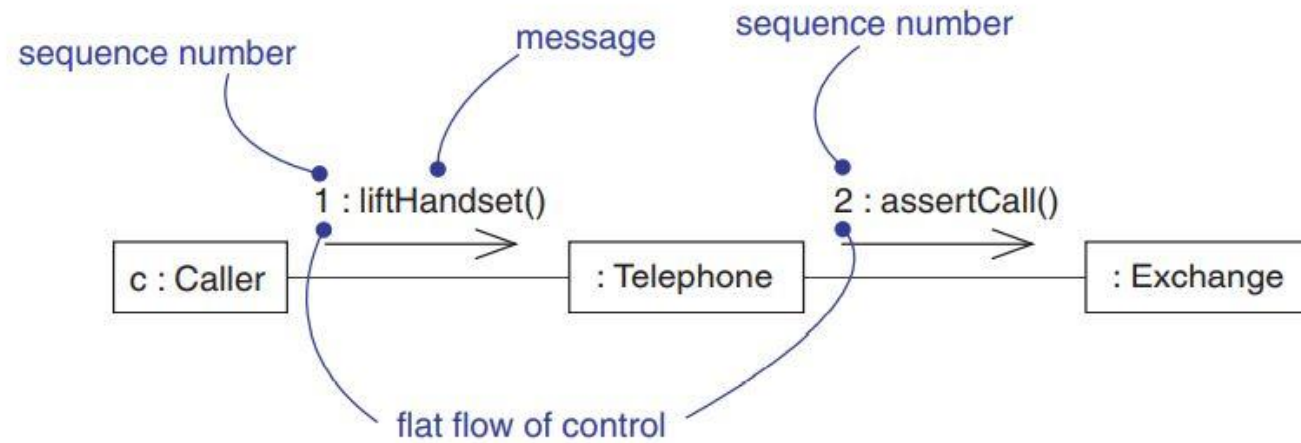


Fig: Flat Sequence

3.8 Interactions

Creation, Modification, and Destruction

- Most of the time the objects you show participating in an interaction exist for the entire duration of the interaction.
- However, in some interactions objects may be created (specified by a create message) and destroyed (specified by a destroy message).
- The same is true of links: The relationships among objects may come and go.
- To specify if an object or link enters and/or leaves during an interaction, you can attach a note to its role within a communication diagram.
- During an interaction, an object typically changes the values of its attributes, its state, or its roles.
- We can represent the modification of an object in a sequence diagram by showing the state or the values on the lifeline.

3.8 Interactions

- Within a sequence diagram, the lifetime, creation, and destruction of objects or roles are explicitly shown by the vertical extent of their lifelines.
- Use sequence diagrams if object lifetimes are important to show.

3.8 Interactions

Representation

- When you model an interaction, you typically include both roles (each one representing objects that appear in an instance of the interaction) and messages (each one representing the communication between objects, with some resulting action).
- We can visualize those roles and messages involved in an interaction in two ways:
 - by emphasizing the time ordering of its messages,
 - and by emphasizing the structural organization of the roles that send and receive messages.
- In the UML, the first kind of representation is called a sequence diagram; the second kind of representation is called a communication diagram.
- Both sequence diagrams and communication diagrams are kinds of interaction diagrams.

3.11 Interaction Diagrams

- Sequence diagrams and communication diagrams—which are collectively called interaction diagrams.
- An interaction diagram shows an interaction, consisting of a set of objects and their relationships, including the messages that may be dispatched among them.
- A **sequence diagram** is an interaction diagram that emphasizes the time ordering of messages.
- Graphically, a sequence diagram is a table that shows objects arranged along the X axis and messages, ordered in increasing time, along the Y axis.
- A **communication diagram** is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages.
- Graphically, a communication diagram is a collection of vertices and arcs.
- We use interaction diagrams to model the dynamic aspects of a system.

3.11 Interaction Diagrams

- For the most part, this involves modeling concrete or prototypical instances of classes, interfaces, components, and nodes, along with the messages that are dispatched among them, all in the context of a scenario that illustrates a behavior.
- Interaction diagrams may stand alone to visualize, specify, construct, and document the dynamics of a particular society of objects, or they may be used to model one particular flow of control of a use case.
- Interaction diagrams are not only important for modeling the dynamic aspects of a system, but also for constructing executable systems through forward and reverse engineering.

3.11 Interaction Diagrams

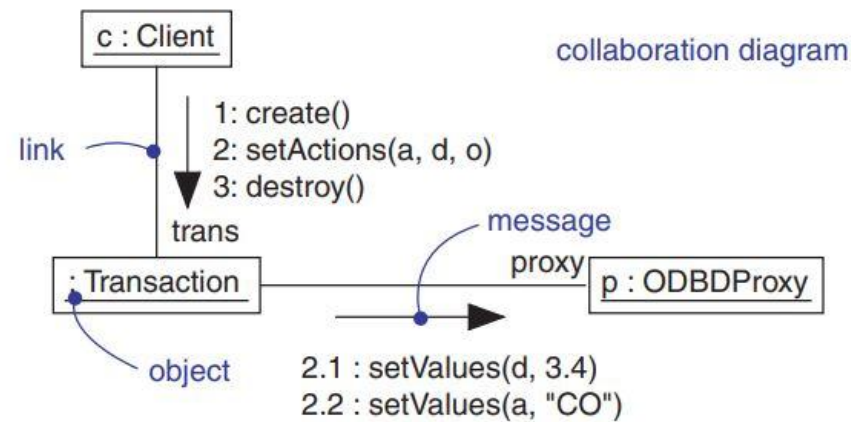
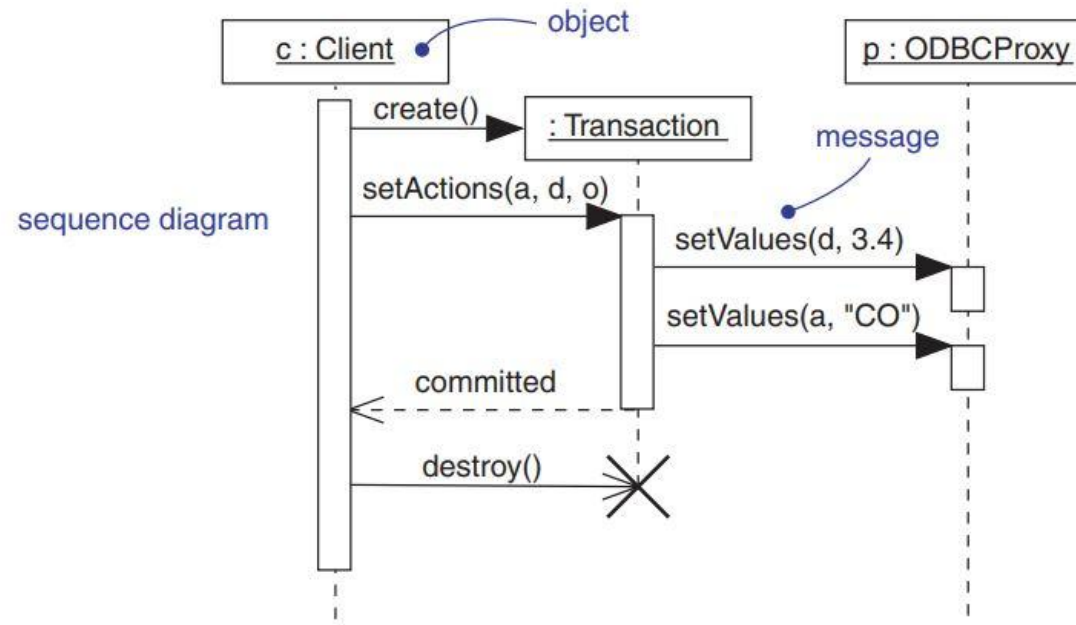


Fig: interaction Diagram

3.11 Interaction Diagrams

- Interaction diagrams commonly contain
 - Roles or objects
 - Communications or links
 - Messages
- Like all other diagrams, interaction diagrams may contain notes and constraints.

3.11 Interaction Diagrams

Sequence Diagrams

- A sequence diagram emphasizes the time ordering of messages.
- As Figure in next slide shows, we form a sequence diagram by first placing the objects or roles that participate in the interaction at the top of your diagram, across the horizontal axis.
- Typically, you place the object or role that initiates the interaction at the left, and increasingly more subordinate objects or roles to the right.
- Next, you arrange the messages that these objects send and receive along the vertical axis in order of increasing time from top to bottom.
- This gives the reader a clear visual cue to the flow of control over time.
- Sequence diagrams have two features that distinguish them from communication diagrams.

3.11 Interaction Diagrams

- First, there is the lifeline. An object lifeline is the vertical dashed line that represents the existence of an object over a period of time. Most objects that appear in an interaction diagram will be in existence for the duration of the interaction, so these objects are all aligned at the top of the diagram, with their lifelines drawn from the top of the diagram to the bottom.
- Objects may be created during the interaction. Their lifelines start with the receipt of the message create (drawn to box at the head of the lifeline). Objects may be destroyed during the interaction. Their lifelines end with the receipt of the message destroy (and are given the visual cue of a large X, marking the end of their lives). If the interaction represents the history of specific, individual objects, then object symbols with underlined names are placed at the head of the lifelines. Most of the time, however, you will be showing prototypical interactions. The lifelines do not represent specific objects; they represent prototypical roles that represent different objects in each instance of the interaction. In that normal case, you do not underline the names, as they are not specific objects.

3.11 Interaction Diagrams

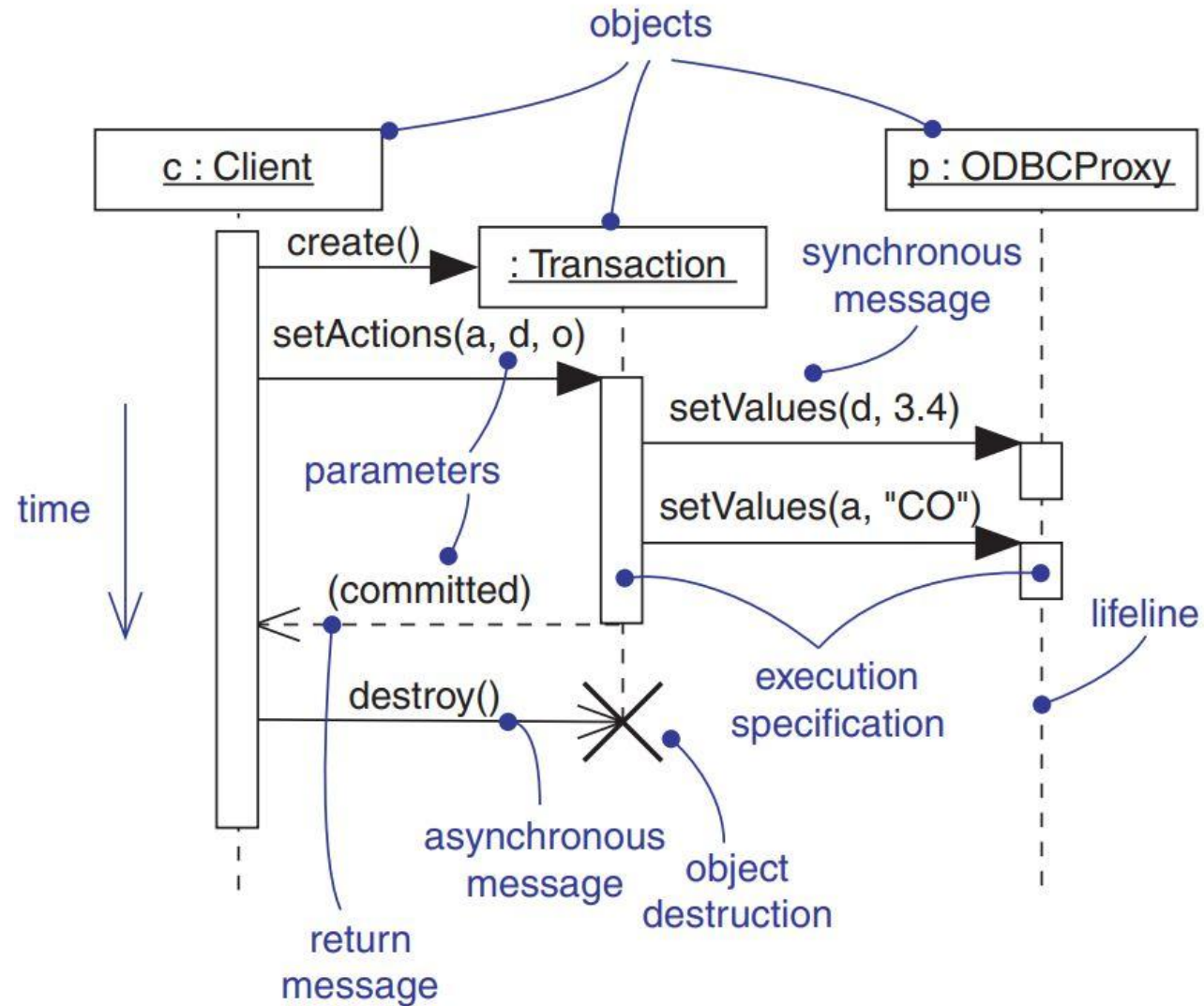


Fig: Sequence Diagram

3.11 Interaction Diagrams

- Second, there is the focus of control. The focus of control is a tall, thin rectangle that shows the period of time during which an object is performing an action, either directly or through a subordinate procedure. The top of the rectangle is aligned with the start of the action; the bottom is aligned with its completion (and can be marked by a return message). You can show the nesting of a focus of control (caused by recursion, a call to a self-operation, or by a callback from another object) by stacking another focus of control slightly to the right of its parent (and can do so to an arbitrary depth). If you want to be especially precise about where the focus of control lies, you can also shade the region of the rectangle during which the object's method is actually computing and control has not passed to another object, but this is rather fussy.

3.11 Interaction Diagrams

- The main content in a sequence diagram is the messages.
- A message is shown by an arrow from one lifeline to another.
- The arrowhead points to the receiver.
- If the message is asynchronous, the line has a stick arrowhead.
- If the message is synchronous (a call), the line has a filled triangular arrowhead.
- A reply to a synchronous message (a return from a call) is shown by a dashed arrow with a stick arrowhead.
- The return message may be omitted, as there is an implicit return after any call, but it is often useful for showing return values.
- The ordering of time along a single lifeline is significant.
- Usually the exact distance does not matter; lifelines only show relative sequences, so the lifeline is not a scale diagram of time.

3.11 Interaction Diagrams

- Usually the positions of messages on separate pairs of lifelines do not imply any sequencing information; the messages could occur in any order.
- The entire set of messages on separate lifelines forms a partial ordering.
- A series of messages establishes a chain of causality, however, so that any point on another lifeline at the end of the chain must always follow the point on the original lifeline at the start of the chain.

3.11 Interaction Diagrams

Communication Diagrams

- A communication diagram emphasizes the organization of the objects that participate in an interaction.
- As Figure next shows, you form a communication diagram by first placing the objects that participate in the interaction as the vertices in a graph.
- Next, we render the links that connect these objects as the arcs of this graph.
- The links may have rolenames to identify them.
- Finally, we adorn these links with the messages that objects send and receive.
- This gives the reader a clear visual cue to the flow of control in the context of the structural organization of objects that collaborate

3.11 Interaction Diagrams

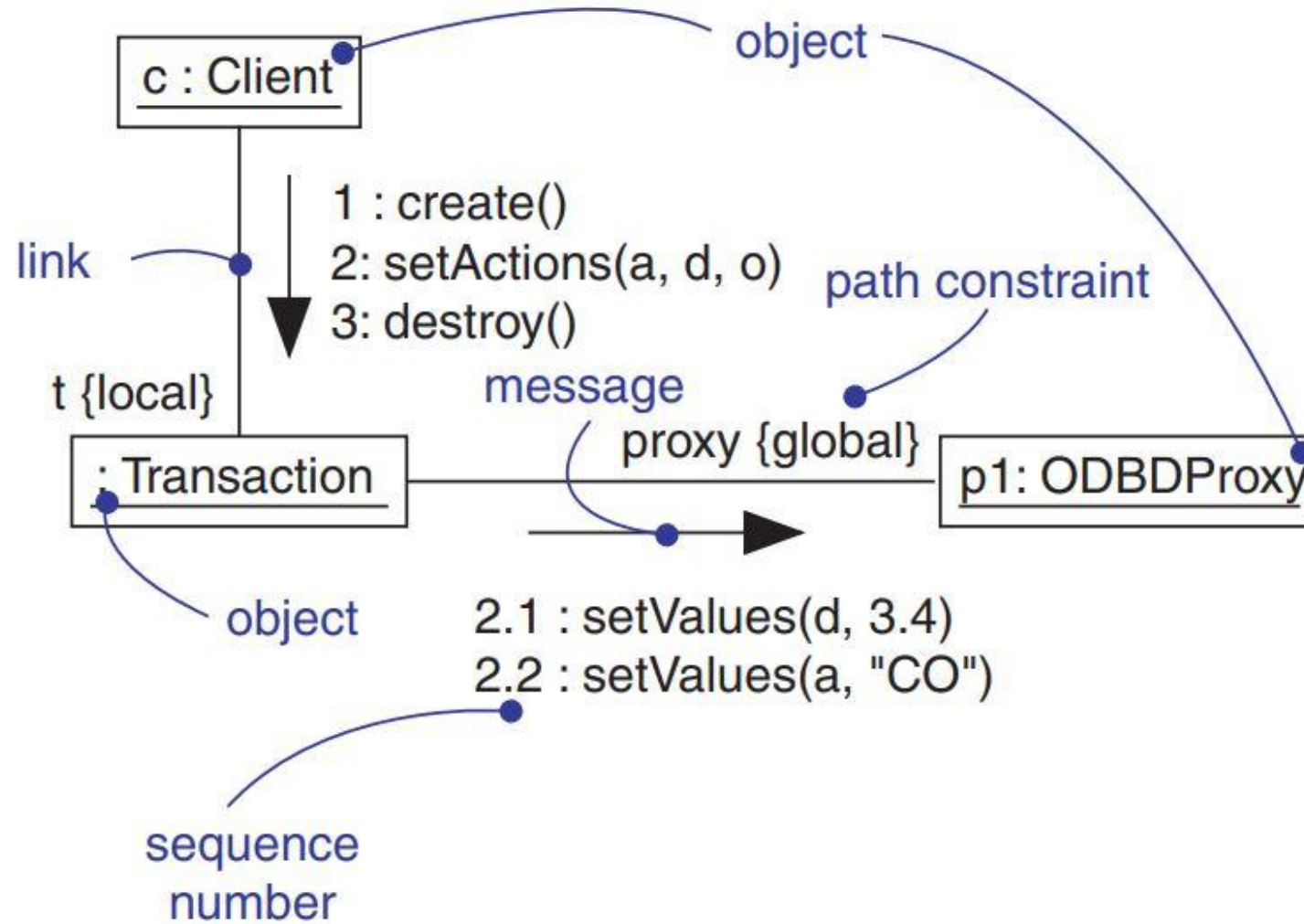


Fig: Communication Diagram

3.11 Interaction Diagrams

- Communication diagrams have two features that distinguish them from sequence diagrams.
- First, there is the path. We render a path corresponding to an association. We also render paths corresponding to local variables, parameters, global variables, and self access. A path represents a source of knowledge to an object.
- Second, there is the sequence number. To indicate the time order of a message, you prefix the message with a number (starting with the message numbered 1), increasing monotonically for each new message in the flow of control (2, 3, and so on). To show nesting, you use Dewey decimal numbering (1 is the first message, which contains message 1.1 and message 1.2, and so on). You can show nesting to an arbitrary depth. Note also that, along the same link, you can show many messages (possibly being sent from different directions), and each will have a unique sequence number.

3.11 Interaction Diagrams

Semantic Equivalence

- Because they both derive from the same information in the UML's metamodel, sequence diagrams and communication diagrams are semantically equivalent.
- As a result, you can take a diagram in one form and convert it to the other without any loss of information, as you can see in the previous two figures, which are semantically equivalent.
- However, this does not mean that both diagrams will explicitly visualize the same information.
- For example, in the previous two figures, the communication diagram shows how the objects are linked (note the {local} and {global} annotations); the corresponding sequence diagram does not.
- Similarly, the sequence diagram shows message return (note the return value committed), but the corresponding communication diagram does not.

3.11 Interaction Diagrams

- In both cases, the two diagrams share the same underlying model, but each may render some things the other does not.
- However, a model entered in one format may lack some of the information shown on the other format, so although the underlying model can include both kinds of information, the two kinds of diagrams may lead to different models.

3.13 Component & Component Diagrams

- **A component** is a logical, replaceable part of a system that conforms to and provides the realization of a set of interfaces.
- Good components define crisp abstractions with well-defined interfaces, making it possible to easily replace older components with newer, compatible ones.
- Interfaces bridge your logical and design models.
- For example, you may specify an interface for a class in a logical model, and that same interface will carry over to some design component that realizes it.
- Interfaces allow you to build the implementation of a component using smaller components by wiring ports on the components together.
- **A port** is a specific window into an encapsulated component accepting messages to and from the component conforming to specified interfaces.

3.13 Component & Component Diagrams

- **Internal structure** is the implementation of a component by means of a set of parts that are connected together in a specific way
- **A part** is the specification of a role that composes part of the implementation of a component. In an instance of the component, there is an instance corresponding to the part.
- **A connector** is a communication relationship between two parts or ports within the context of a component.
- The relationship between component and interface is important. All the most common component-based operating system facilities (such as COM+, CORBA, and Enterprise Java Beans) use interfaces as the glue that binds components together.

3.13 Component & Component Diagrams

- To construct a system based on components, you decompose your system by specifying interfaces that represent the major seams in the system. You then provide components that realize the interfaces, along with other components that access the services through their interfaces. This mechanism permits you to deploy a system whose services are somewhat location-independent and replaceable.
- As in Figure in next slide indicates, a component is shown as a rectangle with a small two-pronged icon in its upper right corner. The name of the component appears in the rectangle. A component can have attributes and operations, but these are often elided in diagrams. A component can show a network of internal structure.

3.13 Component & Component Diagrams

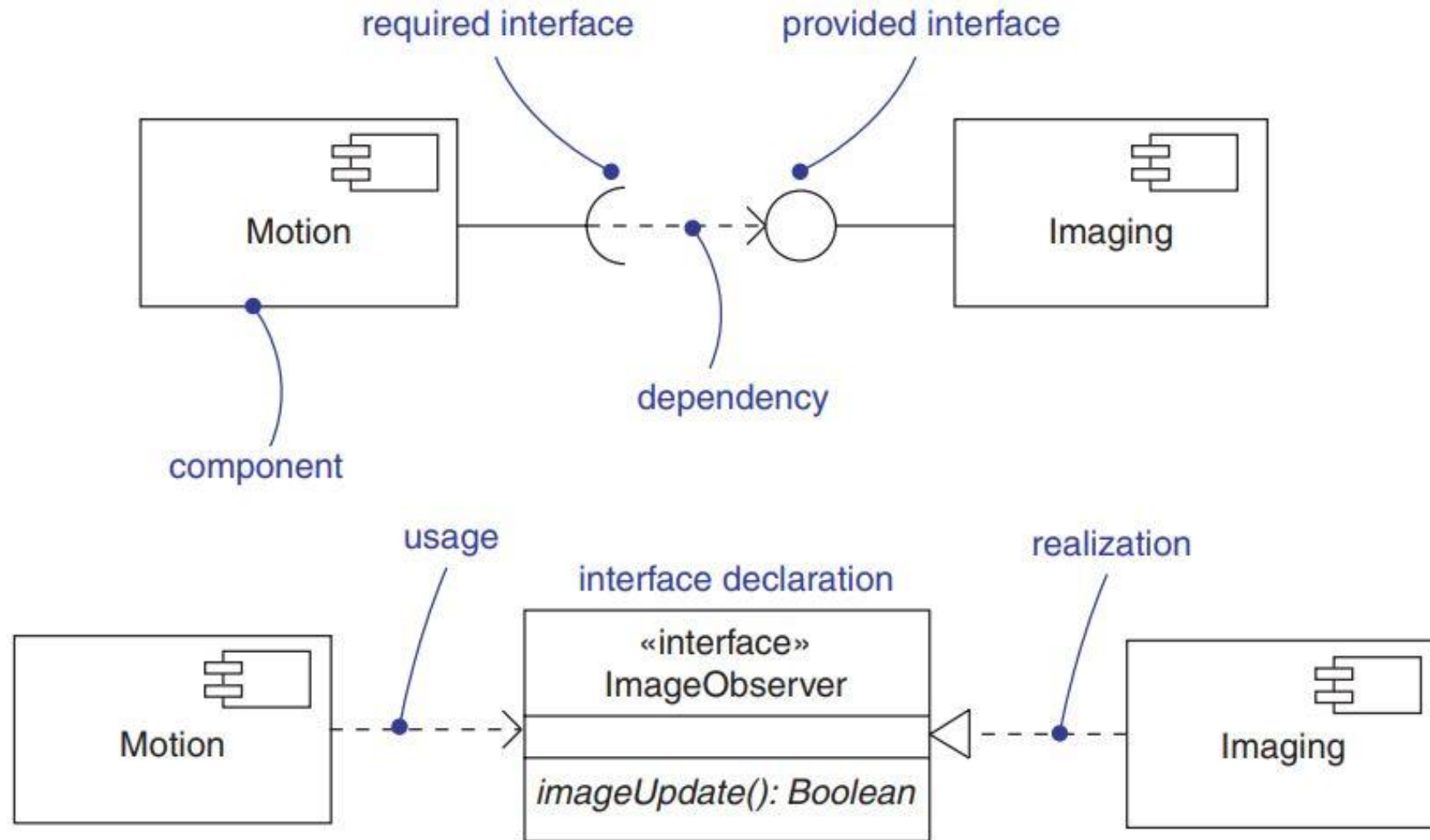


Fig: Components & Interfaces

3.13 Component & Component Diagrams

- We can show the relationship between a component and its interfaces in one of two ways:
- The first (and most common) style renders the interface in its elided, iconic form.
- A provided interface is shown as a circle attached to the component by a line (a “lollipop”).
- A required interface is shown as a semicircle attached to the component by a line (a “socket”).
- In both cases, the name of the interface is placed next to the symbol.

3.13 Component & Component Diagrams

- The second style renders the interface in its expanded form, perhaps revealing its operations.
- The component that realizes the interface is connected to the interface using a full realization relationship.
- The component that accesses the services of the other component through the interface is connected to the interface using a dependency relationship.
- A given interface may be provided by one component and required by another.
- The fact that this interface lies between the two components breaks the direct dependency between the components.
- A component that uses a given interface will function properly no matter what component realizes that interface.
- Of course, a component can be used in a context if and only if all its required interfaces are realized as provided interfaces of other components.

3.13 Component & Component Diagrams

Replaceability:

- A component is replaceable.
- A component is substitutable—it is possible to replace a component with another that conforms to the same interfaces.
- At design time, you choose a different component.
- A component is part of a system.
- A component conforms to and provides the realization of a set of interfaces.
- We can organize components by grouping them in packages in the same manner in which you organize classes.
- We can also organize components by specifying dependency, generalization, association (including aggregation), and realization relationships among them. Components can be built from other components.

3.13 Component & Component Diagrams

Ports

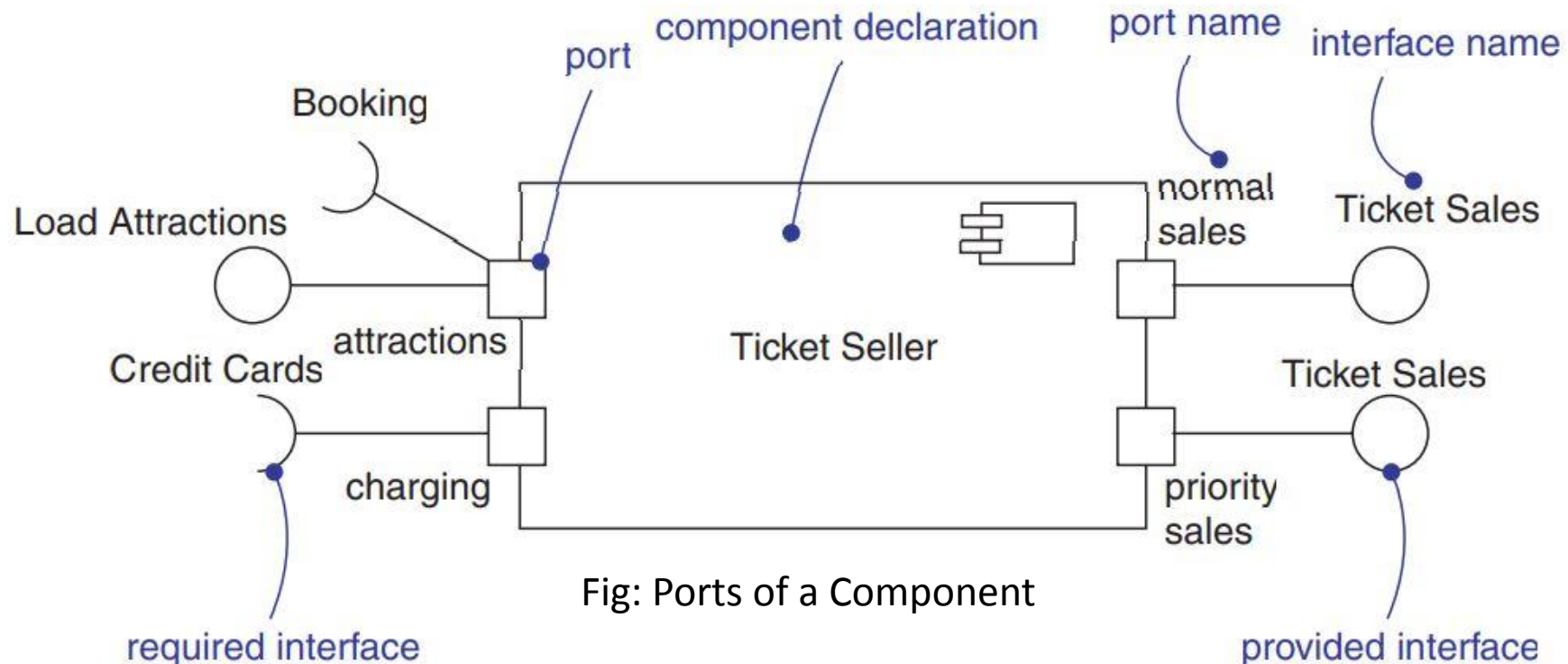
- A port is an explicit window into an encapsulated component.
- In an encapsulated component, all of the interactions into and out of the component pass through ports.
- The externally visible behavior of the component is the sum of its ports, no more and no less.
- In addition, a port has identity.
- Another component can communicate with the component through a specific port.
- The communications are described completely by the interfaces that the port supports, even if the component supports other interfaces.
- In the implementation, the internal parts of the component may interact through a specific external port, so that each part can be independent of the requirements of the other parts.

3.13 Component & Component Diagrams

- Ports permit the interfaces of a component to be divided into discrete packets and used independently.
- The encapsulation and independence provided by ports permit a much greater degree of encapsulation and substitutability.
- A port is shown as a small square straddling the border of a component—it represents a hole through the encapsulation boundary of the component.
- Both provided and required interfaces may be attached to the port symbol.
- Each port has a name so that it can be uniquely identified given the component and the port name.
- The port name can be used by internal parts of the component to identify the port through which to send and receive messages.
- The component name and port name together uniquely identify a specific port in a specific component for use by other components.

3.13 Component & Component Diagrams

- Instances of ports are created and destroyed along with the instance of the component to which they belong.
- Ports may also have multiplicity; this indicates the possible number of instances of a particular port within an instance of the component.



3.13 Component & Component Diagrams

- Figure in previous slide shows the model of a Ticket Seller component with ports. Each port has a name and, optionally, a type to tell what kind of a port it is. The component has ports for ticket sales, attractions, and credit card charging.
- There are two ports for ticket sales, one for normal customers and one for priority customers. They both have the same provided interface of type Ticket Sales. The credit card processing port has a required interface; any component that provides the specified services can satisfy it. The attractions port has both provided and required interfaces. Using the Load Attractions interface, a theater can enter shows and other attractions into the ticket database for sale. Using the Booking interface, the ticket seller component can query the theaters for the availability of tickets and actually buy the tickets.

3.13 Component & Component Diagrams

Internal Structure

- The internal structure of a component is the parts that compose the implementation of the component together with the connections among them.
- In many cases, the internal parts can be instances of smaller components that are wired together statically through ports to provide the necessary behavior without the need for the modeler to specify extra logic.
- A part is a unit of the implementation of a component. A part has a name and a type. In an instance of the component, there is one or more instance corresponding to each part having the type specified by the part. A part has a multiplicity within its component. If the multiplicity of the part is greater than one, there may be more than one part instance in a given component instance. If the multiplicity is something other than a single integer, the number of part instances may vary from one instance of the component to another. A component instance is created with the minimum number of parts; additional parts can be added later.

3.13 Component & Component Diagrams

- An attribute of a class is a kind of part: it has a type and a multiplicity, and each instance of the class has one or more instance of the given type.

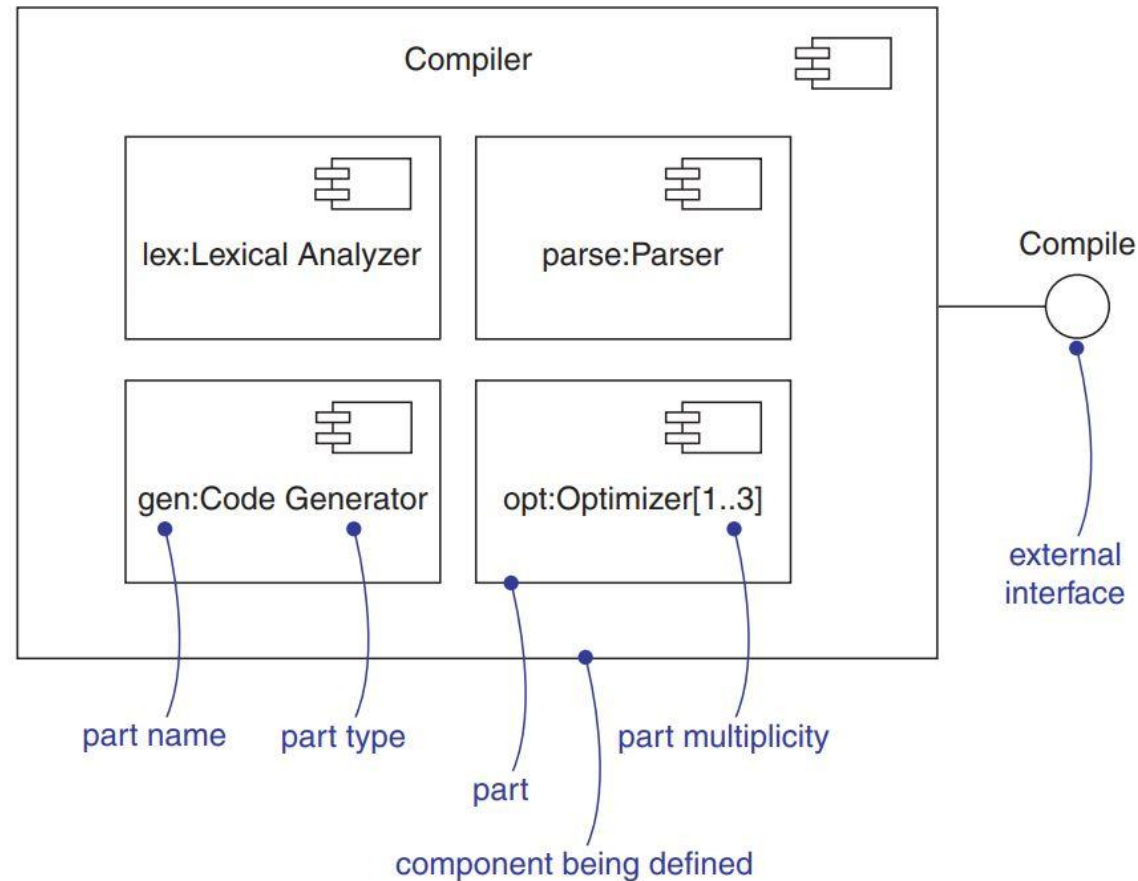


Fig: Parts of a component

3.13 Component & Component Diagrams

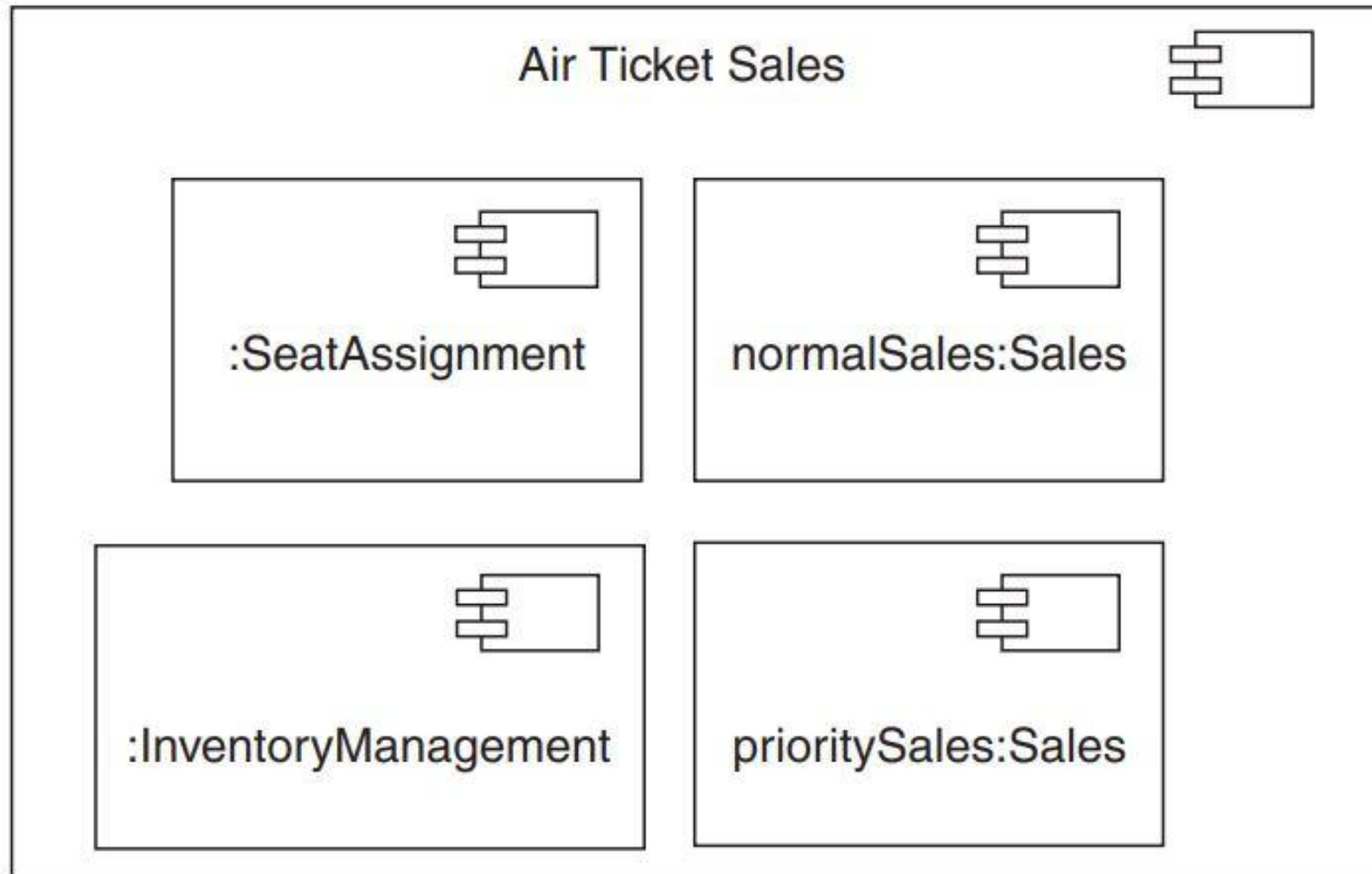


Fig: Parts of the same type

3.13 Component & Component Diagrams

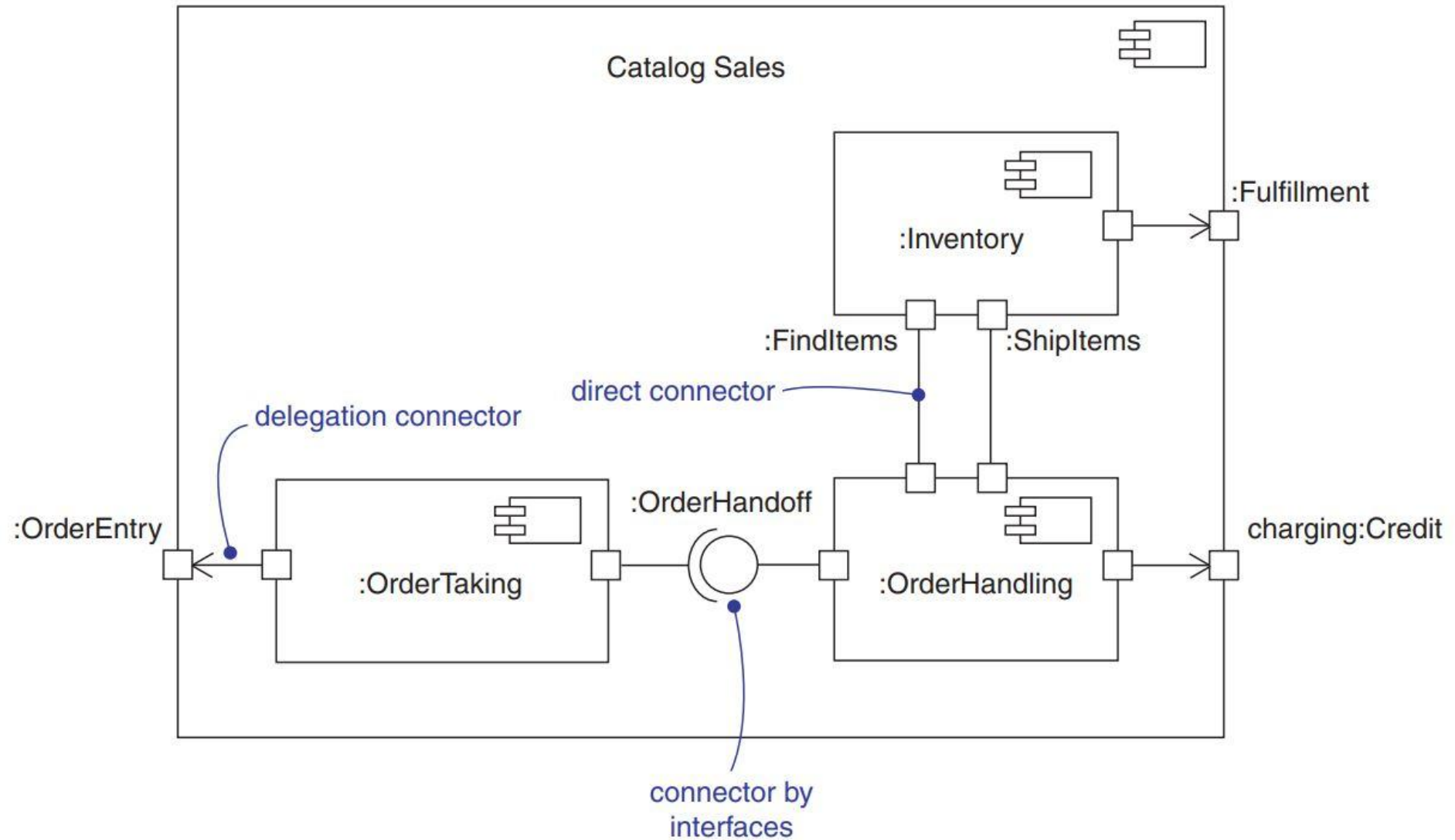


Fig: Connectors

3.14 Deployment Diagrams

- Deployment diagrams are one of the diagrams used in modeling the physical aspects of an object-oriented system.
- A deployment diagram shows the configuration of run time processing nodes and the artifacts that live on them.
- We use deployment diagrams to model the static deployment view of a system.
- For the most part, this involves modeling the topology of the hardware on which your system executes.
- Deployment diagrams are essentially class diagrams that focus on a system's nodes.
- Deployment diagrams are not only important for visualizing, specifying, and documenting embedded, client/server, and distributed systems, but also for managing executable systems through forward and reverse engineering.

3.14 Deployment Diagrams

- Deployment diagrams commonly contain
 - Nodes
 - Dependency and association relationships
- Like all other diagrams, deployment diagrams may contain notes and constraints.
- Deployment diagrams may also contain artifacts, each of which must live on some node.
- Deployment diagrams may also contain packages or subsystems, both of which are used to group elements of your model into larger chunks.
- Sometimes, you'll want to place instances in your deployment diagrams, as well, especially when you want to visualize one instance of a family of hardware topologies

3.14 Deployment Diagrams

- A node is a physical element that exists at run time and represents a computational resource, generally having at least some memory and, often, processing capability.
- A node typically represents a processor or a device on which artifacts may be deployed.
- The artifacts you develop or reuse as part of a software-intensive system must be deployed on some set of hardware in order to execute. This is in effect what a software-intensive system is all about—such a system encompasses both software and hardware.
- When you architect a software-intensive system, you have to consider both its logical and physical dimensions.
- On the logical side, you'll find things such as classes, interfaces, collaborations, interactions, and state machines.

3.14 Deployment Diagrams

- On the physical side, you'll find artifacts (which represent the physical packaging of these logical things) and nodes (which represent the hardware on which these artifacts are deployed and execute).
- The UML provides a graphical representation of node, as in next slide shows. This canonical notation permits you to visualize a node apart from any specific hardware.
- Using stereotypes—one of the UML's extensibility mechanisms—you can (and often will) tailor this notation to represent specific kinds of processors and devices.

3.14 Deployment Diagrams

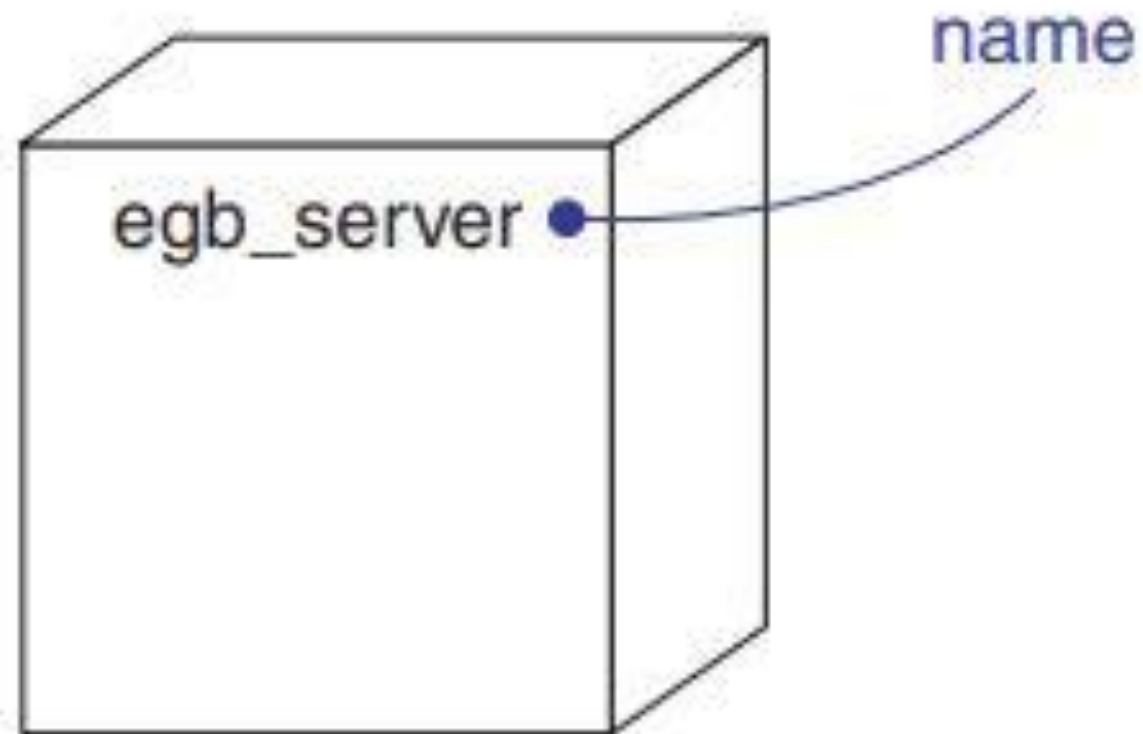


Fig: Node

3.14 Deployment Diagrams

- Every node must have a name that distinguishes it from other nodes.
- A name is a textual string.
- That name alone is known as a simple name; a qualified name is the node name prefixed by the name of the package in which that node lives.
- A node is typically drawn showing only its name, as in Figure in next slide.
- Just as with classes, you may draw nodes adorned with tagged values or with additional compartments to expose their details.

3.14 Deployment Diagrams

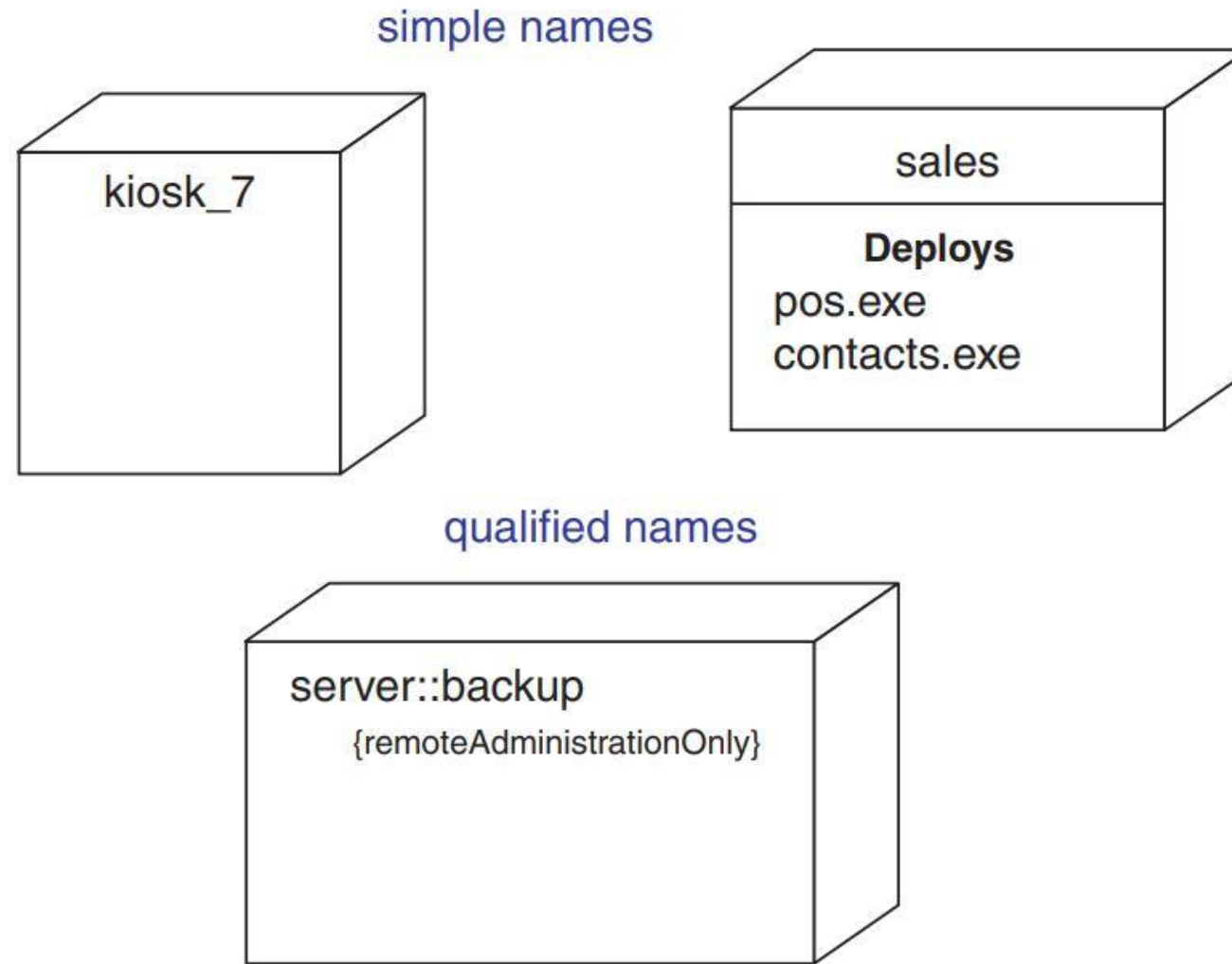


Fig: Nodes with Simple & Qualified Names

3.14 Deployment Diagrams

Nodes and Artifacts

- In many ways, nodes are a lot like artifacts: Both have names; both may participate in dependency, generalization, and association relationships; both may be nested; both may have instances; both may be participants in interactions.
- However, there are some significant differences between nodes and artifacts.
 - Artifacts are things that participate in the execution of a system; nodes are things that execute artifacts.
 - Artifacts represent the physical packaging of otherwise logical elements; nodes represent the physical deployment of artifacts.
- This first difference is the most important. Simply put, nodes execute artifacts; artifacts are things that are executed by nodes.
- The second difference suggests a relationship among classes, artifacts, and nodes.

3.14 Deployment Diagrams

- In particular, an artifact is the manifestation of a set of logical elements, such as classes and collaborations, and a node is the location upon which artifacts are deployed.
- A class may be manifested by one or more artifacts, and, in turn, an artifact may be deployed on one or more nodes.
- As Figure in next slide shows, the relationship between a node and the artifacts it deploys can be shown explicitly by using nesting.
- Most of the time, you won't need to visualize these relationships graphically but will indicate them as a part of the node's specification, for example, using a table.
- A set of objects or artifacts that are allocated to a node as a group is called a *distribution unit*.
- Nodes are also class-like in that you can specify attributes and operations for them. For example, you might specify that a node provides the attributes processorSpeed and memory, as well as the operations turnOn, turnOff, and suspend.

3.14 Deployment Diagrams

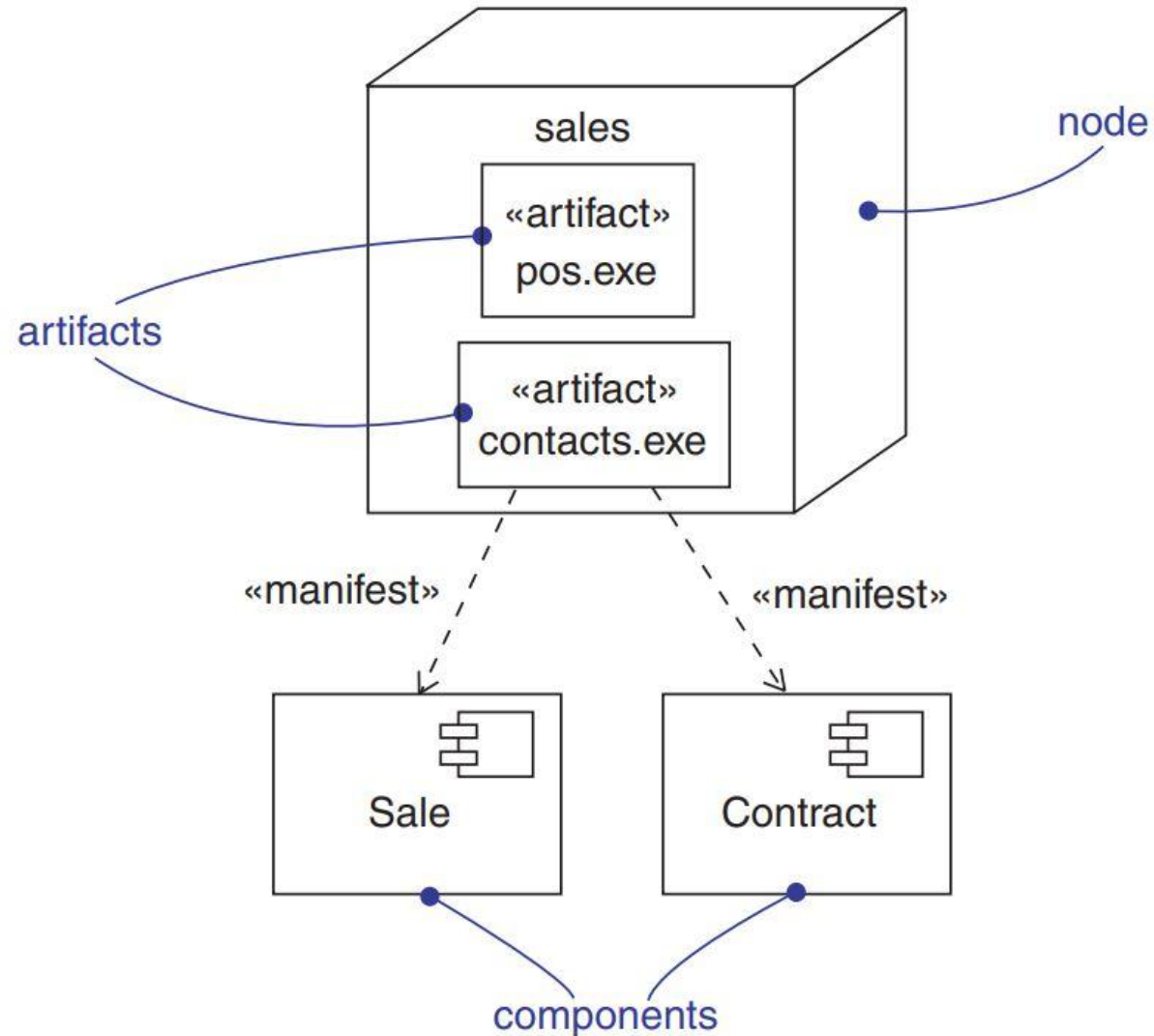
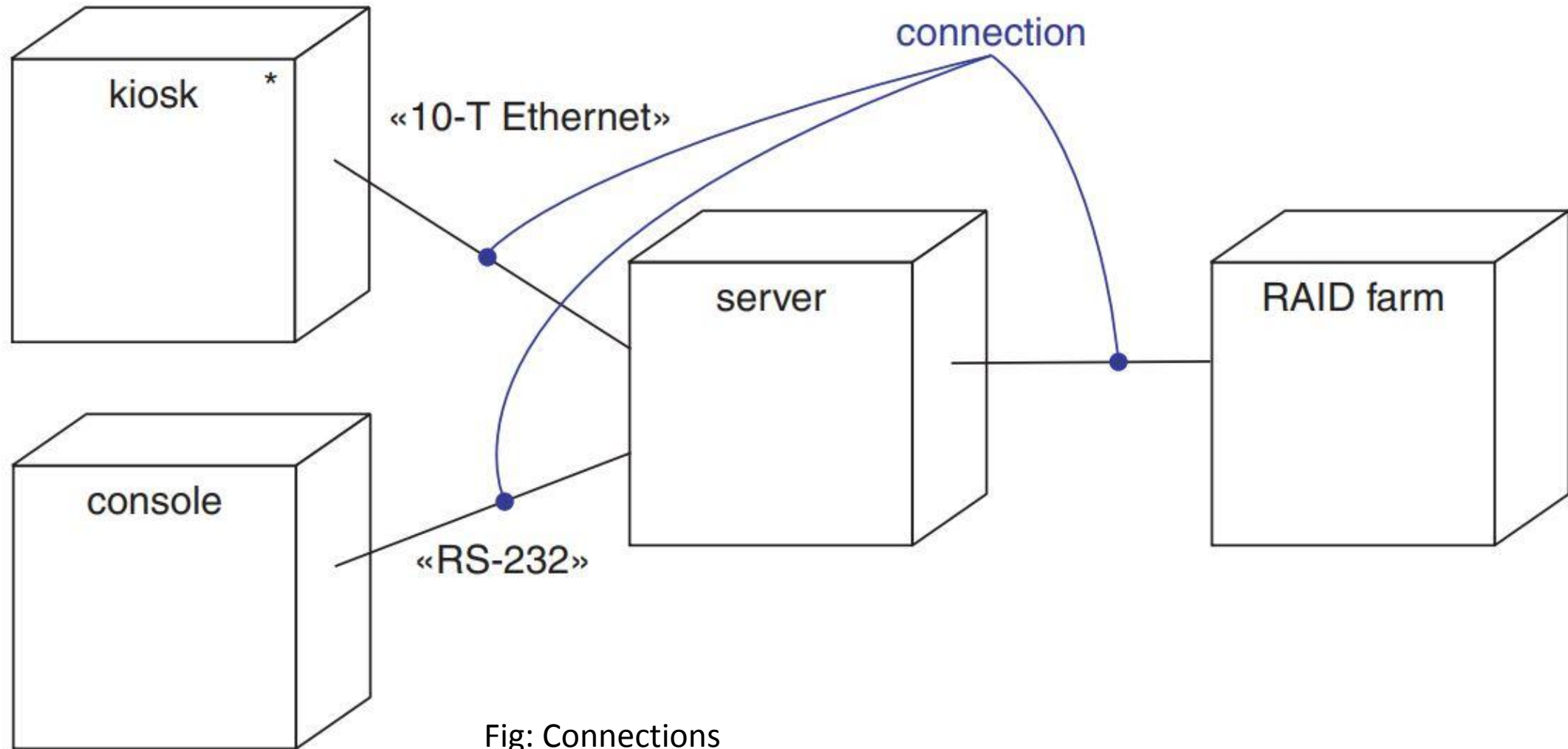


Fig: Nodes & Artifacts

3.14 Deployment Diagrams

- We can organize nodes by grouping them in packages in the same manner in which you can organize classes and artifacts.
- We can also organize nodes by specifying dependency, generalization, and association (including aggregation) relationships among them.
- The most common kind of relationship you'll use among nodes is an association.
- In this context, an association represents a physical connection among nodes, such as an Ethernet connection, a serial line, or a shared bus, as in the next slide shows.
- We can even use associations to model indirect connections, such as a satellite link between distant process.

3.14 Deployment Diagrams



3.14 Deployment Diagrams

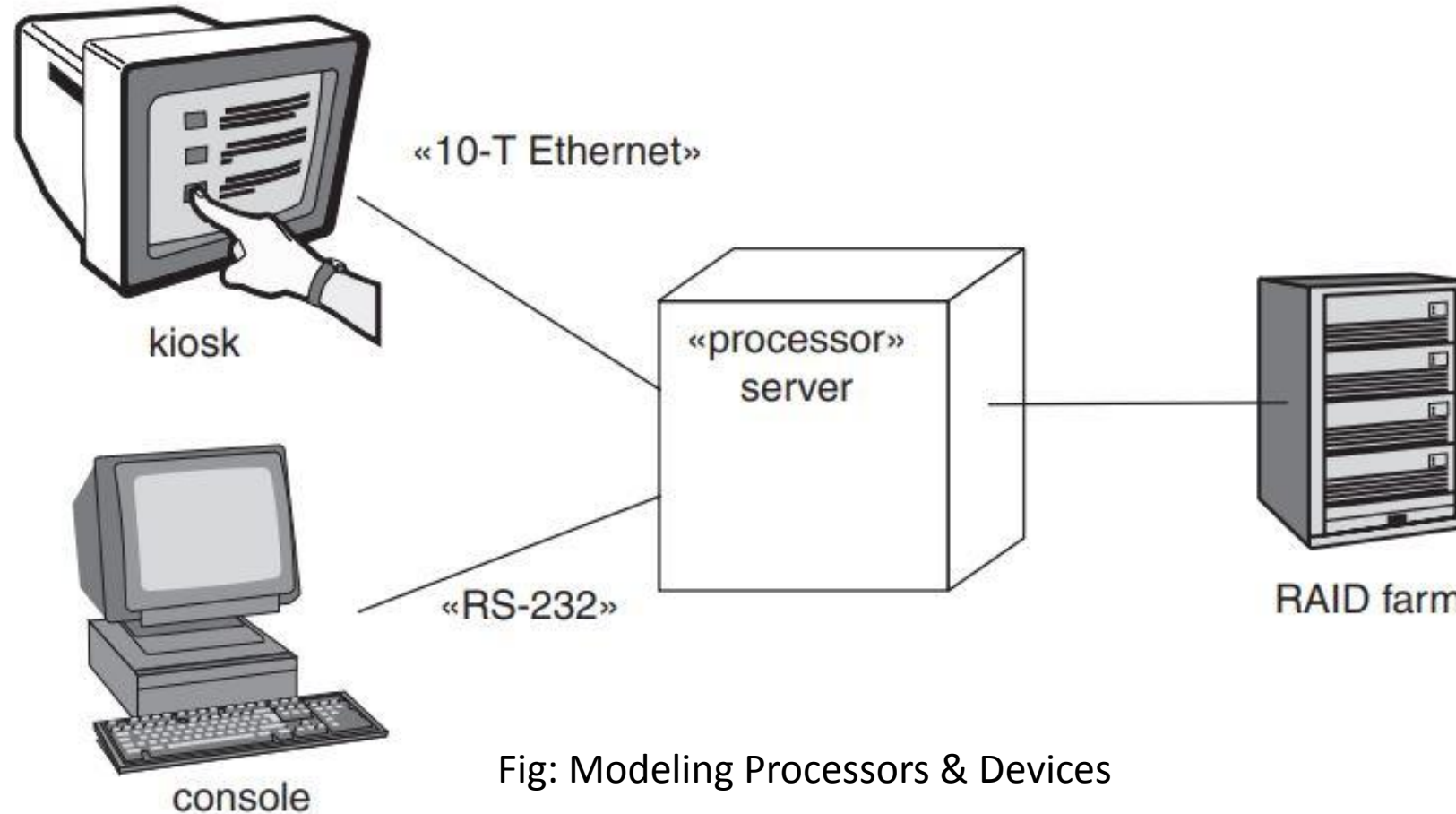


Fig: Modeling Processors & Devices

3.14 Deployment Diagrams

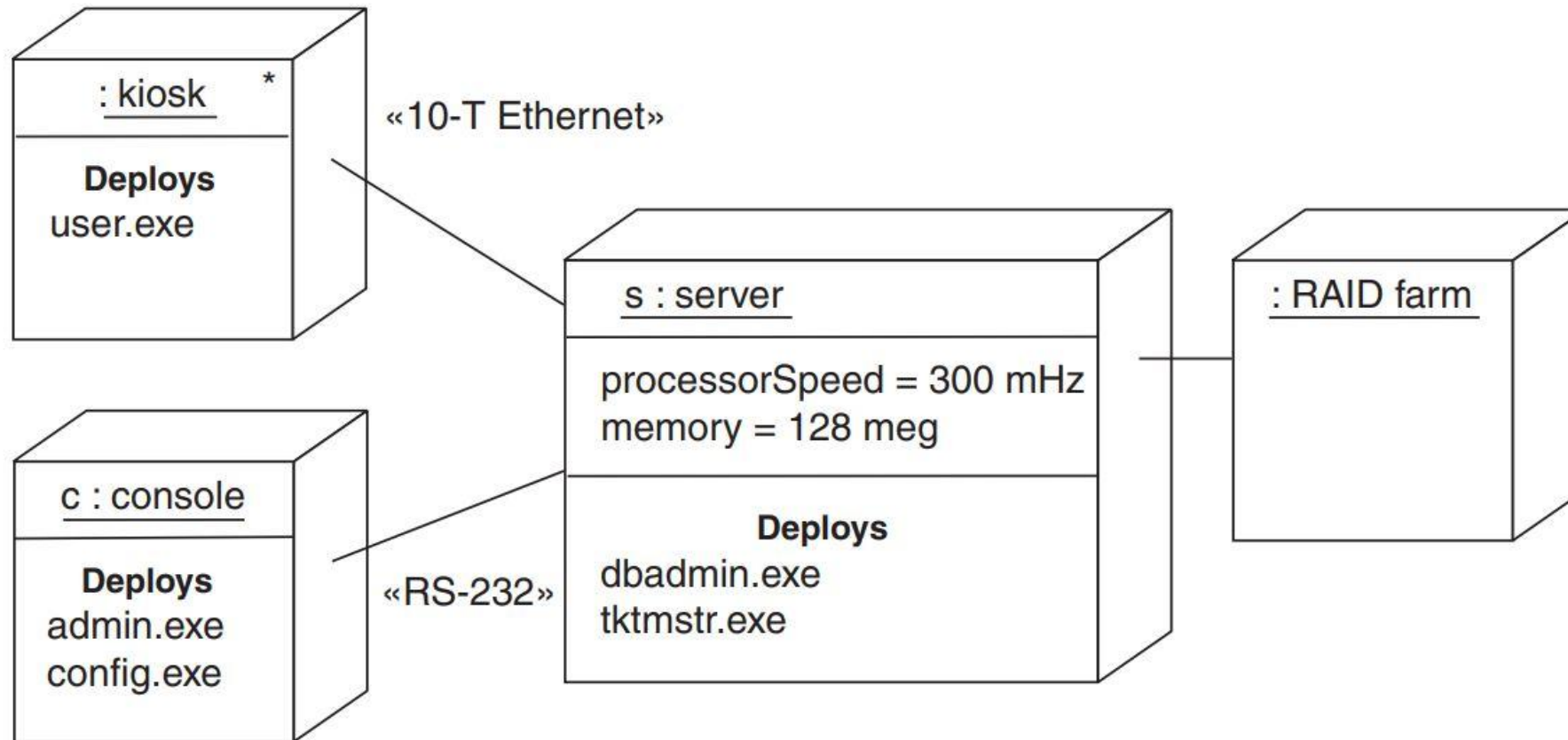


Fig: Modeling the distribution of Artifacts

3.14 Deployment Diagrams

- We will typically use Deployment diagram in one of the three ways:
 1. To model embedded systems
 2. To model client/server systems
 3. To model fully distributed systems

3.14 Deployment Diagrams

1. To model embedded systems

- An embedded system is a software-intensive collection of hardware that interfaces with the physical world.
- Embedded systems involve software that controls devices such as motors, actuators, and displays and that, in turn, is controlled by external stimuli such as sensor input, movement, and temperature changes.
- We can use deployment diagrams to model the devices and processors that comprise an embedded system.

3.14 Deployment Diagrams

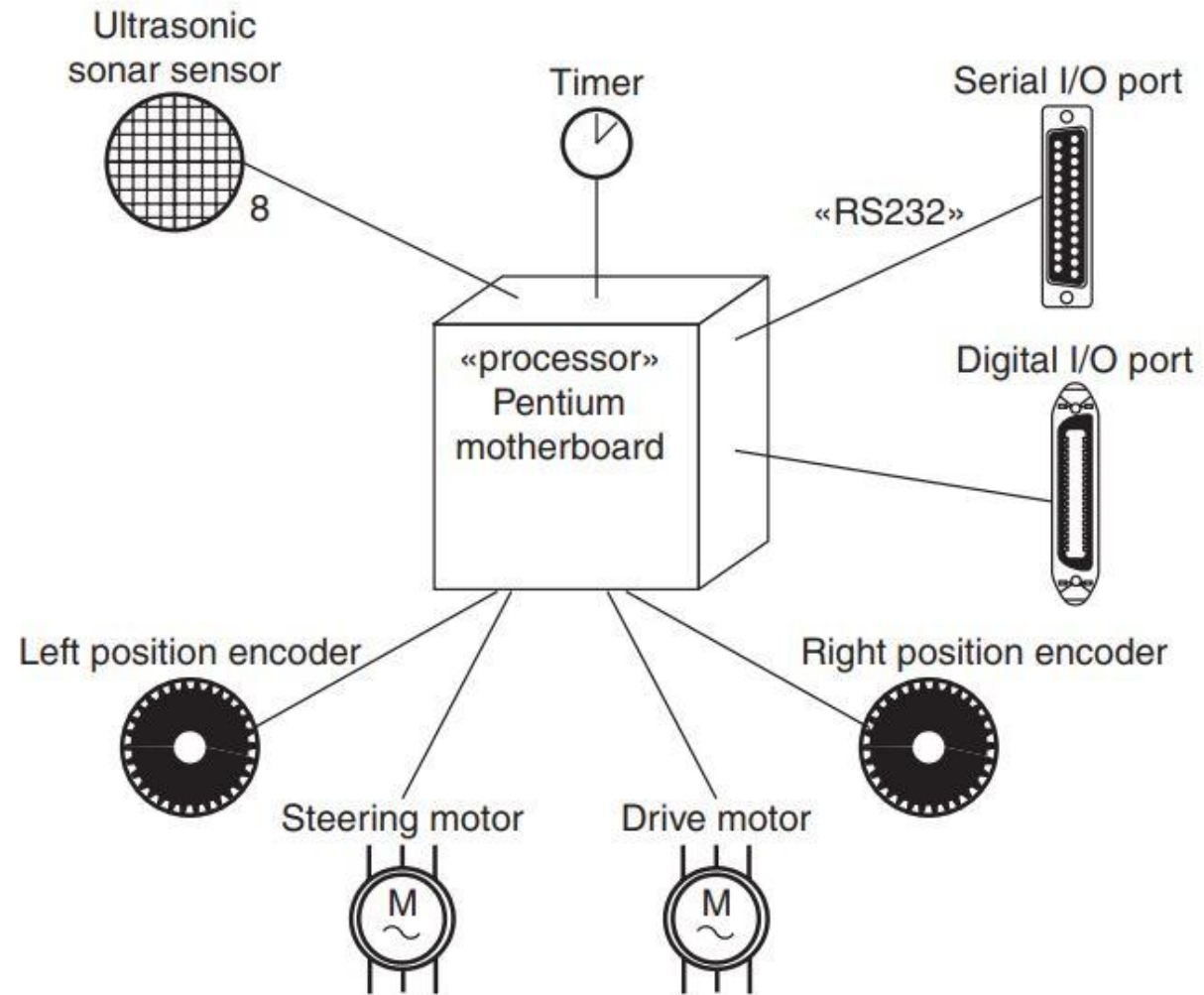


Fig: Modeling an Embedded System

3.14 Deployment Diagrams

2. To model client/server systems

- A client/server system is a common architecture focused on making a clear separation of concerns between the system's user interface (which lives on the client) and the system's persistent data (which lives on the server).
- Client/ server systems are one end of the continuum of distributed systems and require you to make decisions about the network connectivity of clients to servers and about the physical distribution of your system's software artifacts across the nodes.
- We can model the topology of such systems by using deployment diagrams.

3.14 Deployment Diagrams

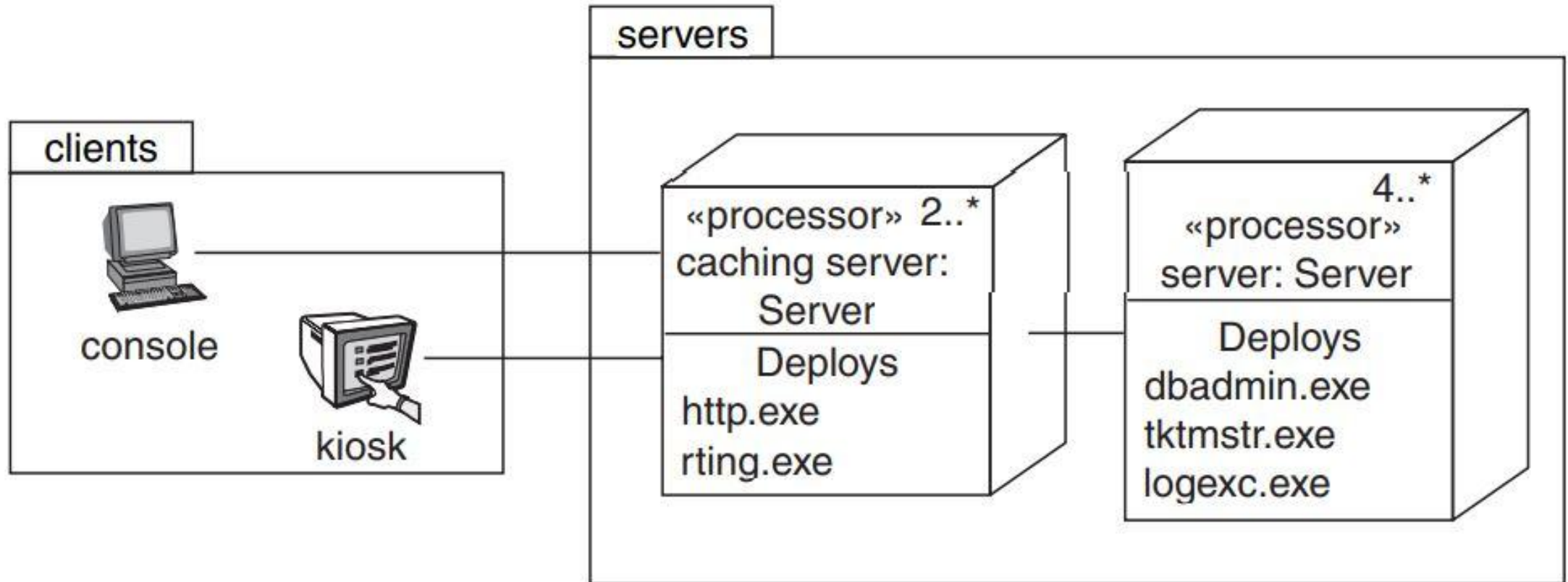


Fig: Modeling of a Client Server System

3.14 Deployment Diagrams

3. To model fully distributed systems

- At the other end of the continuum of distributed systems are those that are widely, if not globally, distributed, typically encompassing multiple levels of servers.
- Such systems are often hosts to multiple versions of software artifacts, some of which may even migrate from node to node.
- Crafting such systems requires you to make decisions that enable the continuous change in the system's topology.
- We can use deployment diagrams to visualize the system's current topology and distribution of artifacts to reason about the impact of changes on that topology.

3.14 Deployment Diagrams

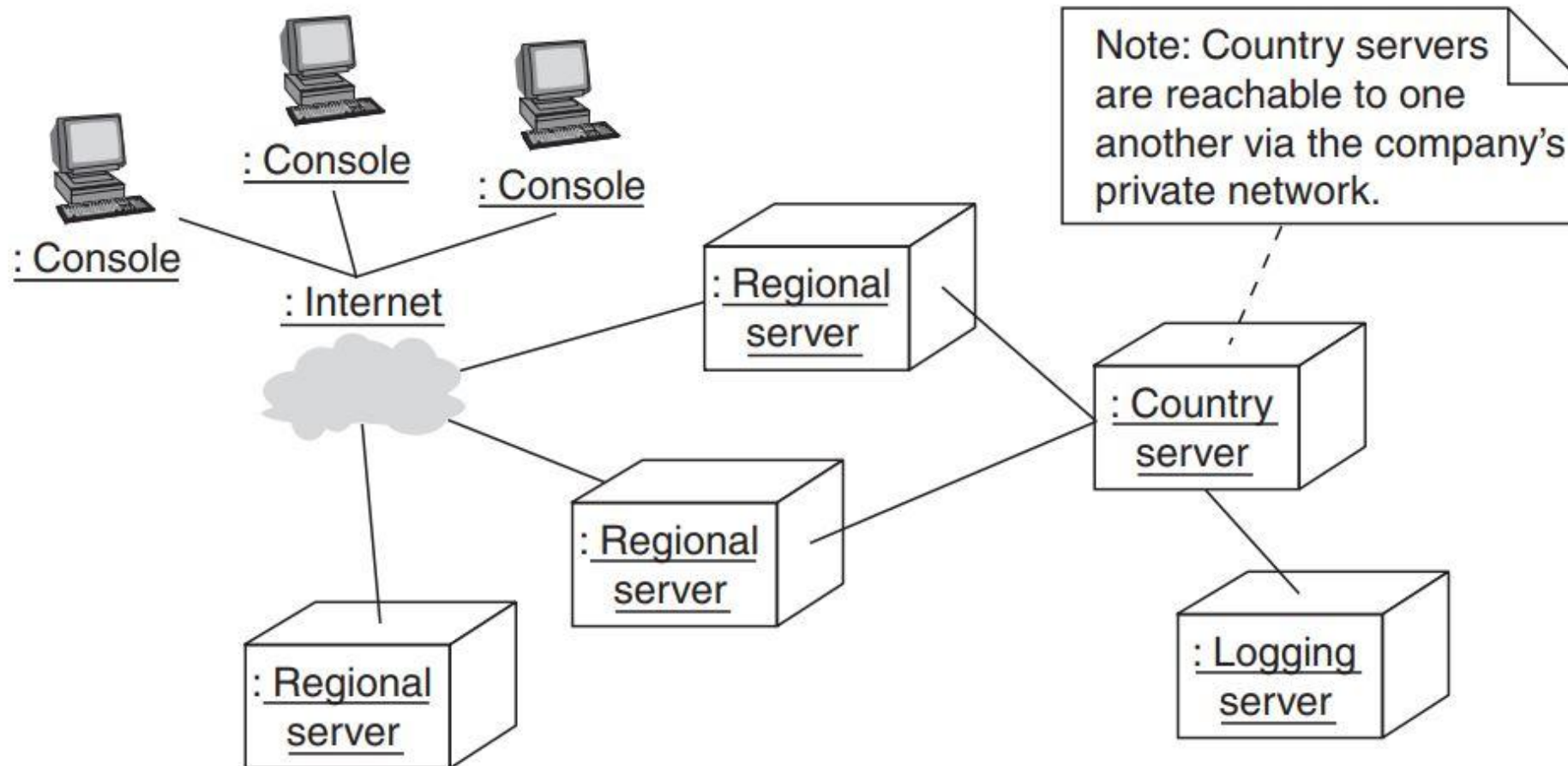


Fig: Modeling of a Distributed System

Any Queries???
Thankyou!!!